

算法设计与分析课程笔记

根据 org/*.pdf 课程课件整理

2026 年 6 月 20 日

目录

使用说明	ix
第一章 课程导论与基础算法	1
1.1 课程视角：算法不只是程序	1
1.2 为什么要学算法设计？	1
1.3 排序与分治	1
1.3.1 归并排序	1
1.3.2 Master Theorem	2
1.4 图搜索：BFS	3
1.5 带正权最短路：Dijkstra	3
第二章 深度优先搜索与强连通分量	5
2.1 为什么需要 DFS？	5
2.2 DFS 的时间戳与边分类	5
2.3 DAG 与拓扑排序	6
2.4 强连通分量	6
2.5 Kosaraju 线性算法	7
第三章 最小生成树	8
3.1 动机	8
3.2 生成树与 MST	8
3.3 割性质与环性质	8
3.4 Kruskal 算法完整运行例子	9
3.5 Prim 算法完整运行例子	9
3.6 Borůvka 简介	9
第四章 算法验证基础	11
4.1 动机——为什么测试不够	11
4.2 形式化验证的三条路线	11
4.3 Kripke 结构	11
4.4 时序逻辑 CTL	12
4.5 CTL 模型检测算法	12
4.6 扩展模型简介	13

第五章 序列比对与动态规划	14
5.1 动机	14
5.2 动态规划的核心思想	14
5.3 编辑距离	14
5.4 最长公共子序列 LCS	15
5.5 Hirschberg 算法	16
第六章 负权最短路	17
6.1 动机——Dijkstra 的局限	17
6.2 Bellman-Ford 算法	17
6.3 负环检测	18
6.4 Floyd-Warshall	18
第七章 树宽	21
7.1 动机	21
7.2 树上独立集 DP	21
7.3 树分解	22
7.4 具体图的树分解例子	22
7.5 树宽的性质	23
7.6 树分解上的 DP	23
第八章 最大流与最小割	25
8.1 流网络	25
8.2 割	25
8.3 残量网络	26
8.4 Ford-Fulkerson 算法	27
8.5 最大流最小割定理	27
第九章 Ford-Fulkerson 的扩展	28
9.1 容量缩放算法	28
9.2 Edmonds-Karp 算法	28
9.3 Dinic 算法	29
第十章 单位容量网络与匹配	31
10.1 二分匹配的流建模	31
10.2 Hall 婚配定理	32
10.3 单位容量网络	32
10.4 边不相交路径与 Menger 定理	32
第十一章 线性规划导论	34
11.1 线性规划模型	34
11.2 几何直觉：可行域与最优顶点	34
11.3 建模方法论	35
11.4 对偶	35

11.5 互补松弛	36
第十二章 单纯形算法	37
12.1 标准型与松弛变量	37
12.2 字典表示与单纯形 Pivot	37
12.3 单纯形算法完整描述	38
12.4 初始顶点: Phase I	39
12.5 退化与 Bland 规则	39
12.6 复杂度	39
第十三章 线性规划的应用	40
13.1 最短路的 LP 建模	40
13.2 最大流的 LP 与对偶	40
13.3 整数规划与 LP 松弛	41
13.4 集合覆盖的 ILP、LP 松弛与对偶	41
第十五章 多项式归约与经典困难问题	43
15.1 多项式时间归约	43
15.2 覆盖与打包: Independent Set 等价于 Vertex Cover	44
15.2.1 直觉	44
15.2.2 5 节点图的具体例子	44
15.3 3SAT 归约到 Independent Set	44
15.3.1 直觉	44
15.3.2 构造	45
15.3.3 完整例子: 3 个子句	45
15.4 Vertex Cover 归约到 Set Cover	46
15.5 SAT 归约到 3SAT	46
15.6 Hamilton Cycle 归约到 TSP	46
15.7 Subset Sum 归约到 Knapsack	46
第十六章 复杂性类	47
16.1 P 类	47
16.2 NP 类	47
16.3 NP-complete	48
16.3.1 Cook-Levin 定理	48
16.3.2 证明 NP-complete 的标准模板	48
16.4 co-NP	49
16.5 NP-hard	49
16.6 复杂性类的包含关系	49
第十七章 超越 NP: PSPACE	50
17.1 PSPACE	50
17.2 量化布尔公式 QBF (PSPACE 完全问题)	50
17.2.1 直觉: SAT 加上“博弈量词”	50

17.2.2 小例子: 2 变量 QBF	51
17.3 Geography 游戏 (PSPACE 完全问题)	51
17.3.1 规则	51
17.3.2 4 节点例子: 完整博弈树	51
第十八章 DPLL 与 SAT 求解	53
18.1 Tseitin 编码	53
18.1.1 直觉: "给每个中间结果起个名字"	53
18.1.2 例子	53
18.2 DPLL 基本框架	53
18.2.1 直觉: "带剪枝的暴力搜索"	53
18.2.2 完整例子: 3 变量 4 子句	54
18.3 CDCL: 从错误中学习	54
18.3.1 直觉	54
18.3.2 蕴含图 (Implication Graph)	55
18.3.3 解析规则	55
18.4 启发式	55
第十九章 近似算法导论	56
19.1 近似比 (Approximation Ratio)	56
19.2 Vertex Cover 2-近似	56
19.2.1 算法	56
19.2.2 6 节点图上的完整运行	57
19.3 Set Cover 贪心算法	57
19.3.1 算法	57
19.3.2 6 元素 4 集合的完整例子	57
19.3.3 H_n 近似比分析	58
第二十章 Steiner Tree 与 TSP	59
20.1 Metric Steiner Tree	59
20.2 Metric TSP: 为何一般 TSP 不可近似	59
20.3 Metric TSP 的 2-近似 (MST 双倍算法)	62
20.3.1 5 节点完全图例子	62
20.4 Christofides 3/2-近似	63
20.4.1 直觉	63
20.4.2 继续上面的 5 节点例子	63
第二十一章 多项式时间近似方案	64
21.1 PTAS 与 FPTAS	64
21.2 输入规模、伪多项式与为什么要缩放	64
21.3 Knapsack: 问题与贪心反例	65
21.4 Knapsack 的伪多项式 DP	65
21.4.1 课件中的 5 物品例子	65

21.5 Knapsack 的 FPTAS	66
21.5.1 FPTAS 的直觉	67
21.5.2 缩放因子的选择	67
21.5.3 缩放后的按价值 DP	68
21.5.4 时间复杂度分析	68
21.5.5 误差分析	69
21.5.6 为什么这是 FPTAS	71
21.5.7 算法总结	71
21.5.8 上面例子上的缩放	72
21.5.9 一个便于手算的粗略缩放例子	73
21.6 Bin Packing	74
21.6.1 First Fit 的完整例子	75
21.6.2 Bin Packing 的不可近似性	75
21.6.3 APTAS: 渐近近似方案	79
第二十二章 基于线性规划的近似	85
22.1 LP 舍入 (LP Rounding)	85
22.1.1 Set Cover 的简单舍入	85
22.2 随机舍入 (Randomized Rounding)	90
22.3 原始-对偶 (Primal-Dual)	91
22.3.1 直觉	91
22.3.2 一般框架与松弛互补条件	91
22.3.3 Vertex Cover 的原始-对偶算法	95
22.3.4 完整例子	99
22.3.5 Set Cover 的原始-对偶算法	100
22.4 Dual Fitting	100
22.4.1 Set Cover 的 LP 与对偶	101
22.4.2 Set Cover 的贪心算法	102
22.4.3 给元素分配价格	103
22.4.4 价格总和等于贪心算法代价	103
22.4.5 为什么价格可能不是对偶可行的	104
22.4.6 关键证明: 任意集合的价格和最多超过 H_n 倍	105
22.4.7 缩放得到对偶可行解	106
22.4.8 利用弱对偶性证明近似比	107
22.4.9 一个具体例子	107
22.4.10 Dual Fitting 与原始-对偶方法的区别	109
第二十三章 MAX-SAT	111
23.1 问题定义	111
23.2 随机算法: 二分之一近似	111
23.2.1 直觉: "抛硬币也能保证一半"	111
23.2.2 更细的长度分析	111
23.2.3 3 变量 4 子句的具体计算	112

23.3 条件期望去随机化 (本章精华)	112
23.3.1 直觉: ”逐步固定, 选更好的一侧”	112
23.3.2 完整例子: 逐步固定	113
23.4 偏置随机: Flipping Biased Coins	114
23.5 LP 舍入与组合策略	114
23.5.1 LP 建模	114
23.5.2 LP 舍入分析	115
23.5.3 组合策略	116
第二十四章 算法与数学分析详解	117
24.1 图搜索: BFS 与 DFS	117
24.1.1 BFS 的层次结构	117
24.1.2 DFS 的括号性质	117
24.2 Dijkstra 与 Bellman-Ford	118
24.2.1 Dijkstra 的贪心证明	118
24.2.2 Bellman-Ford 的 DP 解释	118
24.3 最小生成树: 安全边与交换论证	119
24.3.1 统一的贪心框架	119
24.3.2 Kruskal 的复杂度细算	119
24.3.3 Prim 与 Dijkstra 的相似和不同	119
24.4 动态规划: 从状态到递推	120
24.4.1 设计 DP 的四步	120
24.4.2 序列比对的网格图	120
24.4.3 Hirschberg 的空间分析	120
24.4.4 Knapsack FPTAS 的误差推导	121
24.5 最大流: 残量网络的含义	121
24.5.1 为什么需要反向边	121
24.5.2 流值等于割净流	121
24.5.3 Ford-Fulkerson 的终止与最优性	121
24.5.4 Edmonds-Karp 的距离单调性	122
24.6 线性规划、对偶与单纯形	122
24.6.1 弱对偶逐行推导	122
24.6.2 互补松弛的来源	122
24.6.3 单纯形 pivot 的代数形式	122
24.7 归约与复杂性证明	123
24.7.1 归约证明的双向逻辑	123
24.7.2 NP、co-NP 与证书	123
24.8 近似算法的数学分析	123
24.8.1 Vertex Cover 的匹配下界	123
24.8.2 Set Cover 贪心的 charging 证明	123
24.8.3 LP 舍入的基本不等式	124
24.8.4 MAX-SAT 的期望线性性	124

24.9 如何自己分析一个新算法	125
总复习提纲	126
24.10 算法范式索引	126
24.11 常见证明模板	126
24.12 考试前快速检查	126

使用说明

这份笔记按照目录 [org/](#) 中已有课件整理，覆盖 01.pdf–13.pdf 与 15.pdf–23.pdf。目录中未提供 14.pdf，因此本笔记也保留编号空缺。

笔记不是逐页翻译幻灯片，而是按每章主题重组为适合复习和考试使用的讲义：先给问题模型和关键定义，再给算法步骤、正确性证明线索、复杂度和常见陷阱。若同一思想在多章中反复出现，例如割与对偶、动态规划、归约、线性规划松弛，笔记会在相关章节中交叉提醒。

第一章 课程导论与基础算法

对应课件：01.pdf

主题：排序、BFS、Dijkstra、算法的含义

1.1 课程视角：算法不只是程序

算法设计与分析关注三个层面：

1. **问题建模**：将现实问题抽象为精确的数学/计算模型，明确输入、输出和正确性标准。
2. **算法设计**：给出解决问题的步骤，保证对所有合法输入都产生正确输出。
3. **算法分析**：量化算法的时间和空间消耗，给出渐进复杂度保证。

本课程涉及三种核心范式：**分治** (Divide and Conquer)、**贪心** (Greedy)、**动态规划** (Dynamic Programming)。每种范式都有其适用场景和典型问题。

1.2 为什么要学算法设计？

说明 1.1. 算法 = 解决问题的精确步骤 + 正确性保证 + 复杂度承诺。

写程序并不难，但证明程序对所有输入都正确且高效才是算法的核心挑战。考虑一个简单例子：在一本有一百万条记录的电话簿中查找某人。

- **线性扫描**：逐条比对，最坏需要 $O(n)$ 次比较。对于 $n = 10^6$ ，平均需要 5×10^5 次。
- **二分查找**：利用已排序特性，每次排除一半，只需 $O(\log n)$ 次比较。对于 $n = 10^6$ ，只需约 20 次。

这个差距在数据规模增大时会变得极为显著。算法课程教你如何系统地设计出高效算法，并证明其正确性与复杂度，而不是靠直觉猜测。

1.3 排序与分治

1.3.1 归并排序

说明 1.2. 分治的核心思想：将大问题递归地拆成规模更小的同类子问题，分别解决后再合并结果。

算法思路：将数组从中间分为两半，递归排序左半部分和右半部分，再将两个有序数组合并 (Merge) 为一个有序数组。

完整递归示例：对数组 $[5, 3, 8, 1, 4, 2, 7, 6]$ 展示分解与合并过程：

• **分解阶段：**

- 层 0: $[5, 3, 8, 1, 4, 2, 7, 6]$
- 层 1: $[5, 3, 8, 1]$ $[4, 2, 7, 6]$
- 层 2: $[5, 3]$ $[8, 1]$ $[4, 2]$ $[7, 6]$
- 层 3: $[5]$ $[3]$ $[8]$ $[1]$ $[4]$ $[2]$ $[7]$ $[6]$

• **合并阶段：**

- 层 3 $\rightarrow$ 层 2: $[3, 5]$ $[1, 8]$ $[2, 4]$ $[6, 7]$
- 层 2 $\rightarrow$ 层 1: $[1, 3, 5, 8]$ $[2, 4, 6, 7]$
- 层 1 $\rightarrow$ 层 0: $[1, 2, 3, 4, 5, 6, 7, 8]$

正确性（归纳论证）：设 $P(n)$ 为”归并排序对长度为 n 的数组能正确排序”。

- 基础情形: $n = 1$, 数组已有序, 显然正确。
- 归纳步骤: 假设 $P(k)$ 对所有 $k < n$ 成立。归并排序将数组分为两个长度为 $\lfloor n/2 \rfloor$ 和 $\lceil n/2 \rceil$ 的子数组, 由归纳假设两个子数组均被正确排序; 而 Merge 过程正确地将两个有序数组合并为一个有序数组 (可单独证明), 故 $P(n)$ 成立。

时间复杂度：设 $T(n)$ 为对长度 n 的数组排序的时间, 则有递推式:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n), \quad T(1) = \Theta(1).$$

由 Master 定理 (下节), $T(n) = \Theta(n \log n)$ 。

1.3.2 Master Theorem

说明 1.3. Master Theorem 给递归式 $T(n) = aT(n/b) + f(n)$ ($a \geq 1, b > 1$) 提供了三种情形的封闭形式解。三种情形的图像化理解: 递归树有 $\log_b n$ 层, 根部工作量 $f(n)$, 叶子数量 $n^{\log_b a}$ 。谁”更重”谁主导; 若势均力敌则每层相当, 总量多一个 $\log$ 因子。

定理 1.1 (Master Theorem). 设 $T(n) = aT(n/b) + f(n)$, 令 $E = \log_b a$ 。

1. **叶子主导：**若 $f(n) = O(n^{E-\epsilon})$ ($\epsilon > 0$), 则 $T(n) = \Theta(n^E)$ 。
2. **均衡：**若 $f(n) = \Theta(n^E)$, 则 $T(n) = \Theta(n^E \log n)$ 。
3. **根部主导：**若 $f(n) = \Omega(n^{E+\epsilon})$ ($\epsilon > 0$) 且满足正则性条件 $af(n/b) \leq cf(n)$ ($c < 1$), 则 $T(n) = \Theta(f(n))$ 。

计算例子：

1. $T(n) = 2T(n/2) + n$ (归并排序): $a = 2, b = 2, E = \log_2 2 = 1, f(n) = n = \Theta(n^1)$, 属情形 2, 故 $T(n) = \Theta(n \log n)$ 。
2. $T(n) = 4T(n/2) + n$: $a = 4, b = 2, E = \log_2 4 = 2, f(n) = n = O(n^{2-1})$, 属情形 1 ($\epsilon = 1$), 故 $T(n) = \Theta(n^2)$ 。

1.4 图搜索：BFS

说明 1.4. BFS 的直觉：从源点出发，像水波纹一样逐层向外扩散。第 k 层包含所有距源点恰好为 k 条边的节点。

Listing 1.1: BFS 伪代码

```
def BFS(G, s):
    for v in G.V:
        dist[v] = infinity
    dist[s] = 0
    queue = deque([s])
    while queue:
        u = queue.popleft()
        for v in G.neighbors(u):
            if dist[v] == infinity:
                dist[v] = dist[u] + 1
                queue.append(v)
    return dist
```

完整 BFS 过程示例：考虑 6 节点无向图，节点 $\{1, 2, 3, 4, 5, 6\}$ ，源点 $s = 1$ ，边集为 $\{1-2, 1-3, 2-4, 2-5, 3-6\}$ 。

步骤	出队节点	加入队列	各节点距离 $d[1..6]$
初始	—	[1]	$[0, \infty, \infty, \infty, \infty, \infty]$
1	1	[2, 3]	$[0, 1, 1, \infty, \infty, \infty]$
2	2	[3, 4, 5]	$[0, 1, 1, 2, 2, \infty]$
3	3	[4, 5, 6]	$[0, 1, 1, 2, 2, 2]$
4	4	[5, 6]	$[0, 1, 1, 2, 2, 2]$
5	5	[6]	$[0, 1, 1, 2, 2, 2]$
6	6	[]	$[0, 1, 1, 2, 2, 2]$

正确性：对任意节点 v ，BFS 结束时 $d[v]$ 等于图中 s 到 v 的最短路（边数）。归纳证明：若第 k 层节点的距离已正确设为 k ，则它们的邻居中未访问者距离恰为 $k+1$ ，被正确设置并入队。

复杂度 BFS 时间复杂度为 $\Theta(|V| + |E|)$ ，其中每条边被检查至多 2 次（无向图）或 1 次（有向图），每个节点入队出队各一次。

1.5 带正权最短路：Dijkstra

说明 1.5. Dijkstra 的直觉：贪心地维护一个“已确认最短距离”的节点集合 S 。每次从未确认节点中选出当前距离最小的节点，将其加入 S ，并用它更新邻居距离。这个过程像逐步扩张一个“已知安全区域”。

完整执行示例： 5 节点有向带权图，边及权重： $1 \rightarrow 2 : 4$, $1 \rightarrow 3 : 2$, $2 \rightarrow 4 : 3$, $3 \rightarrow 2 : 1$, $3 \rightarrow 4 : 5$, $4 \rightarrow 5 : 1$, 源点 $s = 1$ 。

轮次	选定节点	$d[1]$	$d[2]$	$d[3]$	$d[4]$	$d[5]$
初始	—	0	∞	∞	∞	∞
1	1	0	4	2	∞	∞
2	3	0	3	2	7	∞
3	2	0	3	2	6	∞
4	4	0	3	2	6	7
5	5	0	3	2	6	7

最短路结果： $d[1] = 0, d[2] = 3, d[3] = 2, d[4] = 6, d[5] = 7$, 路径 $1 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 5$ 。

实现复杂度：

优先队列实现	时间复杂度	适用场景
数组（线性扫描）	$O(V ^2)$	稠密图
二叉堆	$O((V + E) \log V)$	稀疏图
斐波那契堆	$O(E + V \log V)$	理论最优

核心记忆 Dijkstra 要求所有边权非负。贪心选择的正确性依赖于：从源点到已选节点的路径不可能被“绕一个负权边”而变短。若存在负权边，此假设不成立，需改用 Bellman-Ford。

第二章 深度优先搜索与强连通分量

对应课件：02.pdf

主题：DFS、DAG、SCC

2.1 为什么需要 DFS ?

BFS 适合探索“离源点多远”的问题，即找最短路径。而 DFS 适合探索图的结构性质，例如连通性、环的存在性、拓扑顺序等。

说明 2.1. DFS 的直觉：像在迷宫中探险——沿一条路一直走到底，遇到死路（已访问节点或无出边）才回溯，然后尝试其他方向。这种“深入再回溯”的策略天然记录了图的递归结构。

2.2 DFS 的时间戳与边分类

DFS 为每个节点记录两个时间戳：

- $\text{pre}[v]$ ：节点 v 第一次被访问（进入）时的时钟值。
- $\text{post}[v]$ ：节点 v 的所有后代都被探索完毕（退出）时的时钟值。

完整示例：8 节点有向图，节点 $\{1, \dots, 8\}$ ，边集： $1 \rightarrow 2, 1 \rightarrow 3, 2 \rightarrow 4, 2 \rightarrow 5, 3 \rightarrow 6, 4 \rightarrow 7, 5 \rightarrow 7, 6 \rightarrow 8, 7 \rightarrow 8, 3 \rightarrow 5$ 。从节点 1 开始 DFS（邻居按编号升序访问）：

节点	pre	post
1	1	16
2	2	11
3	12	15
4	3	8
5	9	10
6	13	14
7	4	7
8	5	6

四种边的定义与判断规则： 设 DFS 正在处理边 (u, v) ：

边的类型	条件	含义
树边 (Tree edge)	v 未被访问	DFS 树中的父子边
回边 (Back edge)	v 是 u 的祖先 ($\text{pre}[v] < \text{pre}[u]$, $\text{post}[v]$ 未定)	指向祖先, 形成有向环
前向边 (Forward edge)	v 是 u 的后代 ($\text{pre}[u] < \text{pre}[v]$, $\text{post}[v] < \text{post}[u]$)	跨越树中多层向下
横叉边 (Cross edge)	其余情形 ($\text{post}[v] < \text{pre}[u]$)	不同子树之间

证明思路 有向图中存在回边当且仅当图中存在有向环 (即图不是 DAG)。原因: 回边 (u, v) 加上树路径 $v \rightsquigarrow u$ 构成一个有向环; 反之, 若存在环则 DFS 必然遇到一条回边。

2.3 DAG 与拓扑排序

定义 2.1. 有向无环图 (DAG) 是不含有向环的有向图。拓扑排序 是将 DAG 的所有顶点排成一个线性序列, 使得对每条有向边 (u, v) , u 在序列中排在 v 之前。

说明 2.2. 直觉: 大学课程的先修关系构成一个 DAG——不可能存在“A 是 B 的先修, B 是 A 的先修”这样的循环。拓扑排序就是给出一个合理的选课顺序, 使得每门课在其所有先修课之后学习。

算法: 对 DAG 做 DFS, 按 post 值递减顺序输出节点, 即得一个拓扑排序。

证明思路 对任意边 (u, v) , 在有向图中 (u, v) 不是回边 (DAG 无环), 故 v 在 u 的 DFS 调用期间完成 (树边/前向边) 或在 u 之前完成 (横叉边)。两种情况均有 $\text{post}[v] < \text{post}[u]$, 即递减序中 u 排在 v 之前。

完整例子: 6 门课程: A (线代)、 B (微积分)、 C (概率论, 先修 A, B)、 D (算法, 先修 B)、 E (机器学习, 先修 C, D)、 F (毕业设计, 先修 E)。先修关系构成 DAG, DFS 后的 post 递减序 (一种合法拓扑排序) 为:

$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F$$

2.4 强连通分量

定义 2.2. 有向图 $G = (V, E)$ 的强连通分量 (SCC) 是最大顶点子集 $C \subseteq V$, 使得 C 内任意两顶点 u, v 互相可达 ($u \rightsquigarrow v$ 且 $v \rightsquigarrow u$)。

命题 2.1. 将 G 的每个 SCC 收缩为单个节点, 得到的凝聚图 (Component DAG) G^{SCC} 是一个 DAG。

证明思路 若凝聚图中存在环, 则环上所有 SCC 的节点两两可达, 与 SCC 的最大性矛盾。

2.5 Kosaraju 线性算法

说明 2.3. 核心直觉：在反图 G^R （所有边反向）上运行 DFS，post 值最大的节点 v^* 在原图凝聚 DAG 中对应一个源 SCC（没有入边的 SCC）。然后在原图上从 v^* 出发 DFS，恰好能到达（且仅能到达） v^* 所在的 SCC。重复此过程即可找出所有 SCC。

算法步骤：

1. 构造反图 G^R （将 G 中所有边方向反转）。
2. 在 G^R 上对所有节点运行 DFS，记录每个节点的 post 值。
3. 按 post 值递减顺序，在原图 G 上对每个未访问节点运行 DFS。每次 DFS 访问到的节点集合恰为一个 SCC。

完整示例： 7 节点有向图，节点 $\{1, \dots, 7\}$ ，边集： $1 \rightarrow 2, 2 \rightarrow 3, 3 \rightarrow 1, 3 \rightarrow 4, 4 \rightarrow 5, 5 \rightarrow 6, 6 \rightarrow 4, 6 \rightarrow 7$ 。

第一步：反图 G^R 的边： $2 \rightarrow 1, 3 \rightarrow 2, 1 \rightarrow 3, 4 \rightarrow 3, 5 \rightarrow 4, 6 \rightarrow 5, 4 \rightarrow 6, 7 \rightarrow 6$ 。

第二步：在 G^R 上运行 DFS（从节点 1 开始，未访问时从编号最小未访问节点出发），得到 post 顺序（递增）：

节点	post 值 (G^R 上)
1	6
2	5
3	4
4	9
5	8
6	7
7	14

第三步：按 post 递减顺序 (7, 4, 6, 5, 1, 2, 3) 在原图 G 上 DFS，得三个 SCC：

- 从节点 7 出发：仅到达 $\{7\}$ ， $\text{SCC}_1 = \{7\}$ 。
- 从节点 4 出发：到达 $\{4, 5, 6\}$ ， $\text{SCC}_2 = \{4, 5, 6\}$ （因 $4 \rightarrow 5 \rightarrow 6 \rightarrow 4$ 形成环）。
- 从节点 1 出发：到达 $\{1, 2, 3\}$ ， $\text{SCC}_3 = \{1, 2, 3\}$ （因 $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$ 形成环）。

证明思路 设 C^* 是凝聚 DAG 中 post 值最大节点所在的 SCC（在 G^R 上运行时对应原图的源 SCC）。从 C^* 中的节点在原图上 DFS，只能到达 C^* 内部（源 SCC 无入边），恰好覆盖整个 C^* 。去掉 C^* 后重复，归纳完成。

复杂度 Kosaraju 算法时间复杂度为 $\Theta(|V| + |E|)$ ：构造反图 $O(|V| + |E|)$ ，两次 DFS 各 $O(|V| + |E|)$ 。

第三章 最小生成树

对应课件: 03.pdf

主题: 割、环、Prim、Kruskal、Boruvka

3.1 动机

说明 3.1. 直觉: 一家电信公司需要用光纤连接 n 座城市, 使任意两城市间都有通路, 且铺设的总光纤长度最短。这就是**最小生成树 (MST)** 问题。

3.2 生成树与 MST

定义 3.1. 设 $G = (V, E, w)$ 是连通无向带权图。 G 的**生成树**是包含所有 $|V|$ 个顶点、无环、连通的子图, 恰好有 $|V| - 1$ 条边。 G 的**最小生成树 (MST)** 是所有生成树中总权重 $\sum_{e \in T} w(e)$ 最小的那棵。

3.3 割性质与环性质

定义 3.2. 图 $G = (V, E)$ 的一个**割 (Cut)** $(S, V \setminus S)$ 是将顶点集分为两个非空子集 S 和 $V \setminus S$ 的划分。一条边**跨越该割**, 若其一端在 S 中, 另一端在 $V \setminus S$ 中。

定理 3.1 (割性质 (Cut Property)). 设 $(S, V \setminus S)$ 是 G 的任意一个割, e^* 是跨越该割的唯一最轻边 (权重严格最小)。则 e^* 属于 G 的每一棵 MST。

证明思路 反证: 设某 MST T 不含 $e^* = (u, v)$ 。将 e^* 加入 T 形成唯一一个环 C , C 上必有另一条跨越割的边 $e' \neq e^*$ (因 $u \in S, v \notin S$, 路径 $u \rightsquigarrow v$ 在 T 中必再次跨割)。由于 $w(e^*) < w(e')$, 将 e' 替换为 e^* 得到权重更小的生成树, 与 T 是 MST 矛盾。

定理 3.2 (环性质 (Cycle Property)). 设 C 是 G 中的任意一个环, e^* 是 C 上权重严格最大的边。则 e^* 不属于 G 的任何 MST。

证明思路 若某 MST T 含 $e^* = (u, v)$, 删去 e^* 将 T 分为两棵子树 (形成一个割)。 C 上其余某条边 e' 跨越该割, 且 $w(e') < w(e^*)$ 。用 e' 替换 e^* 得到权重更小的生成树, 矛盾。

3.4 Kruskal 算法完整运行例子

考虑 6 节点图，节点 $\{A, B, C, D, E, F\}$ ，边及权重： $A-B : 4$ ， $A-C : 2$ ， $B-C : 1$ ， $B-D : 5$ ， $C-D : 8$ ， $C-E : 10$ ， $D-E : 2$ ， $D-F : 6$ ， $E-F : 3$ 。

边按权重排序： $B-C : 1$ ， $A-C : 2$ ， $D-E : 2$ ， $E-F : 3$ ， $A-B : 4$ ， $B-D : 5$ ， $D-F : 6$ ， $C-D : 8$ ， $C-E : 10$ 。

步骤	考虑的边	并查集状态 (各连通块)	是否加入 MST
1	$B-C : 1$	$\{A\}, \{B, C\}, \{D\}, \{E\}, \{F\}$	加入
2	$A-C : 2$	$\{A, B, C\}, \{D\}, \{E\}, \{F\}$	加入
3	$D-E : 2$	$\{A, B, C\}, \{D, E\}, \{F\}$	加入
4	$E-F : 3$	$\{A, B, C\}, \{D, E, F\}$	加入
5	$A-B : 4$	$\{A, B, C\}, \{D, E, F\}$	跳过 (同一连通块)
6	$B-D : 5$	$\{A, B, C, D, E, F\}$	加入

最终 MST 边集： $\{B-C, A-C, D-E, E-F, B-D\}$ ，总权重 $= 1 + 2 + 2 + 3 + 5 = 13$ 。

3.5 Prim 算法完整运行例子

同一图，从节点 A 出发。维护集合 S (已在 MST 中的节点) 和优先队列 (按边权排序的候选边)。

步骤	加入 MST 的节点 (及所用边)	优先队列中的候选边 (节点: 最小代价)
初始	A (起点)	$B : 4, C : 2$
1	C (边 $A-C : 2$)	$B : 1 (B-C), D : 8, E : 10$
2	B (边 $B-C : 1$)	$D : 5 (B-D), E : 10$
3	D (边 $B-D : 5$)	$E : 2 (D-E), F : 6$
4	E (边 $D-E : 2$)	$F : 3 (E-F)$
5	F (边 $E-F : 3$)	队列空, 算法结束

MST 同 Kruskal 结果：总权重 13。

3.6 Borůvka 简介

Borůvka 算法采用并行策略：在每一轮中，对每个连通块 (初始时每个节点是一个块) 找到离开该块的权重最小边，同时将所有这些边加入生成树 (注意去除重复)。

- **关键性质：** 每轮结束后，连通块数目至少减半 (每次合并至少两个块)。
- **轮数：** 至多 $O(\log |V|)$ 轮。
- **每轮时间：** $O(|E|)$ (扫描所有边，为每个块记录最轻跨越边)。
- **总时间：** $O(|E| \log |V|)$ ，且易于并行化，适合大规模分布式计算。

核心记忆 三种 MST 算法均基于贪心思想，正确性都由割性质或环性质保证。Kruskal 排序边后用并查集，适合稀疏图；Prim 用优先队列，适合稠密图；Borůvka 天然并行，适合分布式场景。对于带权无向连通图，这三种算法均能在多项式时间内找到 MST。

第四章 算法验证基础

对应课件：04.pdf

主题：模型检测、时序逻辑、有限与无限状态系统

4.1 动机——为什么测试不够

说明 4.1. 直觉：一个程序的输入空间可能是无限的（例如，接受任意整数输入的程序）。测试用例无论多么精心设计，都只能覆盖有限个输入，因此无法保证程序在所有输入下都正确。形式化验证的目标是用数学推理给出“对所有可能执行都满足性质 P ”这样的保证。

考虑并发程序中的竞态条件：即使运行一百万次测试都没有触发错误，也不能排除某种极罕见的线程调度顺序导致死锁。形式化验证通过穷举所有可能的状态和执行路径来消除这种不确定性。

4.2 形式化验证的三条路线

1. **模型检测 (Model Checking)**：将系统建模为有限状态机（或更丰富的模型），将性质表示为时序逻辑公式，自动穷举验证所有状态。优点：全自动，能给出反例；缺点：状态爆炸问题。
2. **定理证明 (Theorem Proving)**：用交互式证明辅助工具（如 Coq、Isabelle、Lean）手工构造数学证明。优点：可处理无限状态；缺点：需大量人工投入。
3. **类型系统 (Type Systems)**：通过静态类型检查自动排除一类错误（如类型错误、空指针解引用）。优点：轻量，编译时完成；缺点：只能处理特定性质。

4.3 Kripke 结构

定义 4.1. Kripke 结构 $M = (S, S_0, R, L)$ 由以下四部分组成：

- S ：有限状态集合。
- $S_0 \subseteq S$ ：初始状态集合。
- $R \subseteq S \times S$ ：转移关系（要求对每个状态至少有一个后继，即无死锁状态）。
- $L : S \rightarrow 2^{AP}$ ：标记函数，给每个状态标注在该状态成立的原子命题集合， AP 是原子命题集。

具体例子：交通灯系统。

状态集 $S = \{\text{红}, \text{绿}, \text{黄}\}$ ，初始状态 $S_0 = \{\text{红}\}$ ，转移 $R: \text{红} \rightarrow \text{绿} \rightarrow \text{黄} \rightarrow \text{红}$ ，原子命题 $AP = \{\text{safe}, \text{moving}\}$ 。

状态	成立的原子命题	含义
红	$\{\text{safe}\}$	行人安全，车辆停止
绿	$\{\text{moving}\}$	车辆行驶，行人禁行
黄	$\{\}$	过渡状态

- **安全性 (Safety)**: 系统永远不会进入某个“坏”状态，例如“红灯和绿灯不会同时成立”。形式上: $AG(\neg(\text{safe} \wedge \text{moving}))$ 。
- **活性 (Liveness)**: 某件好事最终一定会发生，例如“每次变红之后，最终一定会变绿”。形式上: $AG(\text{safe} \Rightarrow AF \text{moving})$ 。

4.4 时序逻辑 CTL

计算树逻辑 (CTL) 在 Kripke 结构的计算树（将状态图展开为无限树）上描述性质。
路径量词:

- $A\varphi$: 对所有从当前状态出发的路径， φ 成立 (“必然”)。
- $E\varphi$: 存在从当前状态出发的某条路径， φ 成立 (“可能”)。

时间算子 (作用于一条路径):

- $X\varphi$: 在路径的下一步 (neXt), φ 成立。
- $F\varphi$: 在路径的将来某步 (Finally), φ 成立。
- $G\varphi$: 在路径的所有将来步骤 (Globally), φ 始终成立。
- $\varphi U \psi$: φ 持续成立，直到 (Until) ψ 成立。

CTL 要求路径量词与时间算子总是成对出现 (如 AG, EF, AU, EU 等)。

自然语言翻译举例:

CTL 公式	自然语言含义
$AG(\neg \text{bad})$	系统在所有路径的所有时刻永远不会进入坏状态 (安全性)
$AG(\text{req} \Rightarrow AF \text{ack})$	每次请求最终都会在所有路径上得到应答 (响应性)
$EF \text{goal}$	存在某条执行路径，使得系统最终能到达目标状态 (可达性)
$EG \text{run}$	存在某条无限执行路径，使系统可以永远持续运行 (非终止可能性)

4.5 CTL 模型检测算法

CTL 模型检测的核心思想是：对公式 φ 递归地计算满足 φ 的状态集合 $\llbracket \varphi \rrbracket \subseteq S$ 。

三个核心算子的计算方法:

- $\llbracket EX \varphi \rrbracket$: 满足“存在某后继状态满足 φ ”的状态集合。

$$\llbracket EX \varphi \rrbracket = \{s \in S \mid \exists s'. (s, s') \in R \wedge s' \in \llbracket \varphi \rrbracket\}$$

实现：计算 $\llbracket \varphi \rrbracket$ 的前驱状态集合。

- $\llbracket E[\varphi U \psi] \rrbracket$: 满足“存在路径， φ 持续到某步 ψ 成立”的状态。不动点迭代：初始 $W_0 = \llbracket \psi \rrbracket$ ，迭代 $W_{i+1} = W_i \cup (\llbracket \varphi \rrbracket \cap \llbracket EX W_i \rrbracket)$ ，直到收敛。
- $\llbracket EG \varphi \rrbracket$: 满足“存在一条无限路径， φ 在路径上始终成立”的状态。最大不动点：初始 $W_0 = \llbracket \varphi \rrbracket$ ，迭代 $W_{i+1} = W_i \cap \llbracket EX W_i \rrbracket$ ，直到收敛。

其余算子均可用这三个算子表达（例如 $AF \varphi \equiv \neg EG \neg \varphi$ ）。总时间复杂度： $O(|\varphi| \cdot (|S| + |R|))$ ，其中 $|\varphi|$ 是公式大小。

4.6 扩展模型简介

实际系统往往状态数目极大，是模型检测的主要瓶颈（**状态爆炸问题**）。针对不同场景有多种扩展模型：

- **下推系统 (Pushdown System)**: 用于建模带调用栈的递归程序，状态包括控制位置和栈内容，状态空间无限，但某些性质仍可判定。
- **时间自动机 (Timed Automata)**: 在有限状态机中加入实数时钟变量，能描述实时性约束（如“请求 5 毫秒内必须响应”），时间可达性问题可判定。
- **Petri 网 (Petri Net)**: 用于建模并发系统中的资源分配与同步，通过“库所—变迁”结构自然表达并发，可达性问题一般难以判定但特殊情况有多项式算法。

核心记忆 模型检测提供了一种自动化的形式化验证方法：将系统建模为 Kripke 结构，将性质写成 CTL 公式，算法自动判定所有状态是否满足该性质，并在不满足时给出反例路径。关键挑战是应对状态爆炸——实际工具使用 BDD（二叉决策图）或 SAT/SMT 求解器等符号化方法来压缩状态表示。

第五章 序列比对与动态规划

对应课件：05.pdf

主题：DAG 最短路、LIS、编辑距离、Hirschberg、LCS

5.1 动机

DNA 序列比对是生物信息学的核心问题：给定两段 DNA 序列（由 A/T/C/G 构成的字符串），衡量它们有多“相似”，从而推断物种进化关系或找出功能相似的基因片段。拼写纠错的本质相同：将用户输入的单词与词典中的单词比较，找出编辑代价最小的候选词。

说明 5.1. 直觉：两个字符串的相似度可以量化为“把一个串变成另一个串所需的最少操作次数”，这就是**编辑距离** (Edit Distance)。操作包括：插入一个字符、删除一个字符、替换一个字符，每种操作代价为 1。

5.2 动态规划的核心思想

动态规划 (DP) 适用于同时具备以下两个性质的问题：

- **最优子结构**：问题的最优解包含子问题的最优解。
- **重叠子问题**：递归求解时，相同的子问题会被反复计算。

为什么暴力搜索太慢：对两个长度为 m 和 n 的字符串，所有可能的比对方式数量是指数级的（大致为 $\binom{m+n}{m}$ ），暴力枚举不可行。

DAG 视角：子问题天然形成一个 DAG——每个子问题是一个节点，依赖关系是有向边，且依赖方向不形成环（子问题规模严格递减）。按照 DAG 的拓扑序（即子问题规模从小到大）依次计算，每个子问题只需计算一次，总时间 = 子问题数量 $\times$ 每个子问题的计算时间。

5.3 编辑距离

定义 5.1. 字符串 $X = x_1x_2 \cdots x_m$ 与 $Y = y_1y_2 \cdots y_n$ 的**编辑距离** $\text{ed}(X, Y)$ 是将 X 变为 Y 所需的最少插入/删除/替换操作次数，每种操作代价为 1。

DP 递推式：设 $\text{dp}[i][j]$ 为 $X[1..i]$ 与 $Y[1..j]$ 的编辑距离。

$$\begin{aligned} \text{dp}[0][j] &= j \quad (0 \leq j \leq n), \\ \text{dp}[i][0] &= i \quad (0 \leq i \leq m), \\ \text{dp}[i][j] &= \begin{cases} \text{dp}[i-1][j-1] & \text{若 } x_i = y_j, \\ 1 + \min(\text{dp}[i-1][j], \text{dp}[i][j-1], \text{dp}[i-1][j-1]) & \text{若 } x_i \neq y_j. \end{cases} \end{aligned}$$

三项分别对应：删除 x_i （从上方）、插入 y_j （从左方）、替换 x_i 为 y_j （从左上方）。

完整 DP 表格： $X = \text{SUNNY}$, $Y = \text{SNOWY}$ 。表头行对应 Y （加空串前缀），表头列对应 X （加空串前缀）。

	ε	S	N	O	W	Y
ε	0	1	2	3	4	5
S	1	0	1	2	3	4
U	2	1	1	2	3	4
N	3	2	1	2	3	4
N	4	3	2	2	3	4
Y	5	4	3	3	3	3

编辑距离为 $\text{dp}[5][5] = 3$ 。

回溯最优对齐：沿 DP 表格从右下角往左上角回溯（选代价最小的前驱），得到一个最优对齐方案：

$$\begin{array}{cccccc} S & - & U & N & N & Y \\ S & N & - & O & W & Y \end{array}$$

（删除 U ，将 $N \rightarrow O$ ，将 $N \rightarrow W$ ，共 3 次操作。实际上，将第二个 N 替换为 O ，第三个替换为 W ，删去第一个 U ；或其他等价方案。）

5.4 最长公共子序列 LCS

定义 5.2. 字符串 X 和 Y 的**最长公共子序列 (LCS)** 是同时为 X 的子序列和 Y 的子序列的最长字符串（子序列允许不连续但需保持相对顺序）。

DP 状态与递推：设 $L[i][j]$ 为 $X[1..i]$ 与 $Y[1..j]$ 的 LCS 长度：

$$L[i][j] = \begin{cases} L[i-1][j-1] + 1 & \text{若 } x_i = y_j, \\ \max(L[i-1][j], L[i][j-1]) & \text{若 } x_i \neq y_j. \end{cases}$$

简要例子： $X = \text{ABCB DAB}$, $Y = \text{BDCAB}$ 。LCS 长度为 4，一个 LCS 为 BCAB（或 BDAB）。

说明 5.2. LCS 与编辑距离的关系：若只允许插入和删除操作（不允许替换），编辑距离恰为 $(|X| - L) + (|Y| - L)$ ，其中 L 是 LCS 长度。

5.5 Hirschberg 算法

说明 5.3. 直觉：朴素 DP 需要 $O(mn)$ 空间存储整张 DP 表。Hirschberg 算法用分治避免存储整张表，只需 $O(m+n)$ 空间，同时保持 $O(mn)$ 时间。

核心观察：计算 $dp[i][\cdot]$ (第 i 行) 只需第 $i-1$ 行，故只需 $O(n)$ 空间可以计算出 DP 表的任意一行。

算法思路：

1. 取 X 的中间位置 $i^* = \lfloor m/2 \rfloor$ 。
2. 用正向 DP 计算 $dp[i^*][\cdot]$ ($X[1..i^*]$ 对 $Y[1..j]$ 的编辑距离, $j = 0, \dots, n$), 只需 $O(n)$ 空间。
3. 用反向 DP 计算 $dp'[i^*][\cdot]$ ($X[i^*+1..m]$ 对 $Y[j+1..n]$ 的编辑距离, $j = 0, \dots, n$), 只需 $O(n)$ 空间。
4. 找出使 $dp[i^*][j^*] + dp'[i^*][j^*]$ 最小的切割点 j^* 。
5. 递归对 $(X[1..i^*], Y[1..j^*])$ 和 $(X[i^*+1..m], Y[j^*+1..n])$ 求解。

复杂度分析：设 $T(m, n)$ 为总时间。每层递归处理长度为 m 和 n 的两个字符串，做两次 $O(mn)$ 的 DP (正向和反向)，然后递归两个子问题 (总长度减半)： $T(m, n) \leq 2cmn + T(m/2, n) + T(m/2, n)$ ，解得 $T(m, n) = O(mn)$ 。空间只需维护当前两行，递归栈深度 $O(\log m)$ ，每层 $O(n)$ ，总空间 $O(n \log m) = O(m+n)$ (更精细分析给出 $O(m+n)$)。

易错点 常见错误：混淆子序列 (subsequence) 和子串 (substring)。子串要求字符在原串中连续出现，而子序列只需保持相对顺序但可以不连续。例如，ACE 是 ABCDE 的子序列但不是子串；BCD 既是子序列也是子串。LCS 是子序列问题；”最长公共子串”是不同的问题，有 $O(mn)$ DP 解法，亦可用后缀数组在 $O((m+n) \log(m+n))$ 时间解决。

第六章 负权最短路

对应课件：06.pdf

主题：Bellman-Ford、负环、可靠路径、全源最短路

6.1 动机——Dijkstra 的局限

说明 6.1. Dijkstra 正确性的关键假设是：一旦某节点被”确定”（加入集合 S ），其距离不会再被更新。这依赖于所有边权非负——若所有边权 ≥ 0 ，则从未确定节点出发的路径只会更长，永远不会”绕弯”缩短已确定节点的距离。若存在负权边，此假设失效。

两个重要的负权场景：

- **金融套利**：货币兑换汇率可以建模为带权有向图（对数化后可能产生负权边），若存在负权环则意味着可以无限循环套利。
- **差分约束系统**：线性不等式组 $x_j - x_i \leq w_{ij}$ 可转化为最短路问题，其中权重可以为负。

6.2 Bellman-Ford 算法

说明 6.2. 直觉：”放松所有边，重复 $|V| - 1$ 次”。关键不变式：第 k 轮结束后，对所有节点 v ， $d[v]$ 等于从源点 s 到 v 的至多使用 k 条边的最短路长度。因为最短路（若无负环）最多使用 $|V| - 1$ 条边，故 $|V| - 1$ 轮后所有距离均已收敛。

Listing 6.1: Bellman-Ford 伪代码

```
def BellmanFord(G, s):
    for v in G.V:
        d[v] = infinity
    d[s] = 0
    for k in range(1, |G.V|):          # 共 |V|-1 轮
        for (u, v, w) in G.E:         # 枚举所有边
            if d[u] + w < d[v]:
                d[v] = d[u] + w
    # 检测负环
    for (u, v, w) in G.E:
        if d[u] + w < d[v]:
            return "存在负环"
    return d
```

完整 5 节点运行例子：节点 $\{1, 2, 3, 4, 5\}$ ，边： $1 \rightarrow 2 : 6$ ， $1 \rightarrow 3 : 7$ ， $2 \rightarrow 3 : 8$ ， $2 \rightarrow 4 : -4$ ， $2 \rightarrow 5 : 5$ ， $3 \rightarrow 4 : 9$ ， $3 \rightarrow 5 : -3$ ， $4 \rightarrow 1 : 2$ ， $5 \rightarrow 4 : 7$ ，源点 $s = 1$ 。

轮次	$d[1]$	$d[2]$	$d[3]$	$d[4]$	$d[5]$
初始	0	∞	∞	∞	∞
轮次 1	0	6	7	2	4
轮次 2	0	6	7	2	4
轮次 3	0	6	7	2	4
轮次 4	0	6	7	2	4

最终最短路： $d[1] = 0$ ， $d[2] = 6$ ， $d[3] = 7$ ， $d[4] = 2$ （路径 $1 \rightarrow 2 \rightarrow 4$ ，长度 $6 + (-4) = 2$ ）， $d[5] = 4$ （路径 $1 \rightarrow 3 \rightarrow 5$ ，长度 $7 + (-3) = 4$ ）。

6.3 负环检测

检测方法：在 $|V| - 1$ 轮 Bellman-Ford 结束后，额外运行第 $|V|$ 轮：若仍有边可以松弛（即存在边 (u, v, w) 使 $d[u] + w < d[v]$ ），则图中存在负权环。

证明思路 若不存在负环，则最短路最多使用 $|V| - 1$ 条边， $|V| - 1$ 轮后所有距离已收敛，第 $|V|$ 轮不会再松弛任何边。若存在负环，则沿负环绕圈的路径长度可以无限减小，故第 $|V|$ 轮（以及之后每轮）都还能松弛。

找出具体的负环：若第 $|V|$ 轮中某条边 (u, v) 被松弛，则沿前驱指针 $\text{parent}[\cdot]$ 从 u 回溯 $|V|$ 步，由鸽巢原理必经过某个重复节点，该节点即为负环上的一个节点，沿前驱指针绕一圈即可找出完整负环。

6.4 Floyd-Warshall

说明 6.3. 直觉：**全源最短路**问题需要计算所有节点对 (i, j) 之间的最短路。Floyd-Warshall 的思路：“逐步”放宽“允许经过的中间节点集合”。设 $D^{(k)}[i][j]$ 为仅允许经过节点 $\{1, \dots, k\}$ 作为中间节点时， i 到 j 的最短路长度。

递推式：

$$D^{(k)}[i][j] = \min(D^{(k-1)}[i][j], D^{(k-1)}[i][k] + D^{(k-1)}[k][j]).$$

含义：要么不经过节点 k （直接用 $D^{(k-1)}[i][j]$ ），要么经过节点 k （先到 k ，再从 k 到 j ）。

完整 4 节点例子：节点 $\{1, 2, 3, 4\}$ ，边： $1 \rightarrow 2 : 3$ ， $2 \rightarrow 4 : 1$ ， $1 \rightarrow 3 : 8$ ， $3 \rightarrow 2 : -4$ ， $4 \rightarrow 3 : 2$ ，其余对 (i, j) 为 ∞ ($i \neq j$)，对角线为 0。

$D^{(0)}$ （初始邻接矩阵， ∞ 表示无直接边）：

$D^{(0)}$	1	2	3	4
1	0	3	8	∞
2	∞	0	∞	1
3	∞	-4	0	∞
4	∞	∞	2	0

$D^{(1)}$ (允许经过节点 1): 节点 1 没有入边 (所有 $D^{(0)}[i][1] = \infty, i \neq 1$), 故 $D^{(1)} = D^{(0)}$ (无变化)。

$D^{(2)}$ (允许经过节点 2, 即可以利用路径 $i \rightarrow 2 \rightarrow j$):

$D^{(2)}$	1	2	3	4
1	0	3	8	4
2	∞	0	∞	1
3	∞	-4	0	-3
4	∞	∞	2	0

(新增: $D[1][4] = D[1][2] + D[2][4] = 3 + 1 = 4$; $D[3][4] = D[3][2] + D[2][4] = -4 + 1 = -3$ 。)

$D^{(3)}$ (允许经过节点 3, 即可以利用路径 $i \rightarrow 3 \rightarrow j$):

$D^{(3)}$	1	2	3	4
1	0	3	8	4
2	∞	0	∞	1
3	∞	-4	0	-3
4	∞	-2	2	0

(新增: $D[4][2] = D[4][3] + D[3][2] = 2 + (-4) = -2$ 。)

$D^{(4)}$ (允许经过节点 4, 即可以利用路径 $i \rightarrow 4 \rightarrow j$):

$D^{(4)}$	1	2	3	4
1	0	3	6	4
2	∞	0	3	1
3	∞	-4	0	-3
4	∞	-2	2	0

(新增: $D[1][3] = D[1][4] + D[4][3] = 4 + 2 = 6 < 8$; $D[2][3] = D[2][4] + D[4][3] = 1 + 2 = 3$ 。)

复杂度 Floyd-Warshall 时间复杂度为 $\Theta(|V|^3)$, 空间 $\Theta(|V|^2)$ (利用原地更新只需一个矩阵)。适用于稠密图和全源最短路需求, 实现极为简洁。若需要检测负环, 只需检查最终矩阵的对角线是否有负值。

核心记忆 Bellman-Ford 和 Floyd-Warshall 都能处理负权边。Bellman-Ford 适合单源情形，时间 $O(|V||E|)$ ；Floyd-Warshall 适合全源情形，时间 $O(|V|^3)$ 。两者都能检测负环。若图无负权边，Dijkstra（单源 $O((|V| + |E|) \log |V|)$ ）或对每个源跑 Dijkstra（全源 $O(|V|(|V| + |E|) \log |V|)$ ）通常更快。

第七章 树宽

对应课件：07.pdf

主题：树上的多项式算法、树分解、部分 k-树

7.1 动机

许多在一般图上 NP-hard 的问题（如独立集、图着色、哈密顿路）在树上可以用线性时间动态规划解决。这启发了一个自然的问题：是否存在一种度量，衡量一个图“有多像树”？若一个图“接近树”，能否也高效求解这些困难问题？

说明 7.1. 直觉：树是最简单的连通图——没有环，结构完全“线性化”。**树宽 (Treewidth)** 正是刻画图的树状程度的参数：树宽越小，图越接近树；树宽越大，图越“纠缠”（如完全图树宽最大）。固定树宽后，许多 NP-hard 问题变成多项式（甚至线性）时间可解。

7.2 树上独立集 DP

在根树（有根的无向树）上求最大独立集，可以用 DP 在 $O(|V|)$ 时间内解决。

状态定义：对树中每个节点 v ，定义：

- $IN[v]$ ：以 v 为根的子树中，包含 v 的最大独立集大小。
- $OUT[v]$ ：以 v 为根的子树中，不包含 v 的最大独立集大小。

递推：设 v 的孩子节点集合为 $C(v)$ ：

$$IN[v] = 1 + \sum_{c \in C(v)} OUT[c], \quad OUT[v] = \sum_{c \in C(v)} \max(IN[c], OUT[c]).$$

最终答案为 $\max(IN[root], OUT[root])$ 。

小例子：考虑以下 7 节点有根树，根为节点 1：

- 节点 1 的孩子：{2, 3}
- 节点 2 的孩子：{4, 5}
- 节点 3 的孩子：{6, 7}
- 节点 4, 5, 6, 7：叶节点

计算过程（从叶到根）：

节点 v	IN[v]	OUT[v]
4 (叶)	1	0
5 (叶)	1	0
6 (叶)	1	0
7 (叶)	1	0
2	$1 + \text{OUT}[4] + \text{OUT}[5] = 1 + 0 + 0 = 1$	$\max(1, 0) + \max(1, 0) = 2$
3	$1 + \text{OUT}[6] + \text{OUT}[7] = 1 + 0 + 0 = 1$	$\max(1, 0) + \max(1, 0) = 2$
1	$1 + \text{OUT}[2] + \text{OUT}[3] = 1 + 2 + 2 = 5$	$\max(1, 2) + \max(1, 2) = 4$

最大独立集大小为 $\max(\text{IN}[1], \text{OUT}[1]) = \max(5, 4) = 5$ (选节点 1, 4, 5, 6, 7)。

7.3 树分解

说明 7.2. 直觉：树分解把图”拆”成一袋袋顶点挂在树上。每个袋子 (bag) 是一个顶点集合，代表图的一个”局部界面”。若树宽小，则每个袋子都很小，可以在每个袋子上高效地做 DP。

定义 7.1. 图 $G = (V, E)$ 的**树分解** 是一个二元组 $(T, \{X_t\}_{t \in V(T)})$ ，其中 T 是一棵树， $X_t \subseteq V$ 是每个树节点 t 对应的**袋子**，满足以下三个条件：

1. **覆盖条件**： $\bigcup_{t \in V(T)} X_t = V$ (每个图顶点至少在一个袋子中)。
2. **边条件**：对 G 的每条边 $(u, v) \in E$ ，存在某个树节点 t 使得 $u, v \in X_t$ (每条边的两端在某个共同袋子中)。
3. **连通条件 (Helly 性质)**：对每个图顶点 $v \in V$ ，包含 v 的袋子集合 $\{t \in V(T) \mid v \in X_t\}$ 在树 T 中构成一个连通子树。

宽度 定义为 $\max_{t \in V(T)} |X_t| - 1$ 。图 G 的**树宽** $\text{tw}(G)$ 是所有树分解中最小的宽度。

7.4 具体图的树分解例子

考虑 6 节点图：节点 $\{1, 2, 3, 4, 5, 6\}$ ，边集包含 1-2, 2-3, 3-4, 4-5, 5-6, 6-1 (六环) 以及对角线 1-4。

一个树分解 (宽度为 2)：

- 树节点 T_1 ，袋子 $X_1 = \{1, 2, 3\}$
- 树节点 T_2 ，袋子 $X_2 = \{1, 3, 4\}$
- 树节点 T_3 ，袋子 $X_3 = \{1, 4, 5\}$
- 树节点 T_4 ，袋子 $X_4 = \{1, 5, 6\}$
- 树边： $T_1 - T_2 - T_3 - T_4$ (路径形状)

验证：

- 覆盖： $\{1, 2, 3\} \cup \{1, 3, 4\} \cup \{1, 4, 5\} \cup \{1, 5, 6\} = \{1, 2, 3, 4, 5, 6\}$ 。✓

- 边条件: $1-2, 2-3, 1-3 \subseteq X_1$; $1-3, 3-4, 1-4 \subseteq X_2$; $1-4, 4-5, 1-5 \subseteq X_3$; $1-5, 5-6, 1-6 \subseteq X_4$.
✓
 - 连通条件: 节点 1 出现在所有袋子中 (构成路径, 连通); 节点 3 出现在 T_1, T_2 中 (相邻, 连通); 其余节点验证类似。✓
- 宽度 $= \max(3, 3, 3, 3) - 1 = 2$ 。

7.5 树宽的性质

命题 7.1. 树的树宽为 1。

证明思路 对树 T 本身做树分解: 选取 T 的任意一条边 (u, v) 对应一个袋子 $\{u, v\}$, 将 T 的所有边各对应一个袋子, 这些袋子之间的连接关系即构成一棵树 (与 T 本身对应的“细分”结构)。每个袋子大小为 2, 宽度为 $2 - 1 = 1$ 。

命题 7.2. 完全图 K_n 的树宽为 $n - 1$ 。

证明思路 下界: K_n 的任意树分解中, 考虑任意两个相邻袋子 X_s, X_t (树边 (s, t))。删去这条树边将树分为两棵子树, 分别覆盖某些图顶点集合 A 和 B , 满足 $A \cup B = V$, $A \cap B \subseteq X_s \cap X_t$ 。由于 K_n 中 A 和 B 中的所有顶点之间都有边, 需要在某个袋子中同时包含 A 中所有顶点和 B 中所有顶点; 由 Helly 性质 (连通条件), $X_s \cap X_t$ 必须包含“分隔”两侧的所有顶点, 最终推得某个袋子大小至少为 n , 宽度至少为 $n - 1$ 。上界: K_n 的一个树分解是单个袋子 $\{1, \dots, n\}$ (仅一个树节点), 宽度恰为 $n - 1$ 。

总结: 树宽为 1 当且仅当图是树 (或森林); 树宽越小, 图的结构越像树, 越适合用树分解上的 DP 求解 NP-hard 问题。

7.6 树分解上的 DP

平滑树分解 (Nice Tree Decomposition): 为便于 DP, 将树分解整理为标准形式, 树中每个节点属于以下四种类型之一:

- **叶节点:** 无孩子, 袋子大小为 1。
- **引入节点 (Introduce):** 有一个孩子 c , $X_t = X_c \cup \{v\}$ (比孩子袋子多一个顶点 v)。
- **遗忘节点 (Forget):** 有一个孩子 c , $X_t = X_c \setminus \{v\}$ (比孩子袋子少一个顶点 v)。
- **合并节点 (Join):** 有两个孩子 c_1, c_2 , $X_t = X_{c_1} = X_{c_2}$ (袋子相同)。

宽度 k 的树分解上的独立集 DP:

对每个树节点 t , 枚举袋子 X_t 的所有 $2^{|X_t|} \leq 2^{k+1}$ 种子集 $S \subseteq X_t$, 记录 $\text{dp}[t][S]$ 以 S 作为 X_t 中被选入独立集的顶点时, t 的子树所能获得的最大独立集大小 (满足 S 在 G 中是独立集, 且所有与子树内部顶点有边的 X_t 中的顶点选择与 S 兼容)。

四种节点类型各有 $O(2^{k+1})$ 种状态, 每种状态计算时间 $O(2^{k+1})$ (合并节点) 或 $O(1)$ (其余类型), 故每个树节点计算时间 $O(4^{k+1})$, 树节点总数 $O(|V|)$, 总时间为:

复杂度 宽度为 k 的树分解上，独立集问题可在 $O(4^k \cdot |V|)$ 时间内解决。更一般地，若问题的状态空间在每个袋子上可以用 $f(k)$ 种赋值表示，则总时间为 $O(f(k) \cdot |V|)$ 。固定 k （即树宽为常数），这是关于 $|V|$ 的多项式时间算法。

核心记忆 树宽是衡量图结构复杂度的核心参数。树的树宽为 1，完全图 K_n 的树宽为 $n - 1$ 。固定树宽 k 后，独立集、图着色、哈密顿路等 NP-hard 问题都有 $f(k) \cdot \text{poly}(|V|)$ 时间算法，属于固定参数可处理（FPT）。这正是参数化复杂度理论的核心思想：即使一个问题整体是 NP-hard，只要其“复杂度参数”（如树宽）较小，问题仍可高效解决。

第八章 最大流与最小割

对应课件：08.pdf

主题：流网络、残量网络、Ford-Fulkerson、最大流最小割

为什么学这一章？想象一个城市供水网络：水源 s 、用水区 t ，中间有容量各异的管道。问题是：每秒最多能从 s 送多少水到 t ？又比如互联网带宽分配、货运调度、人员匹配……这些问题有一个统一的数学模型——最大流。更深刻的是，“能送出去的最大量”恰好等于“切断网络最需要代价”，这就是著名的最大流最小割定理。

8.1 流网络

定义 8.1 (流网络). 流网络 $G = (V, E, s, t, c)$ 是一个有向图，其中 $s \in V$ 为源点， $t \in V$ 为汇点，每条边 $e \in E$ 有容量 $c(e) > 0$ 。

直觉。把每条有向边想象成一根水管：方向是水流方向，容量 $c(e)$ 是水管的最大截面积（即最大流速）。 s 是水泵， t 是蓄水池，中间的节点（除 s, t 外）不囤水——进来多少就必须流出多少。

定义 8.2 (流). s - t 流 f 是一个函数 $f: E \rightarrow \mathbb{R}_{\geq 0}$ ，满足：

- **容量约束**： $0 \leq f(e) \leq c(e)$ ，对所有 $e \in E$ 。
- **流守恒**：对每个 $v \neq s, t$ ，流入量等于流出量：

$$\sum_{e \text{ 进入 } v} f(e) = \sum_{e \text{ 离开 } v} f(e).$$

流值 $|f|$ 定义为从 s 净流出的量： $|f| = \sum_{e \text{ 离开 } s} f(e) - \sum_{e \text{ 进入 } s} f(e)$ 。

8.2 割

直觉。若想切断 s 与 t 之间的联系（比如敌方想切断补给线），只需删掉一组边使所有 $s \rightarrow t$ 路径都断掉。最小割就是使这一代价（被删边容量之和）最小的切法。

定义 8.3 (s - t 割). s - t 割是节点集合的划分 (S, T) ，其中 $s \in S$ ， $t \in T$ 。割容量为：

$$c(S, T) = \sum_{\substack{u \in S, v \in T \\ (u, v) \in E}} c(u, v).$$

注意：只计算从 S 到 T 的正向边，不计反向边。

命题 8.1 (弱对偶). 任意流 f 与任意割 (S, T) 满足 $|f| \leq c(S, T)$ 。

证明思路 流守恒意味着 $|f|$ 等于跨越割的净流量 ($S \rightarrow T$ 正向流减去 $T \rightarrow S$ 反向流)。而净流量不超过正向容量之和 $c(S, T)$ 。

8.3 残量网络

为什么需要残量网络？ 贪心算法的缺陷：一旦给某条边分配了流量，就不能撤回——可能陷入次优解。残量网络的核心想法是引入反向边，赋予“后悔权”：已经往 $u \rightarrow v$ 送了 $f(e)$ 单位的流，就在 $v \rightarrow u$ 上新建一条容量为 $f(e)$ 的边。这允许“取消之前的决定”——在反向边上送流，相当于撤回原来的流。

定义 8.4 (残量网络). 给定流网络 G 和流 f , 残量网络 $G_f = (V, E_f, s, t, c_f)$ 定义如下：对每条原始边 $e = (u, v)$ ：

- **前向边** (u, v) ：残量容量 $c_f(u, v) = c(e) - f(e)$ (还能再送多少)。
- **反向边** (v, u) ：残量容量 $c_f(v, u) = f(e)$ (可以撤回多少)。

只保留残量容量 > 0 的边。

完整运行示例。 考虑如下 4 节点网络 (s, a, b, t) ：

$$s \xrightarrow{3} a, \quad s \xrightarrow{3} b, \quad a \xrightarrow{2} t, \quad b \xrightarrow{2} t, \quad a \xrightarrow{1} b.$$

初始状态： 所有边流量为 0, $|f| = 0$ 。

第一次增广： 在残量网络中找到路径 $P_1 : s \rightarrow a \rightarrow t$, 瓶颈容量 $\Delta_1 = \min(3, 2) = 2$ 。增广后： $f(s, a) = 2, f(a, t) = 2, |f| = 2$ 。

残量网络变化 (第一次增广后)：

边	原容量	残量容量
(s, a) 前向	3	$3 - 2 = 1$
(a, s) 反向	0	2 (新增!)
(a, t) 前向	2	$2 - 2 = 0$ (饱和)
(t, a) 反向	0	2 (新增!)
(s, b) 前向	3	3
(b, t) 前向	2	2
(a, b) 前向	1	1

第二次增广： 在残量网络中找到路径 $P_2 : s \rightarrow b \rightarrow t$, 瓶颈容量 $\Delta_2 = \min(3, 2) = 2$ 。增广后： $f(s, b) = 2, f(b, t) = 2, |f| = 4$ 。

第三次增广尝试： 残量网络中从 s 能到达 a (余量 1) 和 b (余量 1, 但 (b, t) 已饱和)。路径 $s \rightarrow a \rightarrow b$ ：但 (b, t) 余量为 0, 无法到达 t 。无增广路, 算法终止。

最终流： $f(s, a) = 2, f(s, b) = 2, f(a, t) = 2, f(b, t) = 2, |f| = 4$ 。

验证最大流最小割定理： 从 s 在残量图中可达节点集 $S = \{s, a\}$ (因为 (a, t) 饱和、 (s, b) 还有余量但 (b, t) 饱和, b 也可达 $S = \{s, a, b\}$)。实际上从 s 可达 $\{s, a, b\}$, $T = \{t\}$ 。割 $(\{s, a, b\}, \{t\})$ 的容量 $= c(a, t) + c(b, t) = 2 + 2 = 4 = |f|$ 。最大流最小割等式成立！

8.4 Ford-Fulkerson 算法

```

FORD-FULKERSON(G, s, t):
    // 初始化: 所有边流量为 0
    for each edge e in E:
        f(e) = 0
    // 迭代增广
    while 残量网络 Gf 中存在 s->t 路径 P:
        Delta = min residual capacity on P    // 瓶颈容量
        for each edge e in P:
            if e 是原始边:
                f(e) += Delta
            else: // e 是反向边 (u,v) reverse
                f(reverse(e)) -= Delta
    return f

```

定理 8.1 (Ford-Fulkerson 终止性与正确性). 若所有容量为整数, 则 *Ford-Fulkerson* 在至多 $|f^*|$ 次增广后终止, 其中 f^* 是最大流。时间复杂度 $O(|V| \cdot |E| \cdot C)$, C 为最大容量。

证明思路 整数容量下, 每次增广流值至少增加 1, 故至多经过 $|f^*| \leq |V| \cdot C$ 次增广终止。每次增广用 BFS/DFS 找路径需 $O(|E|)$ 。

易错点 Ford-Fulkerson 不指定选哪条增广路, 因此不是完整定义的多项式算法。非整数容量时甚至可能不终止 (如无理数容量的 Zwick 反例)。

8.5 最大流最小割定理

定理 8.2 (最大流最小割定理). 下列三个条件等价:

1. f 是最大流。
2. 残量网络 G_f 中不存在 s - t 增广路。
3. 存在 s - t 割 (S, T) 使得 $|f| = c(S, T)$ 。

证明思路 (1) $\Rightarrow$ (2): 若有增广路, 还能增大流, 矛盾。(2) $\Rightarrow$ (3): 令 S 为残量图中从 s 可达的节点集。由于无增广路, $t \notin S$ 。对所有从 S 到 T 的原始边, 残量为 0, 即流量已满; 对从 T 到 S 的原始边, 反向残量为 0, 即原始流量为 0。因此 $|f| = \sum_{e: S \rightarrow T} c(e) - 0 = c(S, T)$ 。(3) $\Rightarrow$ (1): 由弱对偶 $|f| \leq c(S, T)$, 再加上等号成立, 得 f 最大。
 用上面的例子验证: $|f| = 4 = c(\{s, a, b\}, \{t\}) = c(a, t) + c(b, t) = 2 + 2 = 4$ 。等式成立, 与定理一致。

核心记忆 残量网络的反向边是 Ford-Fulkerson 能修正贪心错误的键; 最大流最小割定理是本章最重要的结论, 也是 LP 强对偶在图上的特例。

第九章 Ford–Fulkerson 的扩展

对应课件：09.pdf

主题：容量缩放、Edmonds–Karp、Dinic

为什么要改进 Ford–Fulkerson ? 原始 FF 没有指定增广路的选择策略。在最坏情况下，每次只增广 1 单位，若最大流值为 C ，则需要 C 次迭代，当 C 很大时极慢。本章介绍三种越来越聪明的选路策略，将复杂度从 $O(|E| \cdot C)$ 降低到与 C 几乎无关的水平。

9.1 容量缩放算法

直觉。先走”大路”，再走”小路”。高速公路先通大卡车（高容量路径），小路后来才承担剩余流量。我们维护一个阈值 Δ ，只在残量容量 $\geq \Delta$ 的边上增广；当找不到这样的路时，把 Δ 减半，继续。

CAPACITY-SCALING(G):

```
f = 0
Delta = 2floor(log2(C)) // 不超过最大容量 C 的最大 2 的幂
while Delta >= 1:
    // 只在残量容量 >= Delta 的边上寻找增广路
    while exists s-t path P in Gf(Delta):
        f = AUGMENT(f, P)
    Delta = Delta / 2
return f
```

定理 9.1. 容量缩放算法的时间复杂度为 $O(|E|^2 \log C)$ 。

证明思路 共有 $1 + \lceil \log_2 C \rceil$ 个阶段。关键引理：在 Δ -阶段结束时，最大流的剩余可增广量不超过 $|E| \cdot \Delta$ （因为此时残量图 $G_f(\Delta)$ 中已无 s - t 路径，取对应割，每条割边残量 $< \Delta$ ，至多 $|E|$ 条边）。故每阶段增广次数 $\leq 2|E|$ （每次至少增加 Δ ，且初始剩余量 $\leq |E| \cdot 2\Delta$ ）。每次增广 $O(|E|)$ ，总时间 $O(|E|^2 \log C)$ 。

9.2 Edmonds–Karp 算法

直觉。总是走最短增广路（边数最少）。关键性质是：随着增广进行，最短增广路的长度单调不减。直观上，增广只会饱和当前最短路上的边，而那些短路被封死后，只能走更长的路。

EDMONDS-KARP(G):

```
f = 0
while exists s-t path in Gf:
    P = BFS(Gf, s, t) // 找边数最少的路
    f = AUGMENT(f, P)
return f
```

定理 9.2. *Edmonds-Karp* 的增广次数为 $O(|V||E|)$, 总时间为 $O(|V||E|^2)$ 。

证明思路 引理 1: 最短增广路长度 $d_f(s, t)$ 单调不减——因为增广只添加反向边, 而反向边只能使距离增大。引理 2: 每条边成为瓶颈后, 若再次成为瓶颈, 其“尾点”到 s 的距离至少增加 2 (中间必须经历一次“进出”的往返)。由于距离最大为 $|V| - 1$, 每条边最多成为瓶颈 $O(|V|)$ 次, 共 $O(|V||E|)$ 次增广。

9.3 Dinic 算法

直觉. Edmonds-Karp 每次只走一条最短路。Dinic 更激进: 一次性把所有当前长度的最短增广路都处理掉, 这叫做求阻塞流。

定义 9.1 (层次图). 对残量网络 G_f 做 BFS, 令 $\ell(v)$ 为 s 到 v 的最短路边数。层次图 L_G 只保留满足 $\ell(w) = \ell(v) + 1$ 的边 (v, w) 。

定义 9.2 (阻塞流). 层次图 L_G 上的阻塞流是一个流, 使得 L_G 中每条从 s 到 t 的路径至少有一条边被饱和。

Dinic 算法每一阶段: (1); 用 BFS 建层次图; (2); 在层次图上求阻塞流 (DFS 前进/回退); (3); 更新残量图。

完整运行示例. 考虑如下网络:

$$s \xrightarrow{10} a, \quad s \xrightarrow{8} b, \quad a \xrightarrow{4} b, \quad a \xrightarrow{8} c, \quad b \xrightarrow{10} d, \quad c \xrightarrow{8} t, \quad d \xrightarrow{10} t.$$

初始流为 0。对残量网络做 BFS, 得到第一阶段的层次:

$$\ell(s) = 0, \quad \ell(a) = \ell(b) = 1, \quad \ell(c) = \ell(d) = 2, \quad \ell(t) = 3.$$

因此层次图只保留从第 i 层指向第 $i + 1$ 层的边:

$$s \rightarrow a, \quad s \rightarrow b, \quad a \rightarrow c, \quad b \rightarrow d, \quad c \rightarrow t, \quad d \rightarrow t.$$

注意原图中的边 $a \rightarrow b$ 不在层次图中, 因为 a, b 同层。

第一阶段: 求阻塞流。

1. 走路径 $s \rightarrow a \rightarrow c \rightarrow t$, 瓶颈为 $\min(10, 8, 8) = 8$, 送 8 单位流。
2. 走路径 $s \rightarrow b \rightarrow d \rightarrow t$, 瓶颈为 $\min(8, 10, 10) = 8$, 送 8 单位流。

此时层次图中所有 s - t 路径都被阻塞： $a \rightarrow c$ 已饱和， $s \rightarrow b$ 已饱和。第一阶段结束，当前流值

$$|f| = 8 + 8 = 16.$$

第二阶段：重新 BFS。残量网络中仍有从 s 到 t 的路，但最短路径变长了：

$$s \rightarrow a \rightarrow b \rightarrow d \rightarrow t.$$

对应层次为

$$\ell(s) = 0, \quad \ell(a) = 1, \quad \ell(b) = 2, \quad \ell(d) = 3, \quad \ell(t) = 4.$$

这条路的残量容量分别是

$$c_f(s, a) = 2, \quad c_f(a, b) = 4, \quad c_f(b, d) = 2, \quad c_f(d, t) = 2,$$

瓶颈为 2。增广 2 单位后，当前流值变为

$$|f| = 18.$$

终止与最优性验证。再次检查残量网络： $s \rightarrow a$ 和 $s \rightarrow b$ 都已无剩余容量，所以从 s 不能再到达其他点，残量网络中不存在增广路。取割

$$S = \{s\}, \quad T = \{a, b, c, d, t\},$$

其容量为

$$c(S, T) = c(s, a) + c(s, b) = 10 + 8 = 18 = |f|.$$

所以最终流是最大流。这个例子展示了 Dinic 的两个关键点：每个阶段处理完整个当前最短层次图；阶段结束后，最短增广路长度严格变长。

定理 9.3 (Dinic 1970). *Dinic* 算法的时间复杂度为 $O(|V|^2|E|)$ 。

证明思路 每个阶段 $O(|V||E|)$ ：建层次图 $O(|E|)$ ；每次增广删一条饱和边共 $O(|E|)$ ；每次回退删一个死节点共 $O(|V|)$ ；前进步骤总计 $O(|E||V|)$ 。阶段数 $O(|V|)$ ：每阶段后 $d_f(s, t)$ 严格增大（层次图上所有最短路径均被阻塞）。

说明 9.1. 在单位容量网络（如二分匹配）上，Dinic 算法时间降至 $O(|E||V|^{1/2})$ ，见第 10 章。

三种改进对比：

算法	核心思想	时间复杂度
容量缩放	先大后小，控制增广瓶颈	$O(E ^2 \log C)$
Edmonds-Karp	每次最短路径 (BFS)	$O(V E ^2)$
Dinic	层次图 + 阻塞流	$O(V ^2 E)$

核心记忆 三种改进都在控制“选哪条增广路”：容量缩放控制瓶颈大小，Edmonds-Karp 控制路径长度，Dinic 一次处理整层最短增广路，是工程中常用的实用算法。

第十章 单位容量网络与匹配

对应课件：10.pdf

主题：二分匹配、Hall 定理、不相交路径、Menger 定理

为什么学这一章？ 招聘： n 个岗位， m 个候选人，每人有意向岗位列表，问最多能完成几个配对？这是二分匹配问题。本章展示如何用最大流统一解决配对问题，并揭示其背后的组合结构——Hall 定理与 Menger 定理。

10.1 二分匹配的流建模

定义 10.1 (二分匹配). 给定二分图 $G = (L \cup R, E)$ ，匹配 $M \subseteq E$ 是一组边，使每个节点最多出现在一条边中。最大匹配是最大势的匹配。

如何转化为最大流？

构造新网络 $G' = (L \cup R \cup \{s, t\}, E')$:

1. 对每个 $u \in L$ ，加边 $s \rightarrow u$ ，容量 1（每个左节点最多匹配一次）。
2. 对每条 $(u, v) \in E$ ，加边 $u \rightarrow v$ ，容量 1。
3. 对每个 $v \in R$ ，加边 $v \rightarrow t$ ，容量 1（每个右节点最多匹配一次）。

完整示例 (3 + 3 二分图)。设左右两侧顶点和边集分别为

$$\begin{aligned} L &= \{A, B, C\}, & R &= \{X, Y, Z\}, \\ E &= \{(A, X), (A, Y), (B, Y), (B, Z), (C, Z)\}. \end{aligned}$$

构造后的流网络（所有容量为 1）:

$$\begin{aligned} s &\rightarrow A, & s &\rightarrow B, & s &\rightarrow C, \\ A &\rightarrow X, & A &\rightarrow Y, & B &\rightarrow Y, & B &\rightarrow Z, & C &\rightarrow Z, \\ X &\rightarrow t, & Y &\rightarrow t, & Z &\rightarrow t. \end{aligned}$$

运行最大流 (Ford-Fulkerson):

1. 增广路 $s \rightarrow A \rightarrow X \rightarrow t$ ，流值 +1。
2. 增广路 $s \rightarrow B \rightarrow Y \rightarrow t$ ，流值 +1。
3. 增广路 $s \rightarrow C \rightarrow Z \rightarrow t$ ，流值 +1。
4. 无更多增广路，终止， $|f| = 3$ 。

流量为 1 的 L - R 边： $\{(A, X), (B, Y), (C, Z)\}$ ——这就是大小为 3 的完美匹配！

定理 10.1. 大小为 k 的匹配与 G' 中值为 k 的整数流之间存在一一对应。

证明思路 容量约束保证每个左/右节点最多匹配一次；整数容量网络的 Ford–Fulkerson 产生整数流，因此 $f(u, v) = 1$ 的中间边恰好构成匹配。反向构造显然。

复杂度 用 Dinic 算法（单位容量网络的 $O(|E||V|^{1/2})$ 界），二分匹配可在 $O(|E||V|^{1/2})$ 时间内求解。

10.2 Hall 婚配定理

直觉。什么时候二分图有“覆盖左侧所有节点”的完美匹配？答案非常直观：对左侧任意一组节点 S ，它们的“认识”的右侧节点必须足够多（至少 $|S|$ 个），否则 S 中的某些节点一定没有对象。

定理 10.2 (Hall 婚配定理, Frobenius 1917, Hall 1935). 二分图 $G = (L \cup R, E)$ ($|L| = |R|$) 存在完美匹配，当且仅当对任意 $S \subseteq L$ ，有

$$|N(S)| \geq |S|,$$

其中 $N(S)$ 是 S 的所有邻居节点构成的集合。

证明思路 必要性($\Rightarrow$): S 中每个节点必须匹配到不同的 $N(S)$ 中的节点, 故 $|N(S)| \geq |S|$ 。充分性 ($\Leftarrow$): 反设最大流 $|f^*| < |L|$, 由最大流最小割定理存在容量 $< |L|$ 的割。设割 (A, B) , 令 $L_A = L \cap A$, $L_B = L \cap B$, $R_A = R \cap A$ 。割容量 $= |L_B| + |R_A| < |L| = |L_A| + |L_B|$, 故 $|R_A| < |L_A|$ 。又由于割不能切 ∞ 容量边, $N(L_A) \subseteq R_A$, 故 $|N(L_A)| \leq |R_A| < |L_A|$, 违反 Hall 条件, 矛盾。

10.3 单位容量网络

定义 10.2 (简单单位容量网络). 流网络中每条边容量为 1, 且除 s, t 外每个节点要么只有一条入边, 要么只有一条出边 (或两者都满足)。

定理 10.3 (Even–Tarjan 1975). 在简单单位容量网络上, *Dinic* 算法的时间复杂度为 $O(|E||V|^{1/2})$ 。

证明思路 经过 $|V|^{1/2}$ 个阶段后, 最短增广路长度 $> |V|^{1/2}$, 层次图至少有 $|V|^{1/2}$ 层, 由鸽巢原理存在某层节点数 $\leq |V|^{1/2}$, 以该层为割, 剩余可增广量 $\leq |V|^{1/2}$ 。每次增广 $+1$, 再增广 $|V|^{1/2}$ 次即可达到最优。总计: 前 $|V|^{1/2}$ 阶段每阶段 $O(|E|)$, 后 $|V|^{1/2}$ 次增广每次 $O(|E|)$, 共 $O(|E||V|^{1/2})$ 。

10.4 边不相交路径与 Menger 定理

直觉。网络中最多能同时走几条互不共享边的 s - t 路径? (如: k 条独立的通信链路) 直觉上, 这个数字等于“最少需要删除多少条边才能彻底切断 s 到 t 的联系”。

定义 10.3 (边不相交路径). 若两条路径没有公共边, 则它们边不相交。

流建模：将所有边容量设为 1，最大流值即为最大边不相交路径数。

定理 10.4 (Menger 定理，边版，1927). 从 s 到 t 的最大边不相交路径数，等于使 s, t 断联所需删除的最少边数。

证明思路 这正是单位容量流网络上的最大流最小割定理。最大流 = 最大不相交路径数；最小割 = 最少删除边数。

说明 10.1. Menger 定理还有点版（将每个节点拆成入节点和出节点，容量为 1），描述顶点不相交路径与最少删点数的等价关系。

核心记忆 最大流是求解二分匹配、不相交路径、网络连通性等组合问题的统一工具。”单位容量”使问题具有更多结构，使 Dinic 算法的复杂度大幅改善。

第十一章 线性规划导论

对应课件：11.pdf

主题：LP 建模、几何直觉、对偶、互补松弛

为什么学线性规划？工厂生产两种产品，每种产品消耗不同原料，利润也不同，原料总量有限——如何分配生产量使利润最大？这类“在线性约束下优化线性目标”的问题无处不在，统称线性规划（LP）。更重要的是，LP 是后续近似算法、对偶分析、最大流证明的通用语言。

11.1 线性规划模型

定义 11.1 (线性规划). 线性规划是在一组线性约束下优化线性目标函数的问题。标准最大化形式：

$$\max c^T x, \quad \text{s.t.} \quad Ax \leq b, \quad x \geq 0.$$

其中 $x \in \mathbb{R}^n$ 是决策变量， $c \in \mathbb{R}^n$ 是目标系数， $A \in \mathbb{R}^{m \times n}$ ， $b \in \mathbb{R}^m$ 。

直觉（2 变量例子）。以具体工厂问题说明：

- 工厂生产两种产品 x_1 （普通型）和 x_2 （豪华型）。
- 每箱 x_1 利润 \$3，每箱 x_2 利润 \$5。
- 约束： $x_1 \leq 4$ （ x_1 需求上限）， $2x_2 \leq 12$ （ x_2 产能上限）， $3x_1 + 5x_2 \leq 15$ （原料总量）， $x_1, x_2 \geq 0$ 。

LP 写出：

$$\max 3x_1 + 5x_2, \quad x_1 \leq 4, \quad 2x_2 \leq 12, \quad 3x_1 + 5x_2 \leq 15, \quad x_1, x_2 \geq 0.$$

11.2 几何直觉：可行域与最优顶点

可行域是凸多面体。2 变量 LP 中，每条线性不等式在平面上定义一个半平面，可行域是所有半平面的交——一个凸多边形。

目标函数等高线。目标 $3x_1 + 5x_2 = c$ 是一条斜率为 $-3/5$ 的直线。当 c 增大时，这条线向右平移。我们要找 c 最大、且直线仍与可行域有交点时的情况。

最优解在顶点。等高线最后接触可行域的点必然是可行域的顶点（多边形的角）。这是凸集的关键性质——目标函数在凸集上的最优值必在极点处取到。

枚举顶点（完整例子）。可行域由若干半平面围成。顶点来自两个约束边界的交点，但不是每个交点都可行，必须代回所有约束检查。对本例，

$$x_1 \leq 4, \quad x_2 \leq 6, \quad 3x_1 + 5x_2 \leq 15, \quad x_1, x_2 \geq 0.$$

约束交点	x_1	x_2	结论
$x_1 = 0, x_2 = 0$	0	0	可行, 目标 0
$x_1 = 4, x_2 = 0$	4	0	可行, 目标 12
$x_1 = 4, 3x_1 + 5x_2 = 15$	4	3/5	可行, 目标 15
$x_1 = 0, 3x_1 + 5x_2 = 15$	0	3	可行, 目标 15
$x_2 = 6, 3x_1 + 5x_2 = 15$	-5	6	$x_1 < 0$, 不可行
$x_1 = 0, x_2 = 6$	0	6	违反 $3x_1 + 5x_2 \leq 15$

可行顶点为 $(0,0)$ 、 $(4,0)$ 、 $(4,3/5)$ 、 $(0,3)$ 。最大目标值为 15，在整条线段

$$3x_1 + 5x_2 = 15, \quad 0 \leq x_1 \leq 4$$

上的可行点都能取到。特别地， $(0,3)$ 与 $(4,3/5)$ 都是最优解。这说明 LP 最优解不一定唯一，但若存在最优解，总能找到一个最优顶点。

说明 11.1. LP 最优解若存在，必在某个顶点（极点）处取到。例外情况：问题不可行（约束矛盾），或无界（约束太松，目标可任意大）。

11.3 建模方法论

建 LP 模型时依次考虑四个问题：

1. **决策变量**：想要优化的量是什么？（生产量、流量、价格……）
2. **目标函数**：最大化利润、最小化成本？写成决策变量的线性组合。
3. **约束**：每个资源限制、需求限制、逻辑限制如何写成线性不等式？
4. **整数性**：变量是否必须为整数？若是，则为整数规划（NP 难，通常难得多）。

11.4 对偶

直觉。原 LP 是工厂老板视角：如何分配产量使利润最大？对偶 LP 是资源购买商视角：如何给工厂的每种原料定价，使得“按这个价格买资源”足以覆盖每种产品的利润，同时总购买成本最小？

对原 LP: $\max c^T x, Ax \leq b, x \geq 0$ ，构造对偶 LP:

$$\min b^T y, \quad A^T y \geq c, \quad y \geq 0.$$

其中对偶变量 y_i 可理解为第 i 个约束对应资源的“影子价格”。

对偶构造过程（以本章例子说明）。原问题: $\max 3x_1 + 5x_2$, 约束 $x_1 \leq 4$ (对应 y_1), $2x_2 \leq 12$ (对应 y_2), $3x_1 + 5x_2 \leq 15$ (对应 y_3)。

用 $y_1, y_2, y_3 \geq 0$ 乘各约束相加: $(y_1 + 3y_3)x_1 + (2y_2 + 5y_3)x_2 \leq 4y_1 + 12y_2 + 15y_3$ 。

要使右侧成为目标函数 $3x_1 + 5x_2$ 的上界，需要左侧系数不小于目标系数：

$$y_1 + 3y_3 \geq 3, \quad 2y_2 + 5y_3 \geq 5.$$

因此对偶问题为

$$\begin{aligned} \min \quad & 4y_1 + 12y_2 + 15y_3 \\ \text{s.t.} \quad & y_1 + 3y_3 \geq 3, \quad 2y_2 + 5y_3 \geq 5, \quad y_1, y_2, y_3 \geq 0. \end{aligned}$$

定理 11.1 (弱对偶). 对任意原可行 x 和对偶可行 y , 有 $c^T x \leq b^T y$.

定理 11.2 (强对偶). 若原问题有有界最优解, 则对偶也有最优解, 且 $\max c^T x = \min b^T y$.

用本章例子验证强对偶. 原问题最优: $(x_1^*, x_2^*) = (0, 3)$, 目标值 15. 对偶问题: $\min 4y_1 + 12y_2 + 15y_3$, 约束 $y_1 + 3y_3 \geq 3$, $2y_2 + 5y_3 \geq 5$. 取 $y_1^* = 0$, $y_2^* = 0$, $y_3^* = 1$: 验证 $0 + 3 = 3 \geq 3\checkmark$, $0 + 5 = 5 \geq 5\checkmark$, 目标值 $0 + 0 + 15 = 15$. 原问题目标值 = 对偶目标值 = 15, 强对偶定理验证。

11.5 互补松弛

直觉. 互补松弛是最优性的充要条件:

- “没有用完的资源不值钱”: 若某约束还有剩余 (不紧), 则对应的对偶变量 (影子价格) 为 0。
- “没有生产的产品不带来边际收益”: 若某决策变量为 0, 则对应的对偶约束可以有松弛 (即影子定价超过实际盈利也没关系)。

定理 11.3 (互补松弛条件). x^*, y^* 分别是原/对偶可行解, 它们同时最优当且仅当:

$$y_i^*(b_i - (Ax^*)_i) = 0 \quad \forall i, \quad x_j^*((A^T y^*)_j - c_j) = 0 \quad \forall j.$$

用本章例子验证互补松弛. $(x_1^*, x_2^*) = (0, 3)$, $(y_1^*, y_2^*, y_3^*) = (0, 0, 1)$:

- 约束 1 ($x_1 \leq 4$): $y_1^* = 0$, $b_1 - (Ax^*)_1 = 4 - 0 = 4 \neq 0$, 但 $y_1^* \cdot 4 = 0$, 满足。
- 约束 2 ($2x_2 \leq 12$): $y_2^* = 0$, $b_2 - (Ax^*)_2 = 12 - 6 = 6 \neq 0$, 但 $y_2^* \cdot 6 = 0$, 满足。
- 约束 3 ($3x_1 + 5x_2 \leq 15$): $y_3^* = 1 \neq 0$, 故需 $b_3 - (Ax^*)_3 = 15 - 15 = 0$, 满足。
- 变量 $x_1^* = 0$: 对偶约束 1 为 $y_1^* + 3y_3^* = 0 + 3 = 3 \geq 3$, 可以有松弛, 满足。
- 变量 $x_2^* = 3 > 0$: 对偶约束 2 需为紧, $2y_2^* + 5y_3^* = 0 + 5 = 5 = 5$, 满足。

核心记忆 LP 对偶是理解最大流最小割、集合覆盖松弛、原始-对偶近似算法的通用框架。互补松弛给出了一种验证解是否最优的“证书”。

第十二章 单纯形算法

对应课件：12.pdf

主题：标准型、字典、pivot、退化与复杂度

为什么学单纯形？ LP 几何上是在凸多面体中找最优点。单纯形算法的思想是：从一个顶点出发，沿着多面体的”边”走到相邻更好的顶点，直到没有更好的邻居——局部最优即全局最优（因为可行域是凸的）。这就是为什么”沿着多边形的边爬坡”能找到全局最优。

12.1 标准型与松弛变量

把不等式变等式。 引入松弛变量，类似于最大流中的”剩余容量”：约束 $a_1x_1 + a_2x_2 \leq b$ 变为 $a_1x_1 + a_2x_2 + s = b, s \geq 0$ (s 是”剩余量”)。

以第 11 章的工厂 LP 为例：

$$\max 3x_1 + 5x_2, \quad x_1 \leq 4, \quad 2x_2 \leq 12, \quad 3x_1 + 5x_2 \leq 15, \quad x_1, x_2 \geq 0.$$

引入松弛变量 s_1, s_2, s_3 ，标准型为：

$$\max 3x_1 + 5x_2, \quad x_1 + s_1 = 4, \quad 2x_2 + s_2 = 12, \quad 3x_1 + 5x_2 + s_3 = 15, \quad x_1, x_2, s_1, s_2, s_3 \geq 0.$$

定义 12.1 (基可行解). 给定标准型 LP，基是一组 m 个变量 (m 为等式约束数)，使得对应的系数矩阵可逆。将非基变量设为 0，解出基变量，得到基可行解。几何上对应多面体的一个顶点。

初始顶点： 令 $x_1 = x_2 = 0$ ，则 $s_1 = 4, s_2 = 12, s_3 = 15$ (均非负，可行)。初始基 $= \{s_1, s_2, s_3\}$ ，非基 $= \{x_1, x_2\}$ ，目标值 $= 0$ 。

12.2 字典表示与单纯形 Pivot

字典 (Dictionary) 是单纯形算法的核心数据结构，将基变量表示为非基变量的线性函数：

$$\text{目标} = [\text{当前值}] + [\text{非基变量的系数}]$$

初始字典 (基 $= \{s_1, s_2, s_3\}$ ，非基 $= \{x_1, x_2\}$):

$$s_1 = 4 - x_1$$

$$s_2 = 12 - 2x_2$$

$$s_3 = 15 - 3x_1 - 5x_2$$

$$z = 0 + 3x_1 + 5x_2$$

Pivot 步骤 (迭代 1):

1. **选进基变量**: x_2 的系数为 +5 (最大正系数), 选 x_2 进基。

2. **比值测试 (选离基变量)**:

- s_1 : x_2 系数为 0, 无约束 (不限制 x_2 增大)。
- $s_2 = 12 - 2x_2 \geq 0$: $x_2 \leq 12/2 = 6$ 。
- $s_3 = 15 - 5x_2 \geq 0$: $x_2 \leq 15/5 = 3$ 。

最小比值: $\min(6, 3) = 3$, 对应 s_3 离基, $x_2 = 3$ 。

3. **Pivot**: 从 $s_3 = 15 - 3x_1 - 5x_2$ 解出 x_2 : $x_2 = 3 - \frac{3}{5}x_1 - \frac{1}{5}s_3$ 。代入其他式:

$$\begin{aligned} s_1 &= 4 - x_1 \\ s_2 &= 12 - 2\left(3 - \frac{3}{5}x_1 - \frac{1}{5}s_3\right) = 6 + \frac{6}{5}x_1 + \frac{2}{5}s_3 \\ x_2 &= 3 - \frac{3}{5}x_1 - \frac{1}{5}s_3 \\ z &= 0 + 3x_1 + 5\left(3 - \frac{3}{5}x_1 - \frac{1}{5}s_3\right) = 15 + 0 \cdot x_1 - s_3 \end{aligned}$$

迭代 1 后的字典 (基 = $\{s_1, s_2, x_2\}$, 非基 = $\{x_1, s_3\}$):

$$\begin{aligned} s_1 &= 4 - x_1 \\ s_2 &= 6 + \frac{6}{5}x_1 + \frac{2}{5}s_3 \\ x_2 &= 3 - \frac{3}{5}x_1 - \frac{1}{5}s_3 \\ z &= 15 + 0 \cdot x_1 - s_3 \end{aligned}$$

终止检查: 目标函数中 x_1 系数为 0, s_3 系数为 $-1 < 0$ 。所有非基变量系数 ≤ 0 , 无改善方向, **当前顶点最优**。最优解: 令非基变量 $x_1 = 0$, $s_3 = 0$, 代入得 $x_2 = 3$, $s_1 = 4$, $s_2 = 6$, 目标值 $z = 15$ 。

对应几何: 只经过 1 次 pivot, 从顶点 $(0, 0)$ ($z = 0$) 直接跳到顶点 $(0, 3)$ ($z = 15$)!

12.3 单纯形算法完整描述

```
SIMPLEX(A, b, c):
// 初始化字典 (假设原点可行)
初始化基变量 = 松弛变量, 非基变量 = 原始变量
loop:
  if 目标行中无正系数:
    return 当前基可行解 (最优)
  选择目标行中系数最大的非基变量 e 进基
  对每个基变量做比值测试 (系数>0 时的比值)
  if 无限小比值:
    return 无界 (unbounded)
  选最小比值对应基变量 l 离基
  pivot: 消元更新字典
```


12.4 初始顶点: Phase I

若原点 ($x = 0$) 不可行 (即 b 中有负分量), 需先求一个可行初始顶点。方法: 构造辅助 LP, 引入人工变量 $z_i \geq 0$, 最小化 $\sum z_i$:

- 若最优值为 0 (所有 $z_i = 0$), 得到原问题的可行初始基。
- 若最优值 > 0 , 原问题不可行。

12.5 退化与 Bland 规则

定义 12.2 (退化). 若基可行解中某基变量值为 0, 则称顶点退化。Pivot 后目标值可能不增, 极端情况下可能循环。

Bland 规则 (防循环): 总选下标最小的改善方向进基; 当有多个变量可离基时, 选下标最小的。可证明在 Bland 规则下单纯形不会循环。

12.6 复杂度

定理 12.1. 单纯形算法在最坏情况下可能需要指数步 (*Klee-Minty* 立方体是经典反例)。

Klee-Minty 直觉: 构造一个”变形超立方体”, 使单纯形沿某种 pivot 规则需要遍历所有 2^n 个顶点。这说明单纯形对特定 pivot 规则不是多项式算法。

易错点 ”LP 可多项式求解” (椭球法 1979, 内点法 1984) 与”单纯形最坏多项式”是不同的命题。前者为真, 后者对经典 pivot 规则一般不成立。但实践中单纯形极快, 平滑分析 (Spielman-Teng 2004) 解释了这一现象。

核心记忆 单纯形是”沿多面体的棱爬坡”的直观算法; 字典 (Dictionary) 表示使每次 pivot 的代价为 $O((m+n)n)$; 实践中单纯形远比理论最坏界快。

第十三章 线性规划的应用

对应课件：13.pdf

主题：最短路、最大流、整数规划、集合覆盖

为什么学 LP 应用？ LP 是组合优化问题的“万能建模语言”：最短路、最大流、匹配……都可以写成 LP，从而统一地研究它们的结构和近似算法。LP 松弛 (relaxation) 则是设计近似算法的标准手法：先解更容易的 LP，再通过舍入得到整数解。

13.1 最短路 LP 建模

直觉。把最短路问题想象成“从 s 发出 1 单位水，水流沿边传播，希望最终到达 t 的费用最小”。每条边 e 的“开阀量” $x_e \geq 0$ 表示流经该边的“水量”，流守恒约束保证水不消失，费用为 $\sum c_e x_e$ 。

流式 LP 写法：

$$\min \sum_{e \in E} c_e x_e$$

满足流守恒： s 净流出 1， t 净流入 1，其他节点 v 净流量为 0， $x_e \geq 0$ 。即：

$$\begin{aligned} \sum_{e \text{ 出 } s} x_e - \sum_{e \text{ 入 } s} x_e &= 1, \\ \sum_{e \text{ 入 } v} x_e - \sum_{e \text{ 出 } v} x_e &= 0, \quad v \neq s, t, \\ \sum_{e \text{ 入 } t} x_e - \sum_{e \text{ 出 } t} x_e &= -1. \end{aligned}$$

由于网络矩阵全幺模 (*totally unimodular*)，最优基本解自动为整数，对应一条 s - t 路径 (或若干路径的极点组合)。

对偶写法 (势函数视角)：

$$\max d_t - d_s, \quad d_v - d_u \leq c_{uv} \quad (\forall (u, v) \in E), \quad d_s = 0.$$

其中 d_v 是节点 v 的“势” (距离标签)，这与 Dijkstra/Bellman-Ford 中的距离标签完全一致！强对偶定理告诉我们：最短路长度 = 最大势差，这与 Dijkstra 的正确性证明是同一件事。

13.2 最大流的 LP 与对偶

最大流 LP 形式化。引入从 t 到 s 的虚拟边，将最大流转为循环：

$$\begin{aligned}
& \max f_{ts} \\
& f_{ij} \leq c_{ij}, \quad (i, j) \in E, \\
& \sum_{(w,i) \in E} f_{wi} - \sum_{(i,z) \in E} f_{iz} = 0, \quad i \in V, \\
& f_{ij} \geq 0.
\end{aligned}$$

对偶的含义——最小割。引入对偶变量： $d_{ij} \geq 0$ （对应容量约束）， p_i （对应流守恒约束）。对偶 LP 为：

$$\min \sum_{(i,j) \in E} c_{ij} d_{ij}, \quad d_{ij} - p_i + p_j \geq 0 \ ((i, j) \in E), \quad p_s - p_t \geq 1, \quad d_{ij}, p_i \geq 0.$$

整数化 ($d_{ij}, p_i \in \{0, 1\}$): $p_s = 1, p_t = 0, d_{ij} = 1$ 当且仅当 (i, j) 跨越割 ($\{p = 1\}, \{p = 0\}$)。这正是最小 s - t 割问题！因此：

LP 强对偶 $\Rightarrow$ 最大流最小割定理

这给出了最大流最小割定理的另一种证明：通过 LP 对偶，而非增广路论证。

13.3 整数规划与 LP 松弛

直觉。许多组合优化问题自然写成 0-1 整数规划 (ILP)，但整数规划通常是 NP 难的。LP 松弛的思路：把 $x_i \in \{0, 1\}$ 放宽为 $0 \leq x_i \leq 1$ ，得到更容易求解的 LP。

- 松弛最优值对**最小化**问题是**下界**（放宽约束，只会让最优值更小或不变）。
- 松弛最优值对**最大化**问题是**上界**。
- 若松弛解天然整数（如全幺模矩阵），则得到精确算法（最短路、最大流、最大匹配均如此）。
- 若松弛解分数，可通过舍入、随机化或原始-对偶方法得到近似算法，并用松弛值来分析近似比。

13.4 集合覆盖的 ILP、LP 松弛与对偶

定义 13.1 (集合覆盖). 给定全集 U ($|U| = n$) 和子集族 $\mathcal{S} = \{S_1, \dots, S_m\}$ (每个 $S_i \subseteq U$)，选择最少数量的子集使其并集包含 U 的所有元素。

整数规划 (ILP) 写法：决策变量 $x_S \in \{0, 1\}$ (1 表示选集合 S):

$$\min \sum_{S \in \mathcal{S}} x_S, \quad \sum_{S: e \in S} x_S \geq 1 \ (\forall e \in U), \quad x_S \in \{0, 1\}.$$

LP 松弛：将整数约束放宽为 $x_S \geq 0$ （上界 $x_S \leq 1$ 在最小化中自动满足）：

$$\min \sum_{S \in \mathcal{S}} x_S, \quad \sum_{S: e \in S} x_S \geq 1 \ (\forall e \in U), \quad x_S \geq 0.$$

对偶 LP：对偶变量 $y_e \geq 0$ （对应元素 e 的覆盖约束）：

$$\max \sum_{e \in U} y_e, \quad \sum_{e \in S} y_e \leq 1 \ (\forall S \in \mathcal{S}), \quad y_e \geq 0.$$

直觉： y_e 是元素 e 的“价值”，每个集合的总价值不超过 1（选它的代价），最大化所有元素的总价值——这是集合覆盖的分数打包问题（set packing 的松弛）。

对偶的应用：

- **分析贪心算法：**每次选覆盖最多未覆盖元素的集合，其近似比 $= O(\ln n)$ 。用对偶变量构造下界，精确分析贪心的近似比。
- **原始-对偶（Primal-Dual）近似算法：**同时维护原始解和对偶解，通过互补松弛条件设计近似算法，并用对偶界证明近似比。
- **Dual Fitting 方法：**构造满足某个比例的对偶可行解，从而间接分析原始（整数）贪心解的质量。

含代价的集合覆盖（推广）：若集合 S 有代价 $c(S)$ ，ILP 变为：

$$\min \sum_S c(S)x_S, \quad \sum_{S: e \in S} x_S \geq 1, \quad x_S \in \{0, 1\}.$$

对偶为： $\max \sum_e y_e, \sum_{e \in S} y_e \leq c(S) \ (\forall S), y_e \geq 0$ 。

核心记忆 LP 应用的核心链条：组合问题 $\rightarrow$ ILP 建模 $\rightarrow$ LP 松弛（连续化） $\rightarrow$ 对偶（理解结构） $\rightarrow$ 近似算法（整数化）。全幺模矩阵保证某些 LP 自动有整数最优解，不需要舍入。

易错点 LP 松弛解不一定是整数，舍入可能破坏可行性或使近似比变差。需要专门的舍入技术（如随机舍入、贪心舍入）并分析损失。

第十五章 多项式归约与经典困难问题

对应课件：15.pdf

主题：覆盖、SAT、Hamilton、Subset Sum、Knapsack

动机：为什么学归约？

有些问题也许根本没有高效算法——不是我们还没想到，而是它本质上就很难。归约是证明这一点的工具。它的核心逻辑是：**如果问题 A 能”变换”成问题 B ，那么 B 至少和 A 一样难。**

一个日常比喻：假设你知道”翻译古希腊文”很难。现在有人给你一道题，你发现只要会翻译古希腊文就能解这道题，那这道题至少和翻译古希腊文一样难。这就是归约的精神。

易错点 归约的方向极容易写反！我们说” A 归约到 B ” ($A \leq_p B$)，意思是把 A 的实例变成 B 的实例，用 B 的求解器来解 A 。因此：若 B 容易，则 A 也容易；若 A 难，则 B 也难。新手最常犯的错：想证明 B 难，却把 B 归约到 A ，方向完全相反。

15.1 多项式时间归约

定义 15.1. 问题 A **多项式时间归约**到问题 B ，记作 $A \leq_p B$ ，表示存在多项式时间可计算的变换 f ，使得对任意输入 x ：

$$x \in A \iff f(x) \in B.$$

归约有三大用途：

1. **设计算法**：若 $A \leq_p B$ 且 B 有多项式算法，则 A 也有多项式算法。
2. **证明困难性**：若 $A \leq_p B$ 且 A 没有多项式算法（或被认为没有），则 B 也没有。
3. **证明等价**：若 $A \leq_p B$ 且 $B \leq_p A$ ，则两问题同样难，记 $A \equiv_p B$ 。

核心记忆 写一个正确的归约必须说明三件事：(1) 构造 f 是多项式时间的；(2) 若原实例是 yes，则 f 的输出也是 yes；(3) 若 f 的输出是 yes，则原实例也是 yes。缺一不可。

15.2 覆盖与打包: $\text{Independent Set} \equiv_p \text{Vertex Cover}$

15.2.1 直觉

独立集 (Independent Set, IS) 问你能否找到 k 个互不相邻的顶点; 顶点覆盖 (Vertex Cover, VC) 问你能否用 k 个顶点覆盖所有边。乍一看这是两个不同的问题, 但其实它们是**同一件事的两面**: 选了独立集, 剩下的顶点就构成顶点覆盖。

定理 15.1. $\text{Independent Set} \equiv_p \text{Vertex Cover}$ 。具体地: S 是 G 的大小为 k 的独立集, 当且仅当 $V \setminus S$ 是大小为 $|V| - k$ 的顶点覆盖。

证明. ($\Rightarrow$) 设 S 是独立集。对任意边 $(u, v) \in E$, 因为 S 是独立集, u, v 不能同时在 S 中, 故至少一个在 $V \setminus S$ 中, 边被覆盖。

($\Leftarrow$) 设 $V \setminus S$ 是顶点覆盖。对任意边 (u, v) , 至少一个端点在 $V \setminus S$ 中, 故 u, v 不能同时在 S 中, S 是独立集。□

15.2.2 5 节点图的具体例子

考虑如下 5 节点图:

$$V = \{1, 2, 3, 4, 5\}, \quad E = \{(1, 2), (1, 3), (2, 3), (3, 4), (4, 5)\}.$$

找大小为 2 的独立集: 取 $S = \{2, 5\}$ 。检验: $(2, 5)$ 不是边, 2 与 5 互不相邻, 合法。对应的顶点覆盖是 $V \setminus S = \{1, 3, 4\}$, 大小为 3。验证:

- $(1, 2)$: 端点 1 在覆盖中 ✓
- $(1, 3)$: 端点 1 在覆盖中 ✓
- $(2, 3)$: 端点 3 在覆盖中 ✓
- $(3, 4)$: 端点 3 和 4 均在覆盖中 ✓
- $(4, 5)$: 端点 4 在覆盖中 ✓

所有边均被覆盖, $\{1, 3, 4\}$ 确实是顶点覆盖。

15.3 $3\text{SAT} \leq_p \text{Independent Set}$ (本章最重要归约!)

15.3.1 直觉

3SAT 问: 给定 CNF, 每个子句有 3 个字面量, 有没有满足赋值? Independent Set 问: 图中有没有 k 个互不相邻的顶点?

归约的核心想法是: **把每个子句变成一个三角形** (三个字面量顶点) —— 每个子句只能选一个满足的字面量, 三角形保证选多了就相邻了; **再在互补字面量之间连边** —— x_i 和 $\neg x_i$ 不能同时选, 这防止了矛盾赋值。

15.3.2 构造

给定 3SAT 公式 φ 有子句 $C_1, \dots, C_m$, 每个子句有 3 个字面量:

- 对每个子句 $C_j = (\ell_{j,1} \vee \ell_{j,2} \vee \ell_{j,3})$, 建立三个顶点 $v_{j,1}, v_{j,2}, v_{j,3}$, 并在三者之间连边 (三角形)。
- 若 $\ell_{j,r}$ 与 $\ell_{k,s}$ 互补 (一个是 x_i , 另一个是 $\neg x_i$), 则在 $v_{j,r}$ 与 $v_{k,s}$ 之间连边。

问: 图中是否有大小为 m (子句数) 的独立集?

15.3.3 完整例子: 3 个子句

公式:

$$\varphi = \underbrace{(x_1 \vee \neg x_2 \vee x_3)}_{C_1} \wedge \underbrace{(\neg x_1 \vee x_2 \vee x_3)}_{C_2} \wedge \underbrace{(x_1 \vee x_2 \vee \neg x_3)}_{C_3}.$$

顶点:

$$C_1 \rightarrow v_{1,1}(x_1), v_{1,2}(\neg x_2), v_{1,3}(x_3)$$

$$C_2 \rightarrow v_{2,1}(\neg x_1), v_{2,2}(x_2), v_{2,3}(x_3)$$

$$C_3 \rightarrow v_{3,1}(x_1), v_{3,2}(x_2), v_{3,3}(\neg x_3)$$

三角形内部边 (子句内): 每个子句内 3 条, 共 9 条。

互补连边 (跨子句, 仅列举关键几对):

- $v_{1,1}(x_1) \leftrightarrow v_{2,1}(\neg x_1)$
- $v_{1,2}(\neg x_2) \leftrightarrow v_{2,2}(x_2), v_{1,2}(\neg x_2) \leftrightarrow v_{3,2}(x_2)$
- $v_{1,3}(x_3) \leftrightarrow v_{3,3}(\neg x_3), v_{2,3}(x_3) \leftrightarrow v_{3,3}(\neg x_3)$
- $v_{3,1}(x_1) \leftrightarrow v_{2,1}(\neg x_1)$

验证 yes $\rightarrow$ yes: 令 $x_1 = \top, x_2 = \top, x_3 = \top$, 则 C_1 由 x_1 满足, C_2 由 x_2 满足, C_3 由 x_1 满足。选 $v_{1,1}(x_1), v_{2,2}(x_2), v_{3,1}(x_1)$:

- $v_{1,1}$ 与 $v_{2,2}$: 不互补 ($x_1 \neq \neg x_2$), 且来自不同子句, 不在同一三角形, 无边。
- $v_{1,1}$ 与 $v_{3,1}$: 同为 x_1 , 不互补, 无互补边; 来自不同子句, 无三角形边。
- $v_{2,2}$ 与 $v_{3,1}$: $x_2 \neq \neg x_1$, 无互补边; 来自不同子句, 无三角形边。

三点互不相邻, 构成大小为 3 的独立集。✓

验证 yes $\leftarrow$ yes (反向): 若图中存在大小为 m 的独立集 I , 则由于每个子句内部是三角形, I 在每个子句三角形中最多选一个顶点。共有 m 个子句且 $|I| = m$, 所以它恰好从每个子句选出一个字面量。又因为所有互补字面量之间都连边, 独立集不可能同时包含 x_i 和 $\neg x_i$ 。于是把被选中的正文字对应变量设为真、负文字对应变量设为假; 未出现的变量任意赋值。该赋值不会矛盾, 并且每个子句至少有一个被选中的字面量为真, 因此原公式可满足。

15.4 Vertex Cover $\leq_p$ Set Cover

把每条边看作待覆盖的元素，每个顶点 v 对应其 incident 边集合 S_v 。选择顶点覆盖所有边等价于选择集合覆盖所有元素。归约多项式，等价性显然。

15.5 SAT $\leq_p$ 3SAT

任意子句 $(\ell_1 \vee \cdots \vee \ell_k)$ 可引入辅助变量链式拆分：

$$(\ell_1 \vee \ell_2 \vee y_1) \wedge (\neg y_1 \vee \ell_3 \vee y_2) \wedge \cdots \wedge (\neg y_{k-3} \vee \ell_{k-1} \vee \ell_k).$$

长度 $k \geq 4$ 的子句变成 $k - 2$ 个长度为 3 的子句，变量数线性增长。长度为 1 或 2 的子句用“填充”处理（重复文字）。

15.6 Hamilton Cycle $\leq_p$ TSP

直觉：把有向图的存在性问题变成带权图的优化问题。

构造：给定图 $G = (V, E)$ ，建立完全图 K_n ：

$$w(u, v) = \begin{cases} 1 & (u, v) \in E \\ 2 & (u, v) \notin E \end{cases}$$

G 有 Hamilton cycle $\iff K_n$ 中存在权重恰好为 n 的 TSP tour。（任何使用非原图边的 tour 权重至少为 $n + 1$ 。）

15.7 Subset Sum $\leq_p$ Knapsack

Subset Sum：给定整数集 $\{a_1, \dots, a_n\}$ 和目标 T ，问是否有子集和恰为 T ？

嵌入 Knapsack：令物品 i 的重量 $w_i = a_i$ ，价值 $v_i = a_i$ ，背包容量 $W = T$ ，目标价值 $P = T$ 。则 Subset Sum 有解 $\iff$ Knapsack 能装到价值恰为 T （重量也恰为 T ）。

第十六章 复杂性类

对应课件：16.pdf

主题：P、NP、NP-complete、co-NP、NP-hard

动机：给问题的难度分类

多项式归约给了我们比较问题难度的工具，但我们还需要”大分类”来描述整个计算世界的结构。P、NP、NP-complete 就是这样的分类框架。理解它们，就理解了”什么能快速解决、什么可能永远无法快速解决”这一计算的根本边界。

16.1 P 类

定义 16.1. P 是可由确定性图灵机在**多项式时间**内求解的决策问题类。

直觉：P 就是”能快速解决的问题”。典型例子：

- 排序 ($O(n \log n)$)
- 最短路 (Dijkstra, $O(m \log n)$)
- 最大流 ($O(m^2)$ 或更快)
- 线性规划 (多项式时间)
- 素性测试 (AKS 算法, 2002 年证明在 P 中)

16.2 NP 类

定义 16.2. NP 是 *yes* 实例有**多项式长度证书**，且证书可在多项式时间内验证的决策问题类。

直觉：NP 是”答案容易验证但可能很难找到”的问题。

用**数独**来比喻：解一道数独可能需要很长时间，但如果有人给你一个填好的答案，你只需逐行逐列逐宫格检查，几分钟就能验证对不对。SAT 也是如此：找一个满足赋值可能很难，但给你一个赋值，代入每个子句检查一遍就能验证。

典型 NP 问题及其证书：

- **SAT**：证书 = 变量赋值（可直接验证每个子句是否被满足）
- **Hamilton Cycle**：证书 = 顶点排列（验证相邻顶点间有边，且包含所有顶点）

- **3-Coloring**: 证书 = 每个顶点的颜色 (验证每条边两端颜色不同)
- **Subset Sum**: 证书 = 子集本身 (验证和等于目标值)

说明 16.1. 显然 $P \subseteq NP$: 若能在多项式时间内解决, 当然也能在多项式时间内验证 (直接运行算法)。

16.3 NP-complete

定义 16.3. 问题 B 是 *NP-complete* (*NPC*) 若:

1. $B \in NP$;
2. 对任意 $A \in NP$, 有 $A \leq_p B$ (B 是 NP 中最难的一批)。

直觉: NPC 问题是 NP 中“最难的一批”——若你能高效解决任何一个 NPC 问题, 你就能高效解决 NP 中所有问题 (即 $P = NP$)。

16.3.1 Cook-Levin 定理

定理 16.1 (Cook-Levin, 1971/1973). *SAT* 是第一个 *NP-complete* 问题。

证明思路 直觉如下: NP 中任意问题 A 都有一个多项式时间验证器, 这个验证器本质上是一个布尔电路 (或图灵机步骤序列)。我们可以把电路的每个门的“当输入为某值时输出为何值”编码为 SAT 子句——这就是 Tseitin 编码的精神。整个电路的输入 (证书候选) 是未知变量, 要求电路输出为真等价于要求 SAT 公式可满足。因此 $A \leq_p SAT$, 对所有 $A \in NP$ 成立。

有了 SAT 是 NPC 作为起点, 后续只需从 SAT (或其他已知 NPC 问题) 归约, 就可以证明更多问题是 NPC:

$$SAT \leq_p 3SAT \leq_p \text{Independent Set} \leq_p \text{Vertex Cover} \leq_p \text{Set Cover}$$

16.3.2 证明 NP-complete 的标准模板

1. **证明属于 NP**: 给出多项式长度证书和多项式时间验证算法。
2. **选择一个已知 NPC 问题** A (通常选结构相近的)。
3. **构造多项式归约** $A \leq_p B$ 。
4. **证明 yes/no 等价**: yes 侧和 no 侧各证一遍。

易错点 方向必须是从已知 NPC 归约到待证问题 B ($A \leq_p B$), 而不是从 B 归约到 A 。后者只能说明“如果 B 能解, 则 A 也能解”, 对证明 B 的难度毫无帮助。

16.4 co-NP

定义 16.4. coNP 是补问题在 NP 中的问题类，等价地，*no* 实例有多项式可验证证书的问题类。

直觉：NP 是“yes 容易验证”；coNP 是“no 容易验证”。

例如：

- **TAUTOLOGY** (公式是否永真)：NP 的补 SAT (不可满足) 是 coNP 完全的。
- **不存在 Hamilton cycle**：证书是所有可能路径都不满足的理由——很难写出短证书。
- **线性规划可行性**：同时在 NP 和 coNP 中 (原问题可行性和不可行性都有短证书：原解和对偶不可行证书)。

若一个问题同时在 $NP \cap coNP$ 中，说明 yes/no 两侧都有短证书，这是“好刻画”，通常暗示问题可能在 P 中。

16.5 NP-hard

定义 16.5. B 是 **NP-hard** 若对所有 $A \in NP$ ，有 $A \leq_p B$ ，但 B 本身不必属于 NP (不必是决策问题)。

NP-hard = “至少和 NPC 一样难，但可以更难或不是决策问题”。例如：

- **最优化版本的 Knapsack** (求最大价值，不是判断是否超过阈值)：NP-hard 但不在 NP 中
- **停机问题**：NP-hard (其实更难，不可判定)

16.6 复杂性类的包含关系

已知的确定关系：

$$P \subseteq NP \subseteq PSPACE \subseteq EXPTIME$$

其中 $P \neq EXPTIME$ 已被证明 (时间层次定理)，因此上面链条中至少一处不等号成立。

$P = NP$? 是计算机科学最重要的开放问题。若任一 NPC 问题有多项式算法，则 $P = NP$ ；若能证明任一 NPC 问题不在 P 中，则 $P \neq NP$ 。

核心记忆 P = “能快速解”； NP = “解能快速验证” (不一定能快速找到)； NPC = “NP 中最难的，解决一个等于解决所有”； $co-NP$ = “no 能快速验证”； $NP-hard$ = “至少和 NPC 一样难，不必是决策问题”。

第十七章 超越 NP: PSPACE

对应课件: 17.pdf

主题: QBF、规划、游戏

动机: 有些问题比 NP 还难

NP 问题已经够难了, 但还有一类更难的问题: **对抗性博弈**和**规划**。象棋、围棋、地图文字游戏——先手有没有必胜策略? 这类问题的难点不只是” 找一个答案”, 而是要考虑**两个玩家的所有可能对抗序列**, 游戏树可以指数级大。PSPACE 就是描述这类问题的复杂性类。

17.1 PSPACE

定义 17.1. PSPACE 是可用**多项式空间** (但时间不限) 求解的决策问题类。

直觉: 想象你有一个多项式大小的” 草稿纸” (工作空间), 但可以在上面反复擦写, 运算时间不限。你能解决的问题就在 PSPACE 中。

命题 17.1. $P \subseteq NP \subseteq PSPACE \subseteq EXPTIME$

为什么 $NP \subseteq PSPACE$? NP 问题的验证器只需多项式空间, 用深度优先搜索逐个枚举候选证书, 复用空间, 所有候选都否定才输出 no。运行时间可能指数, 但空间是多项式的。

17.2 量化布尔公式 QBF (PSPACE 完全问题)

17.2.1 直觉: SAT 加上” 博弈量词”

SAT 问: ” 存在一个赋值使公式为真?” QBF 问: ” 存在 x_1 使得对所有 x_2 存在 $x_3 \dots$ 公式为真?”

用博弈来理解: $\exists$ 是玩家 A (希望公式为真) 的选择, $\forall$ 是玩家 B (希望为假) 的选择。QBF 问的是: A 有没有必胜策略?

定义 17.2. 量化布尔公式 (QBF) 形如

$$Q_1 x_1 Q_2 x_2 \cdots Q_n x_n \varphi(x_1, \dots, x_n),$$

其中 $Q_i \in \{\exists, \forall\}$, φ 是不含量词的布尔公式。

定理 17.1. *QBF 是 PSPACE-complete 的。*

QBF 在 PSPACE 中：用递归将量词一层层剥开，对 $\exists x$ 枚举 $x = 0, 1$ 取或，对 $\forall x$ 枚举取与，递归深度 n ，空间是多项式的。

17.2.2 小例子：2 变量 QBF

$$\exists x_1 \forall x_2 (x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2).$$

评估过程：

- 令 $x_1 = \text{T}$:
 - $x_2 = \text{T}$: $(\text{T} \vee \text{T}) \wedge (\text{F} \vee \text{F}) = \text{T} \wedge \text{F} = \text{F}$ 。失败。
- 令 $x_1 = \text{F}$:
 - $x_2 = \text{T}$: $(\text{F} \vee \text{T}) \wedge (\text{T} \vee \text{F}) = \text{T}$ 。
 - $x_2 = \text{F}$: $(\text{F} \vee \text{F}) \wedge (\text{T} \vee \text{T}) = \text{F}$ 。失败。

对 $x_1 = \text{T}$, $x_2 = \text{T}$ 时公式假，故 $\exists x_1$ 不能保证 $\forall x_2$ 成立。此 QBF 为假。

17.3 Geography 游戏 (PSPACE 完全问题)

17.3.1 规则

给定有向图 $G = (V, E)$ 和起点 s 。两名玩家轮流移动，每步沿有向边走到一个从未访问过的顶点，不能移动的玩家输。问题：先手是否有必胜策略？

定理 17.2. *Geography 游戏是 PSPACE-complete 的。*

17.3.2 4 节点例子：完整博弈树

$$V = \{s, a, b, c\}, \quad E = \{(s, a), (s, b), (a, b), (a, c), (b, c)\}.$$

起点 s ，先手（玩家 I）先走。

当前位置	已访问	可选移动
s (I 走)	$\{s\}$	a 或 b
→ a (II 走)	$\{s, a\}$	b 或 c
→ b (I 走)	$\{s, a, b\}$	c (唯一)
→ c (II 走)	$\{s, a, b, c\}$	无移动 $\Rightarrow$ II 负, I 胜
→ c (I 走)	$\{s, a, c\}$	无 c 的出边未访问 $\Rightarrow$ I 负, II 胜
→ b (II 走)	$\{s, b\}$	c (唯一)
→ c (I 走)	$\{s, b, c\}$	无移动 $\Rightarrow$ I 负, II 胜

先手从 s 走:

- 走 a : 后手走 b 时先手走 c 后必胜; 但后手走 c 时先手无路可走, 败。后手是理性的, 会走 c , 先手败。
- 走 b : 后手走 c , 先手无路可走, 先手败。

结论: 先手**必败**, 后手有必胜策略。

核心记忆 NP 关注”存在一个短证书”; $PSPACE$ 能表达量词交替——博弈 ($\exists$ 我选 $\forall$ 对手选), 规划 ($\exists$ 我选 $\forall$ 环境反应)。量词交替是理解 $PSPACE$ 的核心图像。

第十八章 DPLL 与 SAT 求解

对应课件：18.pdf

主题：Tseitin 编码、回溯、传播、学习、CDCL

动机：NP-complete 不等于实践中无法解决

SAT 是 NP-complete，理论上没有已知多项式算法，但现代 SAT solver（如 MiniSat、Z3）能在**秒级内求解百万变量的实例**。原因是工业实例有很强的结构，配合传播、冲突学习、启发式，搜索树可以被极大剪枝。

18.1 Tseitin 编码

18.1.1 直觉：“给每个中间结果起个名字”

如果直接把一棵布尔公式树展开为 CNF，可能指数爆炸。Tseitin 编码的方法是：**为每个子公式（门的输出）引入一个新变量**，再加入几个子句来保证新变量和子公式等价。

18.1.2 例子

公式 $z = (x \wedge y)$ ，引入新变量 z ，编码“ z 当且仅当 $x \wedge y$ ”：

$$(z \rightarrow x \wedge y) \wedge (x \wedge y \rightarrow z)$$

化为 CNF：

$$(\neg z \vee x) \wedge (\neg z \vee y) \wedge (z \vee \neg x \vee \neg y).$$

类似地， $z = (x \vee y)$ 编码为：

$$(z \vee \neg x) \wedge (z \vee \neg y) \wedge (\neg z \vee x \vee y).$$

对任意规模 n 的电路，Tseitin 编码产生 $O(n)$ 个子句， $O(n)$ 个变量。

18.2 DPLL 基本框架

18.2.1 直觉：“带剪枝的暴力搜索”

DPLL (Davis–Putnam–Logemann–Loveland) 的本质是在变量赋值的二叉树上深度优先搜索，但在每个节点上先执行“传播”（逻辑推断）以减少搜索空间。

DPLL(公式 F, 部分赋值 alpha):

1. 单子句传播:
 while F 中有单子句 (l):
 令 $\alpha(\text{var}(l)) = \text{使 } l \text{ 为真的值}$
 用 α 化简 F
2. 若 F 为空子句集 (所有子句满足): return SAT, α
3. 若 F 含空子句 (某子句所有文字为假): return UNSAT
4. 选择未赋值变量 x (启发式选择)
5. if DPLL(F, α 并 {x=T}) == SAT: return SAT
6. return DPLL(F, α 并 {x=F})

18.2.2 完整例子: 3 变量 4 子句

公式:

$$F = \underbrace{(x_1 \vee \neg x_2 \vee x_3)}_{C_1} \wedge \underbrace{(\neg x_1 \vee x_2)}_{C_2} \wedge \underbrace{(\neg x_2 \vee \neg x_3)}_{C_3} \wedge \underbrace{(x_1 \vee x_2)}_{C_4}$$

步骤 1: 无单子句, 选 $x_1 = \text{T}$ 。

化简: C_1 满足 (含 x_1), C_4 满足, C_2 变为 $(\neg x_1 \vee x_2) \rightarrow (x_2)$ (单子句!), C_3 不变。

步骤 2: 发现单子句 $C_2 = (x_2)$, 强制 $x_2 = \text{T}$ 。

化简: C_2 满足, C_3 变为 $(\neg x_2 \vee \neg x_3) \rightarrow (\neg x_3)$ (单子句!)

步骤 3: 发现单子句 $C_3 = (\neg x_3)$, 强制 $x_3 = \text{F}$ 。

化简: C_3 满足, $C_1 = (x_1 \vee \neg x_2 \vee x_3)$ 已经由 $x_1 = \text{T}$ 满足。

所有子句满足, 返回 SAT, 赋值 $x_1 = \text{T}, x_2 = \text{T}, x_3 = \text{F}$ 。

验证:

- $C_1 = (\text{T} \vee \text{F} \vee \text{F}) = \text{T} \checkmark$
- $C_2 = (\text{F} \vee \text{T}) = \text{T} \checkmark$
- $C_3 = (\text{F} \vee \text{T}) = \text{T} \checkmark$
- $C_4 = (\text{T} \vee \text{T}) = \text{T} \checkmark$

18.3 CDCL: 从错误中学习

18.3.1 直觉

DPLL 在冲突时只是回溯(撤销最近的决策)。CDCL (Conflict-Driven Clause Learning) 更进一步: 分析冲突原因, 学习一个新子句, 以后避免重蹈覆辙, 还能跳回到更早的决策层。

比喻: DPLL 像是走迷宫碰壁就退一步; CDCL 像是碰壁后记笔记: ”此路不通的根本原因是第 3 步向左转, 以后遇到类似情形直接跳过”。

18.3.2 蕴含图 (Implication Graph)

给每次决策赋值和由单子句传播推导出的赋值建立节点，推导关系建立有向边，最终到达冲突节点 ($\perp$)。

小例子：设 $x_1 = \top$ (第 1 层决策)， $x_2 = \top$ (第 2 层决策)，由 $(\neg x_1 \vee \neg x_2 \vee x_3)$ 推出 $x_3 = \top$ ，由 $(\neg x_3 \vee x_4)$ 推出 $x_4 = \top$ ，由 $(\neg x_2 \vee \neg x_4)$ 导致冲突 ($x_2 = \top$ 且 $x_4 = \top$ 矛盾)。

蕴含图显示：冲突的根本原因是 $x_1 = \top$ 和 $x_2 = \top$ 的组合。学习子句 $(\neg x_1 \vee \neg x_2)$ ，回跳到第 1 层，避免 $x_1 = \top$ 时再尝试 $x_2 = \top$ 。

18.3.3 解析规则

学习子句通过**解析**推导：

$$\frac{(A \vee x) \quad (B \vee \neg x)}{A \vee B}$$

反复解析，直到得到只含当前层一个文字的 1-UIP (Unit Implication Point) 子句。

18.4 启发式

- **VSIDS** (Variable State Independent Decaying Sum): 变量在冲突中出现则增加活跃度，定期乘以衰减因子，优先选活跃度高的变量。这使求解器偏向最近冲突相关的变量，经验上效果极佳。
- **Jeroslow-Wang**: 短子句权重更高，优先满足短子句。

核心记忆 DPLL = 带单子句传播的回溯搜索；CDCL = DPLL + 冲突分析 + 学习子句 + 非时间序回跳 (non-chronological backtracking)。现代 SAT solver 的核心是 CDCL + VSIDS 启发式 + 随机重启。

第十九章 近似算法导论

对应课件：19.pdf

主题：Vertex Cover、Set Cover、贪心分析

动机：放弃精确，追求有保证的近似

面对 NP-hard 的优化问题，我们不得不放弃精确最优解的追求。但”随便”给一个解不够好——我们希望有**数学保证**：算法给出的解最差不超过最优解的 α 倍。这就是近似算法的目标。

19.1 近似比 (Approximation Ratio)

定义 19.1. 对最小化问题，算法 A 是 α -近似算法 ($\alpha \geq 1$)，若对所有实例 I ：

$$A(I) \leq \alpha \cdot \text{OPT}(I).$$

对最大化问题 ($\alpha \leq 1$)：

$$A(I) \geq \alpha \cdot \text{OPT}(I).$$

直觉：”最差情况下，我的解离最优解有多远？”

证明近似比的关键挑战：OPT 是未知的！我们需要找到 OPT 的一个**下界**（最小化问题）或**上界**（最大化问题），然后把算法解与这个界比较。

19.2 Vertex Cover 2-近似

19.2.1 算法

```
VC-2-APPROX(图  $G = (V, E)$ ):
```

```
   $C = \text{空集}$ 
```

```
   $E' = E$  的副本
```

```
  while  $E'$  非空:
```

```
    任意选一条边  $(u, v)$  in  $E'$ 
```

```
     $C = C + \{u, v\}$ 
```

```
    从  $E'$  中删除所有以  $u$  或  $v$  为端点的边 (即删除 incident 边)
```

```
  return  $C$ 
```

证明思路 算法选出的边集 M 构成一个匹配（每步选的边没有公共端点，因为 incident 边被删除）。任何顶点覆盖必须覆盖 M 中的每条边，且覆盖一条边至少需要一个端点，所以 $\text{OPT} \geq |M|$ 。算法选了 $2|M|$ 个顶点，故 $|C| = 2|M| \leq 2\text{OPT}$ 。

19.2.2 6 节点图上的完整运行

图： $V = \{1, 2, 3, 4, 5, 6\}$, $E = \{(1, 2), (1, 3), (2, 4), (3, 4), (4, 5), (5, 6)\}$ 。

最优覆盖： $\{1, 4, 5\}$ （大小 3，可验证）。

算法运行：

1. 选边 $(1, 2)$ ，加入 $\{1, 2\}$ ，删除 $(1, 2), (1, 3), (2, 4)$ （incident to 1 or 2）。剩余边： $\{(3, 4), (4, 5), (5, 6)\}$ 。
2. 选边 $(3, 4)$ ，加入 $\{3, 4\}$ ，删除 $(3, 4), (4, 5)$ （incident to 3 or 4）。剩余边： $\{(5, 6)\}$ 。
3. 选边 $(5, 6)$ ，加入 $\{5, 6\}$ ，删除 $(5, 6)$ 。剩余边： $\emptyset$ 。

算法输出 $C = \{1, 2, 3, 4, 5, 6\}$ ，大小 6。

验证： $M = \{(1, 2), (3, 4), (5, 6)\}$ 是匹配， $|M| = 3$ ， $\text{OPT} \geq 3$ ， $|C| = 6 = 2|M| \leq 2\text{OPT}$ 。

✓

（注：这是最坏情况，算法给出了 2 倍最优。）

19.3 Set Cover 贪心算法

19.3.1 算法

GREEDY-SET-COVER(全集 U ，集合族 $S = \{S_1, \dots, S_m\}$ ，成本 c):

$C = \emptyset$ (已选集合)

$U' = U$ (未覆盖元素)

while U' 非空:

 选择最小化 $c(S_i) / |S_i \cap U'|$ 的集合 S_i

$C = C \cup \{S_i\}$

$U' = U' \setminus S_i$

return C

（无权版本： $c(S_i) = 1$ ，即每步选覆盖最多未覆盖元素的集合。）

19.3.2 6 元素 4 集合的完整例子

$U = \{1, 2, 3, 4, 5, 6\}$ ，集合（无权，单位成本）：

$$S_1 = \{1, 2, 3, 4\}$$

$$S_2 = \{2, 4, 5, 6\}$$

$$S_3 = \{1, 3, 5\}$$

$$S_4 = \{4, 6\}$$

最优解: $\{S_1, S_2\}$ (两个集合覆盖所有 6 个元素)。

贪心运行:

第 1 步: 未覆盖 $U' = \{1, 2, 3, 4, 5, 6\}$ 。各集合覆盖数: $|S_1 \cap U'| = 4$, $|S_2 \cap U'| = 4$, $|S_3 \cap U'| = 3$, $|S_4 \cap U'| = 2$ 。选 S_1 (覆盖最多, 4 个)。 $U' = \{5, 6\}$, $C = \{S_1\}$ 。

第 2 步: 未覆盖 $U' = \{5, 6\}$ 。 $|S_2 \cap U'| = 2$, $|S_3 \cap U'| = 1$, $|S_4 \cap U'| = 2$ 。选 S_2 (或 S_4 , 并列, 取 S_2)。 $U' = \emptyset$, $C = \{S_1, S_2\}$ 。

贪心也找到了最优解! (本例贪心恰好最优, 一般不保证。)

19.3.3 H_n 近似比分析

定理 19.1. 无权 Set Cover 贪心算法是 H_n -近似, 其中 $H_n = \sum_{k=1}^n 1/k \leq \ln n + 1$ 。

证明思路 Charging 论证: 当贪心选择集合 S_i 时, 将成本 1 均摊到 S_i 新覆盖的每个元素, 每个元素收费 $1/|S_i \cap U'_{\text{当时}}|$ 。

考虑最优解中的任意集合 S^* ($|S^*| \leq n$)。 S^* 内的元素被贪心覆盖时, 最多有 $|S^*|$ 个元素未覆盖 (当时), 因此最早被覆盖的元素收费至多 $1/1 = 1$, 第二个至多 $1/2$, $\dots$ 更精细地, S^* 中第 j 个被覆盖的元素, 当时 S^* 内还有 $|S^*| - j + 1$ 个未覆盖, 贪心选的集合覆盖数不少于 $|S^*| - j + 1$ (否则 S^* 更优, 矛盾), 所以该元素收费至多 $1/(|S^*| - j + 1)$ 。 S^* 内所有元素总收费至多 $H_{|S^*|} \leq H_n$ 。最优解有 OPT 个集合, 总收费 (即贪心解大小) 至多 $\text{OPT} \cdot H_n$ 。

核心记忆 近似算法的证明必须找到 OPT 的下界。Vertex Cover 用匹配作下界 ($\text{OPT} \geq |M|$); Set Cover 用最优解的平均覆盖能力作下界。这两种思路来自同一根源: LP 对偶。

第二十章 Steiner Tree 与 TSP

对应课件: 20.pdf

主题: 度量闭包、MST 近似、Christofides

动机: 网络设计与物流路径

如何用最少的线路连接若干城市 (Steiner Tree)? 快递员如何规划走遍所有城市又回来的最短路径 (TSP)? 这两类问题在物流、芯片布线、网络规划中无处不在, 都是 NP-hard, 但度量条件下有很好的近似算法。

20.1 Metric Steiner Tree

定义 20.1. Steiner Tree 问题: 给定图 $G = (V, E, w)$ 和终端集 $R \subseteq V$, 找最小权连通树包含所有 R 中节点 (可使用非终端 "Steiner 点")。

直觉: 比最小生成树更灵活——可以借用额外的中间节点来减少总费用。

度量版本: 边权满足三角不等式 ($w(u, v) \leq w(u, z) + w(z, v)$)。

MST 型 2-近似:

1. 在终端集 R 上建度量闭包 (R 中任意两点距离为原图最短路)。
2. 在度量闭包中求 MST T' 。
3. 将 T' 的闭包边展开为原图最短路, 删环, 得到 Steiner 树。

证明思路 最优 Steiner 树做一次完整的 DFS 遍历 (每条边走两次), 得到连接所有终端的巡游, 代价 2OPT 。把巡游中非终端节点 shortcut, 利用三角不等式代价不增, 得到终端上的哈密顿路径, 代价 $\leq 2\text{OPT}$ 。度量闭包上的 MST 代价不超过任何连接所有终端的哈密顿路径, 故 $w(T') \leq 2\text{OPT}$ 。

20.2 Metric TSP: 为何一般 TSP 不可近似

命题 20.1. 若 $P \neq \text{NP}$, 则一般 TSP, 即没有三角不等式限制的 TSP, 不存在任意常数因子的多项式时间近似算法。

证明. 我们用反证法证明。

假设存在某个常数 $\alpha \geq 1$, 使得一般 TSP 有一个多项式时间 α -近似算法 A 。也就是说, 对于任意 TSP 实例, 算法 A 都能在多项式时间内输出一个 tour, 并且其代价满足

$$\text{cost}(A) \leq \alpha \cdot \text{OPT},$$

其中 OPT 是该 TSP 实例的最优 tour 代价。

我们将说明, 如果这样的算法存在, 那么就可以用它在多项式时间内判定 Hamilton Cycle 问题。但 Hamilton Cycle 是 NP-完全问题, 因此这会推出 $P = \text{NP}$, 与假设矛盾。

现在给定一个无向图

$$G = (V, E),$$

其中 $|V| = n$ 。我们想判断 G 中是否存在 Hamilton Cycle, 即是否存在一个环恰好经过每个顶点一次。

我们根据 G 构造一个完全图 K_n 上的 TSP 实例。对任意两个不同顶点 $u, v \in V$, 定义边权如下:

$$w(u, v) = \begin{cases} 1, & \text{如果 } (u, v) \in E, \\ \alpha n + 1, & \text{如果 } (u, v) \notin E. \end{cases}$$

也就是说:

- 原图 G 中本来存在的边, 代价设为 1;
- 原图 G 中不存在的边, 我们在 TSP 的完全图中补上, 但代价设得非常大, 为 $\alpha n + 1$ 。

注意, 这个构造显然可以在多项式时间内完成。

接下来分两种情况讨论。

情况一: G 中存在 Hamilton Cycle。 如果 G 中存在 Hamilton Cycle, 那么这个环使用的每一条边都属于 E 。

由于属于 E 的边在 TSP 实例中的代价都是 1, 而 Hamilton Cycle 一共包含 n 条边, 因此对应的 TSP tour 代价为

$$n.$$

所以在这种情况下,

$$\text{OPT} \leq n.$$

另一方面, 任意 TSP tour 都必须包含 n 条边, 并且每条边权至少为 1, 所以任意 tour 的代价都至少是 n 。因此

$$\text{OPT} \geq n.$$

于是可得

$$\text{OPT} = n.$$

因此, 若 G 有 Hamilton Cycle, 则最优 TSP tour 的代价正好是 n 。

由于算法 A 是 α -近似算法, 它输出的 tour 代价满足

$$\text{cost}(A) \leq \alpha \cdot \text{OPT} = \alpha n.$$

所以在有 Hamilton Cycle 的情况下, 算法输出的 tour 代价一定不超过

$$\alpha n.$$

情况二：G 中不存在 Hamilton Cycle。 现在假设 G 中没有 Hamilton Cycle。

考虑任意一个 TSP tour。一个 TSP tour 本质上给出了顶点集合 V 的一个环形排列，也就是一个经过所有顶点一次并回到起点的环。

如果这个 tour 中使用的每一条边都属于原图 G 的边集 E ，那么这些边就在 G 中构成了一个 Hamilton Cycle。

但我们假设 G 没有 Hamilton Cycle，所以任意 TSP tour 不可能只使用 G 中的边。换句话说，任意 TSP tour 至少要使用一条原图中不存在的边。

而原图中不存在的边，在我们构造的 TSP 实例中代价为

$$\alpha n + 1.$$

同时，tour 一共有 n 条边，其他边代价至少为 1。因此任意 tour 的总代价至少为

$$(\alpha n + 1) + (n - 1) \cdot 1.$$

化简得

$$(\alpha n + 1) + (n - 1) = \alpha n + n > \alpha n.$$

所以在没有 Hamilton Cycle 的情况下，任意 TSP tour 的代价都严格大于

$$\alpha n.$$

因此此时最优解也满足

$$\text{OPT} > \alpha n.$$

算法 A 输出的也是某个合法 tour，而所有合法 tour 的代价都大于 αn ，所以它输出的 tour 代价也必然满足

$$\text{cost}(A) > \alpha n.$$

用近似算法判定 Hamilton Cycle。 现在我们得到如下结论：

$$G \text{ 有 Hamilton Cycle} \implies \text{cost}(A) \leq \alpha n.$$

而

$$G \text{ 没有 Hamilton Cycle} \implies \text{cost}(A) > \alpha n.$$

因此，我们可以用算法 A 来判定 Hamilton Cycle：

1. 给定图 G ；
2. 按照上面的方式构造一个 TSP 实例；
3. 运行 α -近似算法 A ；
4. 如果输出 tour 的代价 $\leq \alpha n$ ，则判断 G 有 Hamilton Cycle；
5. 如果输出 tour 的代价 $> \alpha n$ ，则判断 G 没有 Hamilton Cycle。

这就给出了一个多项式时间算法来判定 Hamilton Cycle。

但 Hamilton Cycle 是 NP-完全问题。如果它能在多项式时间内求解，则有

$$P = NP.$$

这与假设 $P \neq NP$ 矛盾。

因此，在 $P \neq NP$ 的前提下，一般 TSP 不存在任何常数因子的多项式时间近似算法。 $\square$

20.3 Metric TSP 的 2-近似 (MST 双倍算法)

算法：

1. 求图的 MST T 。
2. 将 T 的每条边复制（每条边变为两条），得到欧拉图（每个顶点度数为偶）。
3. 求欧拉回路。
4. 按欧拉回路顺序遍历顶点，**跳过**已经访问过的顶点（shortcut），得到哈密顿回路。

证明思路 $w(\text{MST}) \leq w(\text{OPT})$ ：删去最优 tour 的任意一条边得到哈密顿路径，路径是生成树，MST 代价不超过。双倍边的欧拉回路代价 $= 2w(\text{MST}) \leq 2\text{OPT}$ 。Shortcut 利用三角不等式不增加代价。

20.3.1 5 节点完全图例子

顶点 $\{1, 2, 3, 4, 5\}$ ，距离矩阵（满足三角不等式）：

$$d = \begin{pmatrix} 0 & 2 & 5 & 4 & 3 \\ 2 & 0 & 3 & 2 & 4 \\ 5 & 3 & 0 & 1 & 6 \\ 4 & 2 & 1 & 0 & 5 \\ 3 & 4 & 6 & 5 & 0 \end{pmatrix}$$

步骤 1： MST (Kruskal)：

- 选 $(3, 4)$ ： $d = 1$
- 选 $(1, 2)$ ： $d = 2$
- 选 $(2, 4)$ ： $d = 2$
- 选 $(1, 5)$ ： $d = 3$

MST 边集 $\{(3, 4), (1, 2), (2, 4), (1, 5)\}$ ，总权 $1 + 2 + 2 + 3 = 8$ 。

步骤 2： 每条边复制，欧拉图。

步骤 3： 欧拉回路（例）： $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 4 \rightarrow 2 \rightarrow 1 \rightarrow 5 \rightarrow 1$ 。

步骤 4： Shortcut (跳过已访问)： $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 5 \rightarrow 1$ 。代价 $= d(1, 2) + d(2, 4) + d(4, 3) + d(3, 5) + d(5, 1) = 2 + 2 + 1 + 6 + 3 = 14$ 。

（注： $2 \times \text{MST} = 16 \geq 14$ ，三角不等式确保 shortcut 有时反而更短。）

20.4 Christofides 3/2-近似

20.4.1 直觉

MST 双倍算法浪费了：每条边走两遍，但只有奇度顶点才是“必须走两遍”的根源。Christofides 算法的想法是：**只对奇度顶点求最小完美匹配来补充度数**，这样每个顶点度数变为偶，再求欧拉回路。

定理 20.1 (Christofides 1976). *Metric TSP* 有 3/2-近似算法。

算法：

1. 求 MST T ，总权 $w(T)$ 。
2. 找出 T 中所有奇度顶点的集合 O （注意 $|O|$ 一定是偶数）。
3. 在 O 的诱导子图（度量距离）上求**最小权完美匹配** M 。
4. 合并 $T \cup M$ （所有顶点度数为偶数），求欧拉回路，shortcut 得哈密顿回路。

证明思路 $w(T) \leq \text{OPT}$ （同上）。

最优 tour 经过 O 中所有顶点形成若干交替路段，可分割成 O 上的两个完美匹配，两个匹配代价之和 $\leq \text{OPT}$ （三角不等式），所以最小完美匹配 $w(M) \leq \text{OPT}/2$ 。

总代价 $w(T) + w(M) \leq \text{OPT} + \text{OPT}/2 = 3\text{OPT}/2$ 。

20.4.2 继续上面的 5 节点例子

MST 中各顶点度数：

- 顶点 1：连接 (1,2), (1,5)，度 2（偶）
- 顶点 2：连接 (1,2), (2,4)，度 2（偶）
- 顶点 3：连接 (3,4)，度 1（奇）
- 顶点 4：连接 (3,4), (2,4)，度 2（偶）
- 顶点 5：连接 (1,5)，度 1（奇）

奇度集合 $O = \{3, 5\}$ ，只需一条边的完美匹配： $M = \{(3, 5)\}$ ， $d(3, 5) = 6$ 。

合并后欧拉图边： $\{(3, 4), (1, 2), (2, 4), (1, 5), (3, 5)\}$ ，所有顶点度数：顶点 3 度 2，顶点 5 度 2，其他保持偶数。

求欧拉回路（例）： $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 5 \rightarrow 1$ 。这已经是哈密顿回路（无重复顶点），代价 $= 2 + 2 + 1 + 6 + 3 = 14$ 。

MST + 匹配总权： $8 + 6 = 14 \leq 3/2 \times \text{OPT}$ ，只要 $\text{OPT} \geq 10$ 即满足。

易错点 TSP 的 shortcut 只在满足三角不等式时保证不增代价。一般 TSP（无三角不等式）不能直接用 MST double-tree 或 Christofides 证明近似比。

第二十一章 多项式时间近似方案

对应课件：21.pdf

主题：PTAS、FPTAS、Knapsack、Bin Packing

动机：能不能想多准就多准？

2-近似或 H_n -近似对有些应用还不够精确。能否设计算法，使得给定任意精度参数 ε ，就能在多项式时间内给出误差不超过 ε 的近似解？这就是 PTAS 和 FPTAS 的目标。

21.1 PTAS 与 FPTAS

定义 21.1. *PTAS* (多项式时间近似方案) 是一族算法 $\{A_\varepsilon\}_{\varepsilon>0}$ ：对任意固定 $\varepsilon > 0$ ， A_ε 在输入规模 n 的多项式时间内给出 $(1 + \varepsilon)$ -近似。允许时间复杂度依赖 ε (如 $O(n^{1/\varepsilon})$)，但对固定 ε 是多项式的。

定义 21.2. *FPTAS* (完全多项式时间近似方案) 要求时间复杂度同时关于 n 和 $1/\varepsilon$ 都是多项式的 (如 $O(n^2/\varepsilon)$ 或 $O(n^3/\varepsilon^2)$)。

直觉区别： PTAS 是“给定精度后运行时间是多项式”，但如果 ε 本身是输入 (允许任意小)，运行时间可能爆炸。FPTAS 则对精度也鲁棒，是最强的近似保证。

说明 21.1. $FPTAS \Rightarrow PTAS$ ，反之不然。Knapsack 有 FPTAS；一般背包问题的更难变种 (如多维背包) 通常只有 PTAS，甚至没有。

21.2 输入规模、伪多项式与为什么要缩放

课件强调一个容易混淆的点：数值型优化问题的输入规模不是数值本身，而是写出这些数需要的位数。若容量 $W = 10^9$ ，二进制只需约 30 位，但按容量枚举的 DP 要做 10^9 级别的状态。因此，运行时间 $O(nW)$ 不是关于输入长度的多项式时间。

定义 21.3. 设实例 I 中的所有数默认用二进制表示， $|I|$ 表示二进制输入长度。若把所有数改用一元表示得到 I^u ，则 $|I^u|$ 大致与这些数值本身成正比。若算法运行时间关于 $|I^u|$ 是多项式，但不一定关于 $|I|$ 是多项式，则称为**伪多项式时间算法**。

例子： 0/1 Knapsack 的容量 DP 用时 $O(nW)$ 。当 W 是小整数时它很好；但如果 $W = 2^n$ ，输入只需 $n + 1$ 位写出 W ，运行时间却是指数级。FPTAS 的核心思想正是：先把大数值压缩到关于 n 和 $1/\varepsilon$ 多项式大小，再运行伪多项式 DP。

21.3 Knapsack: 问题与贪心反例

0/1 Knapsack 输入为物品集合 $S = \{a_1, \dots, a_n\}$, 每个物品有正整数大小 $\text{size}(a_i)$ 和利润 $\text{profit}(a_i)$, 容量为 B 。目标是选择子集 $T \subseteq S$, 满足

$$\sum_{a \in T} \text{size}(a) \leq B,$$

并最大化

$$\sum_{a \in T} \text{profit}(a).$$

按利润/大小比值排序的自然贪心会失败, 而且可以任意差。令容量为 B , 有两个物品:

$$a_1 : \text{size} = 1, \text{profit} = 100, \quad a_2 : \text{size} = B, \text{profit} = 100B - 1.$$

比值分别为 100 和 $100 - 1/B$, 贪心先选 a_1 , 剩余容量 $B - 1$ 放不下 a_2 , 总利润为 100。最优解只选 a_2 , 利润为 $100B - 1$ 。当 B 增大时, 贪心/最优比值趋近于 0。

21.4 Knapsack 的伪多项式 DP

直觉: ”按价值做 DP, 而非按重量”

0/1 Knapsack: n 个物品, 每个有重量 w_i 和价值 v_i , 容量 W , 最大化总价值。

按重量做 DP (传统):

$$K(i, w) = \max\{K(i-1, w), K(i-1, w-w_i) + v_i\},$$

表示只看前 i 个物品、容量为 w 时的最大价值。时间 $O(nW)$, 伪多项式。若允许物品重复选择, 则可写为

$$K(w) = \max_{i: w_i \leq w} \{K(w-w_i) + v_i\},$$

时间也是 $O(nW)$ 。

按价值做 DP (用于 FPTAS): 令 $P = \max_i v_i$, 则任意可行解价值不超过 nP 。

$\text{dp}[i][p]$ = 用前 i 个物品达到恰好价值 p 所需的最小重量。

初始: $\text{dp}[0][0] = 0$, 其余为 $+\infty$ 。转移:

$$\text{dp}[i][p] = \min(\text{dp}[i-1][p], \text{dp}[i-1][p-v_i] + w_i).$$

答案: 最大的 p 使得 $\text{dp}[n][p] \leq W$ 。时间 $O(n^2P)$, 空间 $O(n^2P)$ 或优化为 $O(nP)$ 。

21.4.1 课件中的 5 物品例子

物品如下, 容量取 $B = 10$:

物品	A	B	C	D	E
大小	7	2	9	3	1
利润	3	2	3	1	2

按价值 DP 计算最小重量 $A(i, p)$ 。下面列出最终行 $i = 5$ 中几个关键值：

p	1	2	3	4	5	6	7	8	9	10	11
$A(5, p)$	3	1	4	3	8	11	10	13	20	19	22

解释几个格子：

- $A(5, 2) = 1$ ，只选 E，大小 1，利润 2。
- $A(5, 5) = 8$ ，选 A 和 E，大小 $7 + 1 = 8$ ，利润 $3 + 2 = 5$ 。
- $A(5, 7) = 10$ ，选 A、B、E，大小 $7 + 2 + 1 = 10$ ，利润 $3 + 2 + 2 = 7$ 。
- $A(5, 8) = 13 > B$ ，所以利润 8 已经超出容量。

因此最大可行利润是 7，对应物品集合 $\{A, B, E\}$ 。这个例子展示了 $A(i, p)$ 的含义：它不是“容量 p 的最大利润”，而是“为了达到利润 p 至少要多重”。

21.5 Knapsack 的 FPTAS

前面我们已经介绍过一种按价值做 DP 的方法。回顾一下：令

$$P = \max_i v_i,$$

则任意可行解的价值都不会超过 nP 。按价值 DP 的状态为

$\text{dp}[i][p]$ = 用前 i 个物品达到恰好价值 p 所需的最小重量。

初始条件为

$$\text{dp}[0][0] = 0, \quad \text{dp}[0][p] = +\infty \quad (p > 0).$$

转移为

$$\text{dp}[i][p] = \min(\text{dp}[i-1][p], \text{dp}[i-1][p-v_i] + w_i).$$

最后在所有满足

$$\text{dp}[n][p] \leq W$$

的价值 p 中，取最大的 p 。

这个算法的时间复杂度大约是

$$O(n \cdot nP) = O(n^2 P).$$

如果 P 很大，比如价值有 10^8 量级，那么这个 DP 表格会非常大。因此它是伪多项式时间算法，而不是关于输入长度的多项式时间算法。

FPTAS 的目标是：允许损失很小的一部分精度，换取运行时间变成关于 n 和 $1/\varepsilon$ 的多项式。

21.5.1 FPTAS 的直觉

FPTAS 的核心想法是：不要直接对原始价值 v_i 做 DP，而是先把价值缩小。

具体来说，如果原始价值很大，例如

$$v_i = 123456789,$$

那么我们不一定需要保留它的所有低位信息。可以把它除以一个缩放因子 K ，再向下取整。这样得到一个较小的整数值

$$v'_i = \left\lfloor \frac{v_i}{K} \right\rfloor.$$

然后我们对缩放后的价值 v'_i 做按价值 DP。

这样做的好处是：DP 表格大小由 $\sum_i v'_i$ 决定，而不是由 $\sum_i v_i$ 决定。如果 K 选得合适，那么 v'_i 会小很多，DP 会快很多。

当然，这样会丢失一些价值信息。因为

$$v'_i = \left\lfloor \frac{v_i}{K} \right\rfloor$$

是向下取整的，所以原价值 v_i 不可能被完全保留。

FPTAS 的关键就在于：选取一个合适的 K ，使得丢失的信息总量不超过 εOPT 。

21.5.2 缩放因子的选择

设

$$P = \max_i v_i.$$

这里默认已经删去了所有重量超过背包容量 W 的物品。因为如果某个物品满足

$$w_i > W,$$

那么它不可能出现在任何可行解中，可以直接删掉。

删掉这些不可行物品之后，最大价值物品本身也是可行的。因此有

$$\text{OPT} \geq P.$$

给定误差参数 $0 < \varepsilon < 1$ ，定义缩放因子

$$K = \frac{\varepsilon P}{n}.$$

然后把每个物品的价值缩放为

$$v'_i = \left\lfloor \frac{v_i}{K} \right\rfloor.$$

缩放之后，我们不再对原始价值 v_i 做 DP，而是对新的价值 v'_i 做按价值 DP。

21.5.3 缩放后的按价值 DP

缩放后，每个物品 i 的重量仍然是 w_i ，但是价值从 v_i 变成了

$$v'_i = \left\lfloor \frac{v_i}{K} \right\rfloor.$$

因此我们可以直接使用前面定义过的按价值 DP，只需要把转移中的 v_i 替换成 v'_i 。
令

$$P' = \max_i v'_i.$$

因为任意可行解最多选择 n 个物品，所以缩放后的任意可行解价值不超过

$$nP'.$$

定义

$\text{dp}[i][p]$ = 用前 i 个物品达到恰好缩放价值 p 所需的最小重量.

初始条件为

$$\text{dp}[0][0] = 0, \quad \text{dp}[0][p] = +\infty \quad (p > 0).$$

转移为

$$\text{dp}[i][p] = \min(\text{dp}[i-1][p], \text{dp}[i-1][p-v'_i] + w_i).$$

其中当 $p - v'_i < 0$ 时，第二项视为 $+\infty$ 。

最后，在所有满足

$$\text{dp}[n][p] \leq W$$

的缩放价值 p 中，取最大的一个，并返回对应的物品集合。

需要注意的是：DP 优化的是缩放后的价值

$$\sum_{i \in S} v'_i,$$

但是我们最终评价算法质量时，看的仍然是原始价值

$$\sum_{i \in S} v_i.$$

21.5.4 时间复杂度分析

我们来估计缩放后的最大单个价值 P' 有多大。

由定义，

$$v'_i = \left\lfloor \frac{v_i}{K} \right\rfloor \leq \frac{v_i}{K}.$$

又因为

$$v_i \leq P,$$

所以

$$v'_i \leq \frac{P}{K}.$$

代入

$$K = \frac{\varepsilon P}{n},$$

得到

$$\frac{P}{K} = \frac{P}{\varepsilon P/n} = \frac{n}{\varepsilon}.$$

因此

$$v'_i \leq \frac{n}{\varepsilon}.$$

也就是说，缩放之后的最大单个价值满足

$$P' \leq \frac{n}{\varepsilon}.$$

前面按价值 DP 的时间复杂度是

$$O(n^2 P).$$

这里我们对缩放后的价值做 DP，所以把 P 换成 P' ，得到时间复杂度

$$O(n^2 P').$$

由

$$P' \leq \frac{n}{\varepsilon}$$

可得

$$O(n^2 P') = O\left(n^2 \cdot \frac{n}{\varepsilon}\right) = O\left(\frac{n^3}{\varepsilon}\right).$$

因此，缩放后的 DP 运行时间只依赖于 n 和 $1/\varepsilon$ ，而不依赖于原始价值 v_i 的数值大小。

这说明该算法在时间上是多项式的。

21.5.5 误差分析

下面证明该算法得到的是一个 $(1 - \varepsilon)$ -近似解。

设：

- S^* 是原问题的最优解；
- $\text{OPT} = \sum_{i \in S^*} v_i$ 是最优价值；
- S 是算法返回的物品集合；
- $A(I) = \sum_{i \in S} v_i$ 是算法返回解的原始价值。

由于

$$v'_i = \left\lfloor \frac{v_i}{K} \right\rfloor,$$

所以有

$$v'_i \leq \frac{v_i}{K} < v'_i + 1.$$

两边同时乘以 K ，得到

$$Kv'_i \leq v_i < K(v'_i + 1).$$

因此

$$0 \leq v_i - Kv'_i < K.$$

这说明：每个物品在缩放并还原后，损失的价值小于 K 。

现在考虑最优解 S^* 。对 S^* 中的所有物品求和，有

$$\sum_{i \in S^*} v_i - K \sum_{i \in S^*} v'_i < |S^*|K.$$

因为 S^* 最多包含 n 个物品，所以

$$|S^*|K \leq nK.$$

于是

$$\sum_{i \in S^*} v_i - K \sum_{i \in S^*} v'_i < nK.$$

也就是说，

$$K \sum_{i \in S^*} v'_i > \sum_{i \in S^*} v_i - nK.$$

由于

$$\sum_{i \in S^*} v_i = \text{OPT},$$

所以

$$K \sum_{i \in S^*} v'_i > \text{OPT} - nK.$$

接下来利用 DP 的最优性。

算法返回的集合 S 是在缩放价值 v'_i 下的最优可行解。而 S^* 也是一个可行解。因此， S 的缩放价值不会比 S^* 小：

$$\sum_{i \in S} v'_i \geq \sum_{i \in S^*} v'_i.$$

两边乘以 K ，得到

$$K \sum_{i \in S} v'_i \geq K \sum_{i \in S^*} v'_i.$$

结合前面的不等式，有

$$K \sum_{i \in S} v'_i > \text{OPT} - nK.$$

另一方面，对于每个物品都有

$$v_i \geq Kv'_i.$$

因此对于算法返回的集合 S ，

$$\sum_{i \in S} v_i \geq K \sum_{i \in S} v'_i.$$

所以

$$A(I) = \sum_{i \in S} v_i \geq K \sum_{i \in S} v'_i > \text{OPT} - nK.$$

现在代入

$$K = \frac{\varepsilon P}{n}.$$

有

$$nK = n \cdot \frac{\varepsilon P}{n} = \varepsilon P.$$

又因为删去不可行物品后，最大价值物品本身可以单独放入背包，所以

$$P \leq \text{OPT}.$$

因此

$$nK = \varepsilon P \leq \varepsilon \text{OPT}.$$

于是

$$A(I) > \text{OPT} - \varepsilon \text{OPT} = (1 - \varepsilon) \text{OPT}.$$

因此算法返回的解满足

$$A(I) \geq (1 - \varepsilon) \text{OPT}.$$

这就证明了该算法是一个 $(1 - \varepsilon)$ -近似算法。

21.5.6 为什么这是 FPTAS

FPTAS 的全称是 Fully Polynomial-Time Approximation Scheme。它要求算法满足两点：

1. 对任意给定的 $\varepsilon > 0$ ，算法返回一个 $(1 - \varepsilon)$ -近似解；
2. 算法运行时间关于输入规模和 $1/\varepsilon$ 都是多项式。

在这里，我们已经证明：

- 近似保证为

$$A(I) \geq (1 - \varepsilon) \text{OPT};$$

- 运行时间为

$$O\left(\frac{n^3}{\varepsilon}\right).$$

因此这是 Knapsack 问题的一个 FPTAS。

21.5.7 算法总结

整个算法可以总结如下：

1. 删除所有满足 $w_i > W$ 的物品；
2. 令

$$P = \max_i v_i;$$

3. 给定误差参数 $0 < \varepsilon < 1$, 令

$$K = \frac{\varepsilon P}{n};$$

4. 对每个物品计算缩放价值

$$v'_i = \left\lfloor \frac{v_i}{K} \right\rfloor;$$

5. 对缩放价值 v'_i 运行前面介绍的按价值 DP;

6. 返回 DP 找到的物品集合, 并用原始价值 $\sum_{i \in S} v_i$ 评价它。

21.5.8 上面例子上的缩放

假设课件例子中有 5 个物品, 原始价值为

$$(3, 2, 3, 1, 2).$$

于是

$$n = 5, \quad P = \max\{3, 2, 3, 1, 2\} = 3.$$

如果取

$$\varepsilon = 1,$$

那么缩放因子为

$$K = \frac{\varepsilon P}{n} = \frac{1 \cdot 3}{5} = 0.6.$$

现在计算每个物品的缩放价值:

$$v'_1 = \left\lfloor \frac{3}{0.6} \right\rfloor = 5,$$

$$v'_2 = \left\lfloor \frac{2}{0.6} \right\rfloor = 3,$$

$$v'_3 = \left\lfloor \frac{3}{0.6} \right\rfloor = 5,$$

$$v'_4 = \left\lfloor \frac{1}{0.6} \right\rfloor = 1,$$

$$v'_5 = \left\lfloor \frac{2}{0.6} \right\rfloor = 3.$$

所以缩放后的价值为

$$v' = (5, 3, 5, 1, 3).$$

原来的最大单个价值是

$$P = 3.$$

如果直接用原始价值做 DP, 价值上界可以写成

$$nP = 5 \cdot 3 = 15.$$

缩放后，最大单个缩放价值是

$$P' = \max\{5, 3, 5, 1, 3\} = 5.$$

所以缩放后的价值上界是

$$nP' = 5 \cdot 5 = 25.$$

这个例子里原始价值本来就很小，所以缩放并没有明显减少 DP 表格大小。甚至在取 $\varepsilon = 1$ 时，缩放后的上界 25 比原来的 15 还大。这并不矛盾，因为 FPTAS 的意义不在于这种小数值例子，而在于原始价值很大时，它可以把 DP 表格规模控制在关于 n 和 $1/\varepsilon$ 的多项式范围内。

例如，如果原始价值是

$$(3000000000, 2000000000, 3000000000, 1000000000, 2000000000),$$

那么直接按价值 DP 会非常慢，因为价值上界巨大。但是缩放后，每个物品的缩放价值最多是

$$\frac{n}{\varepsilon},$$

因此总的 DP 表格大小只和 n 、 $1/\varepsilon$ 有关，而不直接依赖这些 10^8 量级的价值。

21.5.9 一个便于手算的粗略缩放例子

为了便于手算，有时可以人为取一个更简单的缩放因子。比如直接取

$$K = 2.$$

此时原始价值

$$(3, 2, 3, 1, 2)$$

会变成

$$v'_i = \left\lfloor \frac{v_i}{2} \right\rfloor.$$

逐个计算：

$$\begin{aligned} \left\lfloor \frac{3}{2} \right\rfloor &= 1, & \left\lfloor \frac{2}{2} \right\rfloor &= 1, \\ \left\lfloor \frac{3}{2} \right\rfloor &= 1, & \left\lfloor \frac{1}{2} \right\rfloor &= 0, \\ \left\lfloor \frac{2}{2} \right\rfloor &= 1. \end{aligned}$$

所以缩放后的价值为

$$v' = (1, 1, 1, 0, 1).$$

可以看到，物品 D 的原始价值是 1，但是缩放后变成了 0。这说明缩放确实会丢失一部分价值信息。DP 在缩放后的问题里可能就不会重视物品 D 。

例如，原最优集合可能是

$$\{A, B, E\},$$

其原始价值为

$$3 + 2 + 2 = 7,$$

缩放价值为

$$1 + 1 + 1 = 3.$$

而集合

$$\{A, E\}$$

的原始价值为

$$3 + 2 = 5,$$

缩放价值为

$$1 + 1 = 2.$$

缩放后, DP 只看缩放价值, 因此会丢失一些细节。但是这种损失是可控的。因为对每个物品都有

$$0 \leq v_i - Kv'_i < K.$$

也就是说, 每个物品最多损失不到 K 的价值。任意解最多包含 n 个物品, 所以总损失小于

$$nK.$$

在 FPTAS 中, 我们不是随便取 K , 而是取

$$K = \frac{\varepsilon P}{n}.$$

于是

$$nK = \varepsilon P \leq \varepsilon \text{OPT}.$$

这就保证了虽然缩放会丢失信息, 但最终得到的解相对于最优解最多只损失 εOPT , 也就是至少达到

$$(1 - \varepsilon) \text{OPT}.$$

21.6 Bin Packing

问题: 物品大小 $s_i \in (0, 1]$, 装入容量 1 的箱子, 最小化箱子数。

简单 2-近似 (First Fit): 物品按任意顺序, 尝试放入已开的箱子, 若都放不下则开新箱子。

证明思路 若用了 k 个箱子, 则前 $k - 1$ 个每个装载量 $> 1/2$ (否则两个相邻箱子可合并), 故总物品大小 $> (k - 1)/2$, 而 $\text{OPT} \geq \lceil \text{总大小} \rceil \geq k/2$, 所以 $k \leq 2\text{OPT}$ 。

21.6.1 First Fit 的完整例子

物品大小依次为

0.6, 0.2, 0.4, 0.7, 0.3, 0.5.

First Fit 逐个处理:

步骤	当前物品	箱子状态
1	0.6	$B_1 = (0.6)$
2	0.2	$B_1 = (0.6, 0.2)$, 剩余 0.2
3	0.4	B_1 放不下, 开 $B_2 = (0.4)$
4	0.7	B_1, B_2 都放不下, 开 $B_3 = (0.7)$
5	0.3	B_1 放不下, 放入 $B_2 = (0.4, 0.3)$
6	0.5	B_1, B_2, B_3 都放不下, 开 $B_4 = (0.5)$

算法用了 4 个箱子。总大小为 2.7, 所以任意方案至少需要 3 个箱子。这里 First Fit 的解不一定最优, 但保证不会超过 2OPT。

21.6.2 Bin Packing 的不可近似性

我们先回忆 Bin Packing 问题。给定若干物品, 每个物品大小为 s_i , 以及箱子容量 B 。目标是用尽可能少的箱子装下所有物品, 并且每个箱子中物品大小之和不能超过 B 。

课件中给出的硬度结论是:

定理 21.1. 若 $P \neq NP$, 则对任意常数 $\varepsilon > 0$, *Bin Packing* 不存在近似比为 $3/2 - \varepsilon$ 的多项式时间算法。

这里的近似比是针对最小化问题的。也就是说, 如果一个算法的近似比为 ρ , 那么对任意输入实例 I , 算法输出的箱子数 $A(I)$ 满足

$$A(I) \leq \rho \cdot OPT(I),$$

其中 $OPT(I)$ 是最优解使用的箱子数。

下面证明这个结论。

证明. 我们从 Partition 问题归约。

Partition 问题的输入是一组非负整数

$$a_1, a_2, \dots, a_n.$$

问题是: 是否可以把这些数分成两组, 使得两组的和相等?

记所有数的总和为

$$S = \sum_{i=1}^n a_i.$$

如果 S 是奇数, 那么显然不可能平分, Partition 的答案一定是 no。因此我们只需要考虑 S 为偶数的情形。令

$$B = \frac{S}{2}.$$

我们构造一个 Bin Packing 实例：

$$s_i = a_i, \quad i = 1, \dots, n,$$

也就是说，每个数 a_i 对应一个物品，物品大小就是 a_i ，箱子容量设为

$$B = \frac{S}{2}.$$

接下来观察 Partition 和这个 Bin Packing 实例之间的关系。

若 Partition 答案为 yes 如果 $a_1, \dots, a_n$ 可以分成两组，并且每组的和都是 $S/2 = B$ ，那么这两组就可以分别放入两个箱子中。每个箱子的容量正好是 B ，所以两个箱子足够装下所有物品。

因此在这种情况下，

$$OPT \leq 2.$$

另一方面，所有物品的总大小是

$$S = 2B.$$

一个箱子的容量只有 B ，因此一个箱子不可能装下总大小为 $2B$ 的物品。所以至少需要两个箱子，即

$$OPT \geq 2.$$

于是有

$$OPT = 2.$$

也就是说，如果 Partition 答案为 yes，那么对应的 Bin Packing 实例的最优箱子数正好是 2。

若 Partition 答案为 no 如果 Partition 答案为 no，说明不存在一种划分方式，使得两组的和都等于 B 。

现在假设对应的 Bin Packing 实例可以用两个箱子装下所有物品。由于所有物品总大小是

$$S = 2B,$$

而两个箱子的总容量也是

$$2B.$$

因此，如果所有物品能装入两个箱子，那么这两个箱子必须都被装得恰好满，每个箱子中的物品大小之和都必须是 B 。

这就给出了一个 Partition 的可行划分：一个箱子中的物品作为第一组，另一个箱子中的物品作为第二组，两组的和都是 B 。

这与 Partition 答案为 no 矛盾。

因此，如果 Partition 答案为 no，那么对应的 Bin Packing 实例不可能用两个箱子装下所有物品，所以

$$OPT \geq 3.$$

综上，我们得到一个关键等价关系：

$$\text{Partition 答案为 yes} \iff OPT = 2.$$

而如果 Partition 答案为 no，则一定有

$$OPT \geq 3.$$

下面我们用这个等价关系证明 Bin Packing 不可能存在近似比严格小于 $3/2$ 的多项式算法。

假设存在一个多项式时间算法 A ，它对 Bin Packing 的近似比为

$$\frac{3}{2} - \varepsilon,$$

其中 $\varepsilon > 0$ 是某个固定常数。

现在给定一个 Partition 实例，我们按照上面的方式构造出 Bin Packing 实例，然后运行算法 A 。

如果原 Partition 实例是 yes，那么对应 Bin Packing 实例满足

$$OPT = 2.$$

由于算法 A 的近似比为 $3/2 - \varepsilon$ ，所以它输出的箱子数满足

$$A(I) \leq \left(\frac{3}{2} - \varepsilon\right) OPT = \left(\frac{3}{2} - \varepsilon\right) \cdot 2 = 3 - 2\varepsilon.$$

因为 $\varepsilon > 0$ ，所以

$$3 - 2\varepsilon < 3.$$

于是算法输出的箱子数严格小于 3。

但是箱子数必须是整数，因此

$$A(I) \leq 2.$$

另一方面，所有物品的总大小是 $2B$ ，一个箱子容量只有 B ，所以至少需要两个箱子。因此

$$A(I) = 2.$$

也就是说, 在 Partition 的 yes 情况下, 这个近似算法一定会输出 2 个箱子。

再看 no 情况。如果 Partition 答案为 no, 那么我们已经证明对应的 Bin Packing 实例不可能用两个箱子装下, 因此任何可行解都至少需要三个箱子。算法 A 输出的是一个可行装箱方案, 所以必然有

$$A(I) \geq 3.$$

于是我们可以用算法 A 来判定 Partition:

$$\begin{cases} \text{如果 } A(I) = 2, & \text{则 Partition 答案为 yes;} \\ \text{如果 } A(I) \geq 3, & \text{则 Partition 答案为 no.} \end{cases}$$

这就给出了一个多项式时间算法来求解 Partition 问题。

但是 Partition 是 NP-hard 的。因此, 如果这样的 $(3/2 - \varepsilon)$ -近似算法存在, 就会推出

$$P = NP.$$

所以在 $P \neq NP$ 的假设下, 对任意 $\varepsilon > 0$, Bin Packing 都不存在近似比为 $3/2 - \varepsilon$ 的多项式时间算法。 $\square$

这个证明的核心是整数性。若最优解为 2, 一个近似比严格小于 $3/2$ 的算法必须输出

$$< 3$$

个箱子。由于箱子数是整数, 所以它只能输出 2 个箱子。这样一来, 近似算法实际上就能区分

$$OPT = 2$$

和

$$OPT \geq 3$$

这两种情况, 从而判定 Partition。

这也解释了为什么 Bin Packing 的近似算法通常使用**渐近近似比**来描述, 而不是要求普通意义下的严格乘法近似比。

普通近似比要求对所有实例都满足

$$A(I) \leq \rho \cdot OPT(I).$$

当 OPT 很小时, 例如 $OPT = 2$, 这个要求非常严格。如果 $\rho < 3/2$, 那么算法必须把所有最优值为 2 的实例也解到最优, 否则只要输出 3 个箱子, 近似比就是

$$\frac{3}{2}.$$

但这已经足够判定 Partition, 因此是不可能的。

渐近近似比允许一个常数加法误差, 形式通常类似于

$$A(I) \leq \rho \cdot OPT(I) + C,$$

其中 C 是常数。这样当 OPT 很小时，允许算法多用常数个箱子；而当 OPT 很大时，这个常数误差相对于 OPT 可以忽略。

因此，Bin Packing 的精细近似结果通常不是讨论普通意义下的近似比，而是讨论渐近意义下的近似比。

21.6.3 APTAS：渐近近似方案

定义 21.4. 对于一个最小化问题，若对任意固定的 $\varepsilon > 0$ ，存在多项式时间算法 A_ε ，使得对任意实例 I 都有

$$A_\varepsilon(I) \leq (1 + \varepsilon)OPT(I) + c,$$

其中 c 是一个与输入规模无关的常数，则称该问题有一个 **APTAS**，即渐近多项式时间近似方案。

这里的“渐近”体现在允许多加一个常数项 c 。因此当最优值 $OPT(I)$ 很大时，这个常数项的影响可以忽略。

下面证明 Bin Packing 有 APTAS。更具体地，我们证明：对任意 $0 < \varepsilon \leq 1/2$ ，存在多项式时间算法，使用的箱子数至多为

$$(1 + 2\varepsilon)OPT + 1.$$

证明分成三步。

第一步：大物品且大小种类数固定时，可以精确求解

首先考虑一个比较特殊的情形：

- 所有物品的大小都至少为 ε ；
- 物品的大小只有 K 种不同取值。

因为每个物品大小至少是 ε ，而每个箱子的容量是 1，所以一个箱子中最多只能放

$$M = \left\lfloor \frac{1}{\varepsilon} \right\rfloor$$

个物品。

现在假设这 K 种物品大小分别为

$$s_1, s_2, \dots, s_K.$$

一个箱子的装法可以用一个 K 维向量表示：

$$(a_1, a_2, \dots, a_K),$$

其中 a_i 表示这个箱子里放了多少个大小为 s_i 的物品。

这个向量必须满足两个条件：

$$a_1 s_1 + a_2 s_2 + \cdots + a_K s_K \leq 1,$$

以及

$$a_1 + a_2 + \cdots + a_K \leq M.$$

因此，每个箱子的可能类型数量是有限的。粗略地说，如果不考虑容量约束，只考虑一个箱子中最多放 M 个物品，那么不同的向量数量至多为

$$R = \binom{M+K}{M}.$$

当 ε 和 K 都固定时， M 和 R 都是常数。

接下来我们枚举每一种箱子类型使用多少个箱子。

设所有可能的箱子类型为

$$T_1, T_2, \dots, T_R.$$

我们枚举非负整数

$$x_1, x_2, \dots, x_R,$$

其中 x_j 表示类型 T_j 的箱子使用了多少个。

因为总共有 n 个物品，所以最优解不可能使用超过 n 个箱子。因此我们只需要考虑

$$x_1 + x_2 + \cdots + x_R \leq n.$$

这样的组合数至多为

$$\binom{n+R}{R}.$$

由于 R 是常数，所以这是关于 n 的多项式。

对每一种枚举出来的组合，我们检查它是否刚好能够容纳所有物品。在所有可行组合中，选择箱子总数

$$x_1 + x_2 + \cdots + x_R$$

最小的那个，就得到了精确最优解。

因此，在“物品都很大，并且大小种类数固定”的情况下，Bin Packing 可以在多项式时间内精确求解。

第二步：只有大物品，但大小种类不固定时，进行分组取整

现在考虑一般的大物品情形：

- 所有物品大小都至少为 ε ；
- 但物品大小可能有很多种，甚至每个物品大小都不同。

第一步的方法要求物品大小种类数 K 是常数。为了解决这个问题，我们把物品按照大小分组，然后把每一组中的物品大小统一取整。

具体做法如下。

将所有物品按照大小从小到大排序。然后把它们分成

$$K = \left\lceil \frac{1}{\varepsilon^2} \right\rceil$$

组，记为

$$G_1, G_2, \dots, G_K.$$

其中 G_1 是最小的一组， G_K 是最大的一组。

每组的物品数大约为

$$Q \approx n\varepsilon^2.$$

为了简化记号，下面假设每组恰好有 Q 个物品。实际情况中由于取整，每组大小可能相差 1，这只会带来常数级的影响，不影响 APTAS 的结论。

现在构造一个新实例 J 。

对于每一组 G_i ，把这一组中所有物品的大小都向上取整为该组最大物品的大小。

也就是说，如果

$$G_i = \{a_{i,1}, a_{i,2}, \dots, a_{i,Q}\},$$

并且其中最大物品大小为

$$\max(G_i),$$

那么在新实例 J 中， G_i 中每个物品的大小都变成

$$\max(G_i).$$

这样处理之后，每组内部所有物品大小相同，因此实例 J 中最多只有

$$K = \left\lceil \frac{1}{\varepsilon^2} \right\rceil$$

种物品大小。

由于 ε 是固定常数，所以 K 也是常数。于是根据第一步，我们可以对实例 J 精确求解。

注意，因为 J 是把原实例 I 中的物品大小向上取整得到的，所以如果一个装箱方案能够装下 J ，那么它一定也能装下原实例 I 。因此，对 J 的最优装箱方案也是原实例 I 的一个可行装箱方案。

关键问题是：向上取整会不会让最优箱子数增加太多？

我们要证明：

$$\text{OPT}(J) \leq (1 + \varepsilon)\text{OPT}(I).$$

为此，再构造一个辅助实例 J' 。

实例 J' 是把每一组中的物品大小向下取整为该组最小物品大小得到的。也就是说，对于每组 G_i ，把其中所有物品大小都变成

$$\min(G_i).$$

因为 J' 中每个物品都不大于原实例 I 中对应物品，所以原实例 I 的任何可行装箱方案，对 J' 也一定可行。因此有

$$\text{OPT}(J') \leq \text{OPT}(I).$$

接下来比较 J' 和 J 。

由于我们是按照物品大小从小到大排序并分组的，所以对于相邻两组，有

$$\max(G_i) \leq \min(G_{i+1}).$$

这意味着：

第 i 组向上取整后的物品大小 $\leq$ 第 $i+1$ 组向下取整后的物品大小。

换句话说， J 中第 i 组的物品，不会比 J' 中第 $i+1$ 组的物品更大。

于是，我们可以这样使用 J' 的装箱方案。

假设我们有一个 J' 的最优装箱方案。这个方案中装了所有组

$$G_1, G_2, \dots, G_K$$

的向下取整版本。

现在我们把 J' 中第 $i+1$ 组物品所在的位置，换成 J 中第 i 组物品。因为

$$\max(G_i) \leq \min(G_{i+1}),$$

所以替换之后物品只会变小，不会导致箱子超容量。

这样一来， J' 的装箱方案可以用来装下 J 中的

$$G_1, G_2, \dots, G_{K-1}$$

这些组。

唯一没有装进去的是 J 中最大的那一组 G_K 。

这一组最多有 Q 个物品。最坏情况下，我们可以给每个物品单独开一个箱子，因此额外需要至多 Q 个箱子。

所以有

$$\text{OPT}(J) \leq \text{OPT}(J') + Q.$$

又因为

$$\text{OPT}(J') \leq \text{OPT}(I),$$

所以

$$\text{OPT}(J) \leq \text{OPT}(I) + Q.$$

现在我们估计 Q 和 $\text{OPT}(I)$ 的关系。

由于当前只考虑大物品，每个物品大小都至少为 ε 。如果一共有 n 个物品，那么所有物品的总体积至少是

$$n\varepsilon.$$

而每个箱子的容量是 1，所以最优箱子数至少等于总体积的下界：

$$\text{OPT}(I) \geq n\varepsilon.$$

另一方面，每组大小大约为

$$Q \leq n\varepsilon^2.$$

因此

$$Q \leq n\varepsilon^2 = \varepsilon \cdot n\varepsilon \leq \varepsilon \text{OPT}(I).$$

代入前面的不等式，得到

$$\text{OPT}(J) \leq \text{OPT}(I) + Q \leq \text{OPT}(I) + \varepsilon \text{OPT}(I) = (1 + \varepsilon) \text{OPT}(I).$$

因此，对大物品实例，分组取整之后再精确求解，得到的装箱数至多为

$$(1 + \varepsilon) \text{OPT}(I).$$

第三步：加入小物品，用 First Fit 填补空隙

现在回到完整的 Bin Packing 实例，其中既有大物品，也有小物品。

我们把物品分成两类：

- 大物品：大小至少为 ε ；
- 小物品：大小小于 ε 。

算法如下：

1. 暂时丢开所有小物品，只保留大物品；
2. 对大物品使用第二步的算法，得到一个装箱方案；
3. 再把小物品一个一个放入已有箱子中，使用 First Fit 策略；
4. 如果所有已有箱子都放不下当前小物品，就新开一个箱子。

First Fit 的意思是：按照某个顺序处理小物品，每次把当前物品放入编号最小的、能够容纳它的箱子；如果没有箱子能容纳它，则新开一个箱子。

下面分析这个算法使用的箱子数。

设最终算法使用了 M 个箱子。

分两种情况讨论。

情况一：没有因为小物品而新開箱子。 也就是说，所有小物品都被放进了装大物品时已经开的箱子里。

此时总箱子数等于大物品装箱阶段使用的箱子数。

设只由大物品组成的实例为 I_L 。根据第二步，大物品阶段使用的箱子数至多为

$$(1 + \varepsilon) \text{OPT}(I_L).$$

由于 I_L 只是原实例 I 的一部分，所以

$$\text{OPT}(I_L) \leq \text{OPT}(I).$$

因此总箱子数至多为

$$(1 + \varepsilon) \text{OPT}(I).$$

这当然也不超过我们最终要证明的

$$(1 + 2\varepsilon) \text{OPT}(I) + 1.$$

情况二：因为小物品而开了新箱子。 这是更重要的情况。

假设最终一共用了 M 个箱子。我们证明：除了最后一个箱子之外，其余每个箱子的装载量都至少为

$$1 - \varepsilon.$$

为什么？

因为所有小物品大小都小于 ε 。

如果某个箱子在算法结束时的剩余容量至少为 ε ，那么任何一个小物品都能够放进这个箱子。

而 First Fit 总是优先把当前物品放进前面已有的箱子。因此，如果最后确实开了新箱子，就说明在开新箱子之前，前面所有箱子的剩余容量都小于当前小物品的大小。

由于当前小物品大小小于 ε ，所以这些箱子的剩余容量都小于 ε ，也就是它们的装载量都大于

$$1 - \varepsilon.$$

因此，在最终方案中，除最后一个箱子之外，每个箱子的装载量都至少为

$$1 - \varepsilon.$$

于是，所有物品的总体积至少为

$$(M - 1)(1 - \varepsilon).$$

另一方面，所有物品的总体积是任意装箱方案所需箱子数的下界。因此

$$\text{OPT}(I) \geq (M - 1)(1 - \varepsilon).$$

整理得到

$$M - 1 \leq \frac{\text{OPT}(I)}{1 - \varepsilon},$$

即

$$M \leq \frac{\text{OPT}(I)}{1 - \varepsilon} + 1.$$

因为我们假设

$$0 < \varepsilon \leq \frac{1}{2},$$

所以有

$$\frac{1}{1 - \varepsilon} \leq 1 + 2\varepsilon.$$

因此

$$M \leq \frac{\text{OPT}(I)}{1 - \varepsilon} + 1 \leq (1 + 2\varepsilon)\text{OPT}(I) + 1.$$

综上，无论小物品是否导致新开箱子，算法使用的箱子数都至多为

$$(1 + 2\varepsilon)\text{OPT}(I) + 1.$$

因此 Bin Packing 存在 APTAS。

核心记忆 FPTAS 通常来自伪多项式算法加数值缩放；PTAS/APTAS 通常来自“枚举大结构 + 贪心处理小误差”。Knapsack FPTAS 和 Bin Packing APTAS 分别展示了这两种范式。

第二十二章 基于线性规划的近似

对应课件：22.pdf

主题：舍入、随机舍入、原始-对偶、Dual Fitting

动机：LP 松弛不只是理论工具

我们在第 13 章看到 LP 可以建模很多问题。现在用 LP 直接设计近似算法：先解松弛问题（允许分数解），再把分数解“舍入”成整数解，分析损失。这套框架既给出了算法，也给出了 OPT 的下界，是近似算法的重要范式。

22.1 LP 舍入 (LP Rounding)

直觉：“LP 给出分数解，我们把它舍入到整数”。

LP 松弛是整数规划的下界（最小化）或上界（最大化）。把分数解转成整数解会增加代价，分析增加了多少倍。

一般框架：

1. 写出整数规划 $\rightarrow$ 松弛为 LP（去掉整数约束）。
2. 求解 LP，得到分数最优解 x^* ， $LP^* \leq OPT$ （最小化）。
3. 设计舍入规则（简单阈值、随机化、依赖问题结构）。
4. 分析舍入后代价 $\leq \alpha \cdot LP^* \leq \alpha \cdot OPT$ 。

22.1.1 Set Cover 的简单舍入

我们考虑带权 Set Cover 问题。给定全集 U ，以及若干集合 $S_1, S_2, \dots, S_m \subseteq U$ ，每个集合 S_i 有非负成本 c_i 。目标是选择一些集合，使得它们的并集覆盖整个 U ，并且总成本最小。

设每个元素最多出现在 f 个集合中，也就是说，对任意元素 $e \in U$ ，都有

$$|\{i : e \in S_i\}| \leq f.$$

这里的 f 称为元素的最大频率，或者简称频率。

整数规划形式 我们先用变量 x_i 表示是否选择集合 S_i :

$$x_i = \begin{cases} 1, & \text{如果选择集合 } S_i, \\ 0, & \text{否则。} \end{cases}$$

那么 Set Cover 可以写成如下整数规划:

$$\begin{aligned} \min \quad & \sum_{i=1}^m c_i x_i \\ \text{s.t.} \quad & \sum_{i:e \in S_i} x_i \geq 1, \quad \forall e \in U, \\ & x_i \in \{0, 1\}, \quad \forall i. \end{aligned}$$

其中约束

$$\sum_{i:e \in S_i} x_i \geq 1$$

的意思是: 对于每个元素 e , 至少要选择包含它的集合。

LP 松弛 整数规划通常比较难解, 因此我们先把条件

$$x_i \in \{0, 1\}$$

放松为

$$0 \leq x_i \leq 1.$$

于是得到线性规划松弛:

$$\begin{aligned} \min \quad & \sum_{i=1}^m c_i x_i \\ \text{s.t.} \quad & \sum_{i:e \in S_i} x_i \geq 1, \quad \forall e \in U, \\ & 0 \leq x_i \leq 1, \quad \forall i. \end{aligned}$$

设这个 LP 的最优解为

$$x^* = (x_1^*, x_2^*, \dots, x_m^*).$$

由于 LP 是整数规划的松弛, 所以它的最优值不会超过整数最优值。也就是说,

$$LP^* \leq \text{OPT}.$$

舍入算法 现在我们需要把分数解 x^* 转换成整数解。一个非常自然的舍入规则是:

$$\text{选择所有满足 } x_i^* \geq \frac{1}{f} \text{ 的集合 } S_i.$$

也就是说, 我们定义新的整数解 $\hat{x}$:

$$\hat{x}_i = \begin{cases} 1, & \text{如果 } x_i^* \geq \frac{1}{f}, \\ 0, & \text{否则。} \end{cases}$$

下面证明这样得到的解一定是一个合法的 set cover, 并且成本最多是最优解的 f 倍。

覆盖性证明 我们需要证明：对于任意元素 $e \in U$ ，它一定会被选中的集合覆盖。

考虑任意一个元素 e 。在 LP 解中，它满足约束：

$$\sum_{i:e \in S_i} x_i^* \geq 1.$$

也就是说，所有包含 e 的集合对应的分数变量加起来至少是 1。

另一方面，根据频率假设，元素 e 最多出现在 f 个集合中。因此，在上面的求和中，最多只有 f 项。

现在我们要用一个简单的平均值思想：

如果最多有 f 个非负数，它们的和至少是 1，那么其中至少有一个数不小于 $\frac{1}{f}$ 。

否则，如果每一项都严格小于 $\frac{1}{f}$ ，那么它们的总和就会小于

$$f \cdot \frac{1}{f} = 1,$$

这与

$$\sum_{i:e \in S_i} x_i^* \geq 1$$

矛盾。

因此，对于元素 e ，必然存在某个包含它的集合 S_i ，使得

$$x_i^* \geq \frac{1}{f}.$$

根据我们的舍入规则，这个集合 S_i 会被选中。所以元素 e 被覆盖。

因为 e 是任意的，所以所有元素都会被覆盖。因此，舍入后的解是一个合法的 set cover。

代价分析 接下来分析舍入后选择的集合总成本。

舍入后被选中的集合满足

$$x_i^* \geq \frac{1}{f}.$$

对于每一个被选中的集合 S_i ，有

$$x_i^* \geq \frac{1}{f}.$$

两边乘以 f ，得到

$$fx_i^* \geq 1.$$

再乘以成本 $c_i \geq 0$ ，得到

$$fc_ix_i^* \geq c_i.$$

也就是说，对于每个被选中的集合，

$$c_i \leq fc_ix_i^*.$$

因此, 舍入后总成本满足

$$\sum_{i: x_i^* \geq 1/f} c_i \leq \sum_{i: x_i^* \geq 1/f} f c_i x_i^*.$$

进一步地,

$$\sum_{i: x_i^* \geq 1/f} f c_i x_i^* \leq f \sum_{i=1}^m c_i x_i^*.$$

而

$$\sum_{i=1}^m c_i x_i^* = LP^*.$$

所以舍入后的成本至多为

$$f \cdot LP^*.$$

又因为

$$LP^* \leq \text{OPT},$$

所以

$$\sum_{i: x_i^* \geq 1/f} c_i \leq f \cdot LP^* \leq f \cdot \text{OPT}.$$

因此, 这个舍入算法是一个 f -近似算法。

总结 这个算法的核心思想是:

- 每个元素 e 在 LP 解中必须被分数覆盖至少 1;
- 但 e 最多只出现在 f 个集合中;
- 所以这最多 f 个变量里, 必然有一个至少是 $\frac{1}{f}$;
- 因此选择所有 $x_i^* \geq \frac{1}{f}$ 的集合, 就一定能覆盖每个元素;
- 成本方面, 由于被选中的集合满足 $x_i^* \geq \frac{1}{f}$, 因此整数成本最多比 LP 成本放大 f 倍。

因此, 当每个元素的出现频率最多为 f 时, Set Cover 存在一个简单的 f -近似算法。

例子 考虑全集

$$U = \{a, b, c, d\},$$

以及四个集合:

$$S_1 = \{a, b\}, \quad S_2 = \{b, c\}, \quad S_3 = \{c, d\}, \quad S_4 = \{a, d\}.$$

假设每个集合的成本都是 1。

我们先看每个元素出现在哪些集合中:

$$a \in S_1, S_4,$$

$$b \in S_1, S_2,$$

$$c \in S_2, S_3,$$

$$d \in S_3, S_4.$$

所以每个元素都恰好出现在 2 个集合中, 因此

$$f = 2.$$

考虑如下 LP 解:

$$x_1^* = x_2^* = x_3^* = x_4^* = \frac{1}{2}.$$

我们检查一下它确实满足 LP 约束。

对于元素 a , 它被 S_1 和 S_4 覆盖, 所以:

$$x_1^* + x_4^* = \frac{1}{2} + \frac{1}{2} = 1.$$

对于元素 b , 它被 S_1 和 S_2 覆盖, 所以:

$$x_1^* + x_2^* = \frac{1}{2} + \frac{1}{2} = 1.$$

对于元素 c , 它被 S_2 和 S_3 覆盖, 所以:

$$x_2^* + x_3^* = \frac{1}{2} + \frac{1}{2} = 1.$$

对于元素 d , 它被 S_3 和 S_4 覆盖, 所以:

$$x_3^* + x_4^* = \frac{1}{2} + \frac{1}{2} = 1.$$

因此这是一个可行的 LP 解。

由于 $f = 2$, 舍入阈值为

$$\frac{1}{f} = \frac{1}{2}.$$

我们的舍入规则是选择所有满足

$$x_i^* \geq \frac{1}{2}$$

的集合。

在这个例子中,

$$x_1^* = x_2^* = x_3^* = x_4^* = \frac{1}{2},$$

所以四个集合都会被选中。舍入后的解是:

$$\{S_1, S_2, S_3, S_4\}.$$

总成本为

$$1 + 1 + 1 + 1 = 4.$$

但是最优整数解其实只需要选两个集合, 例如:

$$S_1 = \{a, b\}, \quad S_3 = \{c, d\}.$$

它们已经覆盖了

$$\{a, b, c, d\}.$$

所以最优整数解成本为

$$\text{OPT} = 2.$$

这个例子中，舍入算法得到的成本是 4，最优解成本是 2，比例为

$$\frac{4}{2} = 2.$$

这正好等于 $f = 2$ ，说明这个舍入算法在这个例子上的表现达到了 2 倍近似。

不过这个例子的主要作用不是说明算法一定很差，而是帮助理解阈值舍入的逻辑：

- 每个元素只出现在两个集合中；
- LP 要求它的分数覆盖总量至少是 1；
- 因此这两个集合中至少有一个变量值达到 $\frac{1}{2}$ ；
- 所以选择所有变量值至少为 $\frac{1}{2}$ 的集合，必然可以覆盖所有元素。

22.2 随机舍入 (Randomized Rounding)

直觉：”把 LP 值当作概率”。

对 Set Cover，课件先考虑一次随机试验：以概率 x_S^* 独立选择每个集合 S 。设 C 为选出的集合族，则

$$\mathbb{E}[\text{cost}(C)] = \sum_S c(S) \Pr[S \text{ 被选}] = \sum_S c(S) x_S^* = LP^*.$$

一次试验不一定覆盖所有元素。若元素 a 出现在 k 个集合中，对应概率为 $p_1, \dots, p_k$ ，LP 约束给出 $\sum_i p_i \geq 1$ 。由乘积在和固定时的最大性，

$$\Pr[a \text{ 未被覆盖}] = \prod_{i=1}^k (1 - p_i) \leq \left(1 - \frac{1}{k}\right)^k \leq \frac{1}{e}.$$

因此一次试验覆盖某个固定元素的概率至少 $1 - 1/e$ ，但还不能保证覆盖所有元素。

课件的做法是独立重复 $c \log n$ 次 ($n = |U|$)，将所有被选集合取并集 C' 。选择常数 c 使得

$$(1/e)^{c \log n} \leq \frac{1}{4n}.$$

则每个元素未覆盖概率至多 $1/(4n)$ ，由 union bound，

$$\Pr[C' \text{ 不是合法 set cover}] \leq n \cdot \frac{1}{4n} = \frac{1}{4}.$$

另一方面，

$$\mathbb{E}[\text{cost}(C')] \leq c \log n \cdot LP^*.$$

由 Markov 不等式，

$$\Pr[\text{cost}(C') \geq 4c \log n \cdot LP^*] \leq \frac{1}{4}.$$

所以以至少 $1/2$ 的概率, C' 同时合法且成本至多 $4c \log n \cdot LP^* \leq O(\log n)OPT$ 。重复运行若干次即可把成功概率放大。

小例子: 仍取 $U = \{a, b, c\}$, $S_1 = \{a, b\}$, $S_2 = \{b, c\}$, $S_3 = \{a, c\}$, 单位成本。LP 可取 $x_1 = x_2 = x_3 = 1/2$, 成本 $3/2$; 整数最优需选两个集合, 成本 2。一次随机舍入中, 每个集合以概率 $1/2$ 被选。元素 a 由 S_1, S_3 覆盖, 未覆盖概率为 $(1/2)^2 = 1/4$; 其他元素同理。单次试验可能失败, 但重复 $O(\log n)$ 次后, 所有元素同时被覆盖的概率迅速提高。

22.3 原始-对偶 (Primal-Dual)

22.3.1 直觉

直觉: ”同时构造原始解 (算法解) 和对偶解 (下界), 保证它们比例不超过 α ”。

原始-对偶不需要真正求解 LP, 而是通过调整对偶变量来驱动算法, 对偶可行解同时作为 OPT 的下界。

22.3.2 一般框架与松弛互补条件

考虑如下覆盖型线性规划及其对偶:

$$\begin{aligned} \min \quad & \sum_{j=1}^n c_j x_j \\ \text{s.t.} \quad & \sum_{j=1}^n a_{ij} x_j \geq b_i, \quad i = 1, \dots, m, \\ & x_j \geq 0, \quad j = 1, \dots, n, \end{aligned}$$

其对偶为:

$$\begin{aligned} \max \quad & \sum_{i=1}^m b_i y_i \\ \text{s.t.} \quad & \sum_{i=1}^m a_{ij} y_i \leq c_j, \quad j = 1, \dots, n, \\ & y_i \geq 0, \quad i = 1, \dots, m. \end{aligned}$$

这里原始问题是一个覆盖型问题: 每个约束要求

$$\sum_{j=1}^n a_{ij} x_j \geq b_i,$$

也就是说, 需要用变量 x_j 提供足够的 “覆盖量” 来满足需求 b_i 。

对偶问题则可以理解为一个打包型问题: 变量 y_i 可以看成第 i 个约束的 “价格”, 但是对于每个原始变量 x_j , 所有相关约束价格的加权和不能超过它的成本 c_j , 即

$$\sum_{i=1}^m a_{ij} y_i \leq c_j.$$

精确互补松弛条件 在线性规划中, 如果 x 和 y 分别是原始问题和对偶问题的最优解, 那么它们满足互补松弛条件。

对于这里的覆盖型原始问题和对偶问题, 互补松弛条件可以写成:

$$x_j > 0 \implies \sum_{i=1}^m a_{ij}y_i = c_j.$$

也就是说, 如果原始变量 x_j 被正地使用了, 那么它对应的对偶约束必须是紧的。直观地说, x_j 的成本是 c_j , 而

$$\sum_{i=1}^m a_{ij}y_i$$

可以理解为 x_j 带来的“对偶收益”。如果 $x_j > 0$, 那么在最优解中, 它的成本必须恰好等于它的对偶收益, 否则就存在改进空间。

另一组互补松弛条件是:

$$y_i > 0 \implies \sum_{j=1}^n a_{ij}x_j = b_i.$$

也就是说, 如果对偶变量 y_i 是正的, 那么对应的原始约束必须是紧的。

直观地说, 如果第 i 个约束在对偶中有正价格 y_i , 那么原始解中这个约束不能被过度满足, 必须恰好满足:

$$\sum_{j=1}^n a_{ij}x_j = b_i.$$

如果它被严格超过, 即

$$\sum_{j=1}^n a_{ij}x_j > b_i,$$

那么说明原始解在这个约束上有浪费, 而该约束又有正价格, 因此不符合最优性。

从精确互补松弛到近似互补松弛 在精确算法中, 我们希望找到完全满足互补松弛条件的原始可行解和对偶可行解。这样就可以证明它们都是最优解。

但是在近似算法中, 尤其是 primal-dual 算法中, 我们通常很难保证互补松弛条件完全成立。因此, 我们放宽这些条件, 只要求它们在某个因子范围内成立。

具体地, 假设我们构造出了一个原始可行解 x 和一个对偶可行解 y 。也就是说, 它们满足:

$$\sum_{j=1}^n a_{ij}x_j \geq b_i, \quad i = 1, \dots, m,$$

以及

$$\sum_{i=1}^m a_{ij}y_i \leq c_j, \quad j = 1, \dots, n.$$

接下来我们不要求互补松弛条件完全成立, 而是要求如下的松弛互补条件。

第一类条件是：如果 $x_j > 0$ ，那么对应的对偶约束虽然不一定完全紧，但是不能离紧太远。也就是说：

$$x_j > 0 \implies \frac{c_j}{\alpha} \leq \sum_{i=1}^m a_{ij} y_i \leq c_j.$$

其中右边的不等式

$$\sum_{i=1}^m a_{ij} y_i \leq c_j$$

本来就是对偶可行性。

真正额外要求的是左边：

$$\sum_{i=1}^m a_{ij} y_i \geq \frac{c_j}{\alpha}.$$

它表示：只要算法选择了变量 x_j ，那么这个变量对应的对偶约束至少要达到成本的 $1/\alpha$ 。也就是说，这个变量不是“太贵而没有对偶价值”的。

当 $\alpha = 1$ 时，这就退化为精确互补松弛条件：

$$\sum_{i=1}^m a_{ij} y_i = c_j.$$

第二类条件是：如果 $y_i > 0$ ，那么对应的原始约束虽然不一定完全紧，但是也不能被满足得过多。也就是说：

$$y_i > 0 \implies b_i \leq \sum_{j=1}^n a_{ij} x_j \leq \beta b_i.$$

其中左边的不等式

$$\sum_{j=1}^n a_{ij} x_j \geq b_i$$

本来就是原始可行性。

真正额外要求的是右边：

$$\sum_{j=1}^n a_{ij} x_j \leq \beta b_i.$$

它表示：如果对偶变量 y_i 是正的，那么第 i 个原始约束虽然可以被超过满足，但最多只能超过 β 倍。也就是说，原始解在这个约束上的“浪费”被控制在 β 倍以内。

当 $\beta = 1$ 时，这也退化为精确互补松弛条件：

$$\sum_{j=1}^n a_{ij} x_j = b_i.$$

为什么松弛互补条件可以推出近似比 下面证明：如果 x 是原始可行解， y 是对偶可行解，并且它们满足上述松弛互补条件，那么 x 的目标函数值至多是最优值的 $\alpha\beta$ 倍。

记原始问题的最优值为 OPT。由于 x 是原始可行解，所以它的成本为

$$\sum_{j=1}^n c_j x_j.$$

我们希望证明：

$$\sum_{j=1}^n c_j x_j \leq \alpha \beta \text{OPT}.$$

首先，由于对于所有满足 $x_j > 0$ 的变量，都有

$$\sum_{i=1}^m a_{ij} y_i \geq \frac{c_j}{\alpha},$$

因此

$$c_j \leq \alpha \sum_{i=1}^m a_{ij} y_i.$$

于是对所有 $x_j > 0$ 的项都有：

$$c_j x_j \leq \alpha \left(\sum_{i=1}^m a_{ij} y_i \right) x_j.$$

如果 $x_j = 0$ ，那么两边都为 0，上述不等式也不会影响求和。因此，对所有 j 求和可得：

$$\sum_{j=1}^n c_j x_j \leq \alpha \sum_{j=1}^n \left(\sum_{i=1}^m a_{ij} y_i \right) x_j.$$

接下来交换求和顺序：

$$\sum_{j=1}^n \left(\sum_{i=1}^m a_{ij} y_i \right) x_j = \sum_{j=1}^n \sum_{i=1}^m a_{ij} y_i x_j = \sum_{i=1}^m \sum_{j=1}^n a_{ij} x_j y_i.$$

因此：

$$\sum_{j=1}^n c_j x_j \leq \alpha \sum_{i=1}^m \left(\sum_{j=1}^n a_{ij} x_j \right) y_i.$$

现在使用第二类松弛互补条件。对于所有满足 $y_i > 0$ 的约束，有：

$$\sum_{j=1}^n a_{ij} x_j \leq \beta b_i.$$

因此：

$$\left(\sum_{j=1}^n a_{ij} x_j \right) y_i \leq \beta b_i y_i.$$

如果 $y_i = 0$ ，那么对应项为 0，同样不影响求和。因此：

$$\sum_{i=1}^m \left(\sum_{j=1}^n a_{ij} x_j \right) y_i \leq \sum_{i=1}^m \beta b_i y_i = \beta \sum_{i=1}^m b_i y_i.$$

代入前面的不等式，得到：

$$\sum_{j=1}^n c_j x_j \leq \alpha \beta \sum_{i=1}^m b_i y_i.$$

最后, 由弱对偶性可知, 任意对偶可行解的目标函数值都不超过原始问题的最优值, 即:

$$\sum_{i=1}^m b_i y_i \leq \text{OPT}.$$

所以:

$$\sum_{j=1}^n c_j x_j \leq \alpha \beta \sum_{i=1}^m b_i y_i \leq \alpha \beta \text{OPT}.$$

因此, 原始可行解 x 的成本最多是最优解成本的 $\alpha\beta$ 倍。也就是说, 算法得到的是一个 $\alpha\beta$ -近似解。

总结 primal-dual schema 的核心思想可以概括为:

- 构造一个原始可行解 x ;
- 同时构造一个对偶可行解 y ;
- 虽然不能保证它们满足精确互补松弛条件, 但保证它们满足松弛互补条件;
- 如果 $x_j > 0$, 则对应的对偶约束至少达到 $1/\alpha$ 的紧度;
- 如果 $y_i > 0$, 则对应的原始约束最多被超过满足 β 倍;
- 于是原始解的成本可以用对偶解的价值控制;
- 再由弱对偶性, 对偶解的价值不超过最优值;
- 最终得到 $\alpha\beta$ 的近似比。

因此, 松弛互补条件可以看成是精确互补松弛条件的近似版本。精确互补松弛用于证明最优性, 而松弛互补条件用于证明近似最优性。

22.3.3 Vertex Cover 的原始-对偶算法

给定无向图 $G = (V, E)$, Vertex Cover 问题要求选择一个点集 $C \subseteq V$, 使得每条边都至少有一个端点在 C 中, 即

$$\forall (u, v) \in E, \quad u \in C \text{ 或 } v \in C,$$

并且希望最小化 $|C|$ 。

我们先写出 Vertex Cover 的整数规划。令变量 $x_v \in \{0, 1\}$ 表示点 v 是否被选入覆盖:

$$x_v = \begin{cases} 1, & v \in C, \\ 0, & v \notin C. \end{cases}$$

那么 Vertex Cover 可以写成:

$$\min \sum_{v \in V} x_v,$$

满足

$$x_u + x_v \geq 1, \quad \forall (u, v) \in E,$$

$$x_v \in \{0, 1\}, \quad \forall v \in V.$$

其中约束 $x_u + x_v \geq 1$ 表示边 (u, v) 至少有一个端点被选中。
将整数约束 $x_v \in \{0, 1\}$ 放松为 $x_v \geq 0$, 得到线性规划松弛:

$$\min \sum_{v \in V} x_v,$$

满足

$$x_u + x_v \geq 1, \quad \forall (u, v) \in E,$$

$$x_v \geq 0, \quad \forall v \in V.$$

这是原始 LP。它的对偶 LP 为:

$$\max \sum_{e \in E} y_e,$$

满足

$$\sum_{e \ni v} y_e \leq 1, \quad \forall v \in V,$$

$$y_e \geq 0, \quad \forall e \in E.$$

这里每条边 e 对应一个对偶变量 y_e 。对偶约束

$$\sum_{e \ni v} y_e \leq 1$$

表示: 所有与顶点 v 相邻的边的对偶变量之和不能超过 1。

直观理解 原始问题是在选择顶点; 对偶问题可以理解为在给边分配权重。每条边 e 有一个权重 y_e , 但对于任意顶点 v , 所有和 v 相邻的边权重之和最多为 1。

原始-对偶算法的思想是:

- 一开始不选任何顶点, 即 $C = \emptyset$ 。
- 找到一条还没有被覆盖的边 (u, v) 。
- 因为这条边还没有被覆盖, 所以 u, v 都还没有被选入 C 。
- 我们增大这条边对应的对偶变量 y_{uv} 。
- 一直增大到 u 或 v 的某个对偶约束变紧, 也就是

$$\sum_{e \ni u} y_e = 1 \quad \text{或} \quad \sum_{e \ni v} y_e = 1.$$

- 哪个顶点的约束变紧, 就把哪个顶点加入 Vertex Cover。

如果 u 和 v 的约束同时变紧, 则可以把二者都加入覆盖。

算法

输入： 无向图 $G = (V, E)$.

输出： 一个 vertex cover C .

1. 初始化：

$$C \leftarrow \emptyset, \quad y_e \leftarrow 0, \quad \forall e \in E.$$

2. 当存在一条未被覆盖的边 $(u, v) \in E$ 时，执行以下操作。

一条边 (u, v) 未被覆盖，指的是

$$u \notin C \quad \text{且} \quad v \notin C.$$

3. 增大这条边的对偶变量 y_{uv} 。

由于我们只增大 y_{uv} ，因此只有 u 和 v 的对偶约束会受到影响：

$$\sum_{e \ni u} y_e \leq 1, \quad \sum_{e \ni v} y_e \leq 1.$$

设当前

$$a_u = \sum_{e \ni u} y_e, \quad a_v = \sum_{e \ni v} y_e.$$

我们可以把 y_{uv} 增加

$$\delta = \min\{1 - a_u, 1 - a_v\}.$$

这样增加之后，至少有一个端点的对偶约束会变紧。

4. 更新

$$y_{uv} \leftarrow y_{uv} + \delta.$$

5. 如果

$$\sum_{e \ni u} y_e = 1,$$

则将 u 加入覆盖：

$$C \leftarrow C \cup \{u\}.$$

如果

$$\sum_{e \ni v} y_e = 1,$$

则将 v 加入覆盖：

$$C \leftarrow C \cup \{v\}.$$

6. 重复以上过程，直到不存在未覆盖边为止。

最后输出 C 。

为什么算法结束后得到的是一个 Vertex Cover ? 算法只有在存在未覆盖边时才继续执行。也就是说, 只要还有一条边 (u, v) 满足

$$u \notin C, \quad v \notin C,$$

算法就不会停止。

因此, 当算法停止时, 不存在未覆盖边。换句话说, 对于每条边 $(u, v) \in E$, 至少有一个端点在 C 中:

$$u \in C \quad \text{或} \quad v \in C.$$

这正是 Vertex Cover 的定义, 所以算法输出的 C 一定是一个合法的 vertex cover。

为什么对偶解始终可行 ? 算法中只会增加某一条边 (u, v) 的对偶变量 y_{uv} 。每次增加时, 我们选择

$$\delta = \min\{1 - a_u, 1 - a_v\},$$

其中

$$a_u = \sum_{e \ni u} y_e, \quad a_v = \sum_{e \ni v} y_e.$$

因此增加之后有

$$\sum_{e \ni u} y_e \leq 1, \quad \sum_{e \ni v} y_e \leq 1.$$

其他顶点的对偶约束不受影响, 所以也仍然满足。

因此在整个算法过程中, 对偶解 y 始终满足

$$\sum_{e \ni v} y_e \leq 1, \quad \forall v \in V,$$

并且显然

$$y_e \geq 0, \quad \forall e \in E.$$

所以 y 始终是一个可行的对偶解。

近似比分析 我们证明算法输出的点集 C 满足

$$|C| \leq 2 \cdot OPT,$$

其中 OPT 是最优 Vertex Cover 的大小。

首先, 由弱对偶性, 对任意可行的对偶解 y , 都有

$$\sum_{e \in E} y_e \leq OPT_{LP} \leq OPT,$$

其中 OPT_{LP} 是原始 LP 的最优值, 而 OPT 是整数 Vertex Cover 的最优值。

接下来分析算法输出的 $|C|$ 。

算法中, 一个顶点 v 只有在它的对偶约束变紧时才会被加入 C 。也就是说, 如果 $v \in C$, 则

$$\sum_{e \ni v} y_e = 1.$$

因此

$$|C| = \sum_{v \in C} 1 = \sum_{v \in C} \sum_{e \ni v} y_e.$$

现在交换求和顺序。每条边 $e = (u, v)$ 最多会在上式中被计算两次：一次来自端点 u ，一次来自端点 v 。所以

$$\sum_{v \in C} \sum_{e \ni v} y_e \leq 2 \sum_{e \in E} y_e.$$

因此

$$|C| \leq 2 \sum_{e \in E} y_e.$$

又因为 y 是一个可行对偶解，根据弱对偶性有

$$\sum_{e \in E} y_e \leq OPT.$$

于是

$$|C| \leq 2 \sum_{e \in E} y_e \leq 2OPT.$$

所以该原始-对偶算法是一个 2-近似算法。

一个小例子 假设当前有一条未覆盖边 (u, v) ，并且目前

$$\sum_{e \ni u} y_e = 0.3, \quad \sum_{e \ni v} y_e = 0.7.$$

那么 u 的约束距离变紧还差

$$1 - 0.3 = 0.7,$$

而 v 的约束距离变紧还差

$$1 - 0.7 = 0.3.$$

因此我们最多只能把 y_{uv} 增加

$$\delta = \min\{0.7, 0.3\} = 0.3.$$

增加之后，

$$\sum_{e \ni u} y_e = 0.6, \quad \sum_{e \ni v} y_e = 1.$$

此时 v 的对偶约束变紧，因此把 v 加入覆盖 C 。

加入 v 后，所有与 v 相邻的边都被覆盖。算法继续寻找其他未覆盖的边。

22.3.4 完整例子

图： $V = \{1, 2, 3\}$, $E = \{(1, 2), (2, 3), (1, 3)\}$ (三角形)，初始 $y_{12} = y_{23} = y_{13} = 0$ 。

第 1 步：选未覆盖边 $(1, 2)$ ，增大 y_{12} ：

- 顶点 1 的对偶约束： $y_{12} + y_{13} \leq 1$ ，当 $y_{12} = 1$ 时达到紧（若 $y_{13} = 0$ ）。
- 顶点 2 的对偶约束： $y_{12} + y_{23} \leq 1$ ，同上。

- 增大 y_{12} 到 1, 顶点 1 和顶点 2 同时紧, 加入覆盖。 $C = \{1, 2\}$ 。

第 2 步: 检查未覆盖边: (2,3) (端点 2 已在覆盖), 已覆盖; (1,3) (端点 1 已在覆盖), 已覆盖。算法结束。

输出覆盖 $C = \{1, 2\}$, 大小 2。

对偶下界为 $\sum_e y_e = 1$, 因此由弱对偶 $\text{OPT} \geq 1$ 。该三角形实例的真实最优值为 2: 任取一个顶点只能覆盖两条边, 三条边必须至少选两个顶点。算法输出 $C = \{1, 2\}$, 确实是一个最优覆盖。

$|C| = 2 \leq 2 \sum_e y_e = 2$, 2-近似。

2-近似的证明: 加入覆盖的每个顶点 v 有对偶约束 $\sum_{e \ni v} y_e = 1$ (紧), 所以 $|C| = \sum_{v \in C} 1 = \sum_{v \in C} \sum_{e \ni v} y_e \leq 2 \sum_e y_e$ (每条边 $e = (u, v)$ 被计 ≤ 2 次) $\leq 2\text{OPT}$ 。

22.3.5 Set Cover 的原始-对偶算法

Set Cover 的 LP 和对偶为

$$\begin{array}{ll} \min & \sum_{S \in \mathcal{S}} c(S) x_S \\ \text{s.t.} & \sum_{S: e \in S} x_S \geq 1, \quad e \in U, \\ & x_S \geq 0, \end{array} \quad \begin{array}{ll} \max & \sum_{e \in U} y_e \\ \text{s.t.} & \sum_{e \in S} y_e \leq c(S), \quad S \in \mathcal{S}, \\ & y_e \geq 0. \end{array}$$

若每个元素最多出现在 f 个集合中, 则原始-对偶算法如下:

1. 初始 $x = 0, y = 0$, 所有元素未覆盖。
2. 选一个未覆盖元素 e , 连续增大 y_e , 直到某些包含 e 的集合变紧。
3. 把所有变紧集合加入解, 并把这些集合中的元素都标记为已覆盖。
4. 重复直到所有元素都覆盖。

算法只选择紧集合, 所以 primal 条件取 $\alpha = 1$ 。若 $y_e > 0$, 说明 e 曾经作为未覆盖元素被选中; 最终它可能被多个集合覆盖, 但由于它总共只属于至多 f 个集合, 故

$$\sum_{S: e \in S} x_S \leq f.$$

于是 dual 条件取 $\beta = f$, 得到 f -近似。

例子: 令 $U = \{a, b, c\}$, $S_1 = \{a, b\}, S_2 = \{b, c\}, S_3 = \{a, c\}$, 单位成本, 频率 $f = 2$ 。初始选未覆盖元素 a , 增大 y_a 到 1 时, S_1, S_3 都变紧, 算法加入 S_1, S_3 。此时 a, b, c 全部覆盖, 结束, 成本 2。对偶值为 $y_a = 1$, 给出下界 1, 算法成本 $2 \leq f \cdot 1$ 。实际最优也是 2。

22.4 Dual Fitting

Dual Fitting 是分析近似算法的一种常用方法, 尤其适合分析 Set Cover 这类问题的贪心算法。

它的核心思想可以概括为:

先不管对偶约束, 运行一个自然的贪心算法。

在算法运行过程中，给元素分配一些“价格”或“费用”。这些价格的总和正好等于算法的总代价。

这些价格可以看成是一个“候选对偶解”，但它可能不满足对偶约束。

最后证明：虽然它不一定对偶可行，但它最多违反对偶约束一个 H_n 倍。

因此把所有价格整体除以 H_n ，就得到一个真正的对偶可行解。利用弱对偶性，这个对偶可行解给出最优解的下界，从而证明贪心算法是 H_n 近似。

这里 H_n 是第 n 个调和数：

$$H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}.$$

它满足

$$H_n = O(\log n).$$

22.4.1 Set Cover 的 LP 与对偶

Set Cover 问题如下。

给定一个元素全集

$$U = \{1, 2, \dots, n\},$$

以及若干集合

$$S_1, S_2, \dots, S_m \subseteq U.$$

每个集合 S_i 有一个非负成本 $c(S_i)$ 。

目标是选择一些集合，使得所有元素都被覆盖，并且总成本最小。

也就是说，我们希望最小化

$$\sum_{i: S_i \text{ 被选}} c(S_i).$$

Set Cover 可以写成如下整数规划：

$$\begin{aligned} \min \quad & \sum_{i=1}^m c(S_i) x_i \\ \text{s.t.} \quad & \sum_{i: e \in S_i} x_i \geq 1, \quad \forall e \in U, \\ & x_i \in \{0, 1\}, \quad \forall i. \end{aligned}$$

其中 $x_i = 1$ 表示选择集合 S_i ， $x_i = 0$ 表示不选择。

把整数约束放松成 $x_i \geq 0$ ，得到线性规划松弛：

$$\begin{aligned} \min \quad & \sum_{i=1}^m c(S_i) x_i \\ \text{s.t.} \quad & \sum_{i: e \in S_i} x_i \geq 1, \quad \forall e \in U, \\ & x_i \geq 0, \quad \forall i. \end{aligned}$$

它的对偶是：

$$\begin{aligned} \max \quad & \sum_{e \in U} y_e \\ \text{s.t.} \quad & \sum_{e \in S_i} y_e \leq c(S_i), \quad \forall i, \\ & y_e \geq 0, \quad \forall e \in U. \end{aligned}$$

对偶变量 y_e 可以理解为给每个元素 e 标一个“价值”或者“价格”。

对偶约束

$$\sum_{e \in S_i} y_e \leq c(S_i)$$

的意思是：

任意一个集合 S_i 里面所有元素的总价值，不能超过这个集合的成本。

根据弱对偶性，任何对偶可行解 y 都给出原问题最优解的下界：

$$\sum_{e \in U} y_e \leq \text{OPT}_f \leq \text{OPT},$$

其中 OPT_f 是 LP 松弛的最优值， OPT 是整数问题的最优值。

因此，如果我们能构造出一个对偶可行解 y ，并且贪心算法的代价可以用 $\sum_e y_e$ 来控制，就能证明近似比。

22.4.2 Set Cover 的贪心算法

Set Cover 的经典贪心算法如下。

设 U' 表示当前还没有被覆盖的元素集合。初始时

$$U' = U.$$

每一步，贪心算法选择一个“平均覆盖成本”最小的集合。也就是说，在所有还能够覆盖新元素的集合 S_i 中，选择使得

$$\frac{c(S_i)}{|S_i \cap U'|}$$

最小的集合。

这里 $S_i \cap U'$ 表示集合 S_i 当前还能新覆盖的元素。

因此

$$\frac{c(S_i)}{|S_i \cap U'|}$$

可以理解为：如果现在选择 S_i ，那么它每新覆盖一个元素的平均成本是多少。

贪心算法每次选择平均成本最低的集合。

22.4.3 给元素分配价格

Dual Fitting 的关键是：在贪心算法运行过程中，给每个元素分配一个价格。

假设某一步贪心选择了集合 S_i 。

当前还未覆盖的元素集合是 U' ，所以这一轮新覆盖的元素是

$$S_i \cap U'.$$

贪心算法为这一轮支付成本 $c(S_i)$ 。

我们把这个成本平均分摊给这一轮新覆盖的所有元素。

也就是说，对于每个新覆盖的元素 $e \in S_i \cap U'$ ，定义

$$\alpha_e = \frac{c(S_i)}{|S_i \cap U'|}.$$

这个 α_e 就是元素 e 被覆盖时分到的价格。

注意，每个元素只会在它第一次被覆盖时被分配一次价格。

因为算法最终会覆盖所有元素，所以每个元素 $e \in U$ 都有一个价格 α_e 。

22.4.4 价格总和等于贪心算法代价

为什么要这样定义价格？

原因是这样一来，所有元素的价格总和恰好等于贪心算法的总代价。

假设某一步贪心选择了集合 S_i ，并且这一步新覆盖了

$$|S_i \cap U'|$$

个元素。

每个新覆盖元素被分配价格

$$\frac{c(S_i)}{|S_i \cap U'|}.$$

因此这一轮新覆盖元素的价格总和是

$$|S_i \cap U'| \cdot \frac{c(S_i)}{|S_i \cap U'|} = c(S_i).$$

也就是说，每一轮分配出去的价格总和，正好等于这一轮选择集合的成本。

把所有轮加起来，就得到

$$ALG = \sum_{e \in U} \alpha_e.$$

这里 ALG 表示贪心算法的总代价。

所以， α_e 是一个非常自然的候选对偶变量。

如果直接令

$$y_e = \alpha_e,$$

那么对偶目标值就是

$$\sum_{e \in U} y_e = \sum_{e \in U} \alpha_e = ALG.$$

如果这个 y 是对偶可行的, 那么我们立刻就能得到

$$ALG = \sum_e y_e \leq OPT,$$

这甚至说明贪心是最优的。

但这显然不可能, 因为 Set Cover 的贪心算法并不总是最优。

问题出在哪里?

问题在于: 直接令 $y_e = \alpha_e$ 时, 得到的向量 α 不一定满足对偶约束。

也就是说, 可能存在某个集合 S , 使得

$$\sum_{e \in S} \alpha_e > c(S).$$

这就违反了对偶约束。

22.4.5 为什么价格可能不是对偶可行的

对偶约束要求对每个集合 S ,

$$\sum_{e \in S} y_e \leq c(S).$$

如果令 $y_e = \alpha_e$, 就要求

$$\sum_{e \in S} \alpha_e \leq c(S).$$

但是 α_e 是在贪心运行过程中由不同轮次分配出来的。

一个集合 S 中的元素可能在不同时间被不同集合覆盖。

后面被覆盖的元素可能价格很高, 因为越到后面, 剩下的未覆盖元素越少, 平均成本可能越大。

因此, 一个集合 S 内所有元素的价格加起来, 可能超过 S 自己的成本。

Dual Fitting 的关键就是证明:

虽然 α 不一定对偶可行, 但是对于任意集合 S ,

$$\sum_{e \in S} \alpha_e \leq H_n \cdot c(S).$$

这说明 α 至多违反每个对偶约束 H_n 倍。

于是我们可以把所有价格缩小 H_n 倍, 定义

$$y_e = \frac{\alpha_e}{H_n}.$$

这样就有

$$\sum_{e \in S} y_e = \frac{1}{H_n} \sum_{e \in S} \alpha_e \leq \frac{1}{H_n} \cdot H_n c(S) = c(S).$$

所以 y 就是一个真正的对偶可行解。

22.4.6 关键证明：任意集合的价格和最多超过 H_n 倍

现在证明最关键的一步：

对于任意集合 S ，

$$\sum_{e \in S} \alpha_e \leq H_n c(S).$$

事实上，如果 $|S| = k$ ，可以证明更强的结论：

$$\sum_{e \in S} \alpha_e \leq H_k c(S).$$

因为 $k \leq n$ ，所以

$$H_k \leq H_n.$$

下面给出证明。

固定任意一个集合 S ，设

$$|S| = k.$$

考虑集合 S 中的元素被贪心算法覆盖的先后顺序。

把它们按照被覆盖的时间从早到晚排列为

$$e_1, e_2, \dots, e_k.$$

也就是说，在 S 中， e_1 是最早被覆盖的元素， e_2 是第二早被覆盖的元素，依此类推， e_k 是最后被覆盖的元素。

现在看元素 e_i 被覆盖的那一轮。

在 e_i 被覆盖之前，集合 S 中已经有

$$e_1, \dots, e_{i-1}$$

这些元素被覆盖了。

因此，集合 S 中至少还有

$$e_i, e_{i+1}, \dots, e_k$$

这 $k - i + 1$ 个元素尚未被覆盖。

换句话说，在那一轮开始时，

$$|S \cap U'| \geq k - i + 1.$$

如果那一轮贪心算法选择集合 S ，那么它能够新覆盖至少 $k - i + 1$ 个元素，所以它的平均覆盖成本至多是

$$\frac{c(S)}{k - i + 1}.$$

注意，这里不是说贪心真的选择了 S 。

贪心可能选择了别的集合。

但是贪心选择的是当前平均成本最小的集合。

因此，贪心实际选择的集合的平均成本一定不超过集合 S 的平均成本。

而元素 e_i 被覆盖时的价格 α_{e_i} ，正是贪心实际选择集合的平均成本。

所以有

$$\alpha_{e_i} \leq \frac{c(S)}{k-i+1}.$$

对所有 $i = 1, 2, \dots, k$ 求和, 得到

$$\begin{aligned} \sum_{e \in S} \alpha_e &= \sum_{i=1}^k \alpha_{e_i} \\ &\leq \sum_{i=1}^k \frac{c(S)}{k-i+1} \\ &= c(S) \sum_{i=1}^k \frac{1}{k-i+1}. \end{aligned}$$

令

$$t = k - i + 1.$$

当 $i = 1$ 时, $t = k$; 当 $i = k$ 时, $t = 1$ 。

所以

$$\sum_{i=1}^k \frac{1}{k-i+1} = \frac{1}{k} + \frac{1}{k-1} + \dots + \frac{1}{1} = H_k.$$

因此

$$\sum_{e \in S} \alpha_e \leq H_k c(S) \leq H_n c(S).$$

这就证明了: 对于任意集合 S ,

$$\sum_{e \in S} \alpha_e \leq H_n c(S).$$

22.4.7 缩放得到对偶可行解

现在定义

$$y_e = \frac{\alpha_e}{H_n}.$$

我们验证 y 是对偶可行的。

对于任意集合 S , 有

$$\begin{aligned} \sum_{e \in S} y_e &= \sum_{e \in S} \frac{\alpha_e}{H_n} \\ &= \frac{1}{H_n} \sum_{e \in S} \alpha_e \\ &\leq \frac{1}{H_n} \cdot H_n c(S) \\ &= c(S). \end{aligned}$$

并且显然

$$y_e \geq 0.$$

所以 y 满足对偶 LP 的所有约束, 是一个对偶可行解。

22.4.8 利用弱对偶性证明近似比

因为 y 是对偶可行解，所以由弱对偶性，

$$\sum_{e \in U} y_e \leq \text{OPT}_f \leq \text{OPT}.$$

另一方面，根据定义

$$y_e = \frac{\alpha_e}{H_n},$$

所以

$$\alpha_e = H_n y_e.$$

于是贪心算法总代价为

$$\begin{aligned} ALG &= \sum_{e \in U} \alpha_e \\ &= \sum_{e \in U} H_n y_e \\ &= H_n \sum_{e \in U} y_e \\ &\leq H_n \text{OPT}_f \\ &\leq H_n \text{OPT}. \end{aligned}$$

因此，Set Cover 的贪心算法是一个 H_n 近似算法。

由于

$$H_n = O(\log n),$$

所以它也是一个 $O(\log n)$ 近似算法。

22.4.9 一个具体例子

考虑如下 Set Cover 实例：

$$U = \{e, f, g\},$$

有三个集合：

$$S_1 = \{e, f\}, \quad S_2 = \{f, g\}, \quad S_3 = \{e, g\}.$$

每个集合的成本都是 1。

也就是说，

$$c(S_1) = c(S_2) = c(S_3) = 1.$$

贪心算法第一步会选择一个平均成本最小的集合。

一开始所有集合都能覆盖两个新元素，因此每个集合的平均成本都是

$$\frac{1}{2}.$$

假设贪心第一步选择

$$S_1 = \{e, f\}.$$

那么这一轮新覆盖元素 e, f , 集合成本为 1, 所以每个新覆盖元素分到价格

$$\alpha_e = \alpha_f = \frac{1}{2}.$$

此时还没有覆盖的元素只剩下

$$g.$$

第二步贪心可以选择 $S_2 = \{f, g\}$ 或者 $S_3 = \{e, g\}$ 来覆盖 g 。

假设选择

$$S_2 = \{f, g\}.$$

这一轮新覆盖元素只有 g , 集合成本为 1, 所以

$$\alpha_g = 1.$$

于是所有元素的价格为

$$\alpha_e = \frac{1}{2}, \quad \alpha_f = \frac{1}{2}, \quad \alpha_g = 1.$$

价格总和为

$$\alpha_e + \alpha_f + \alpha_g = \frac{1}{2} + \frac{1}{2} + 1 = 2.$$

这正好等于贪心算法选了两个集合的总代价:

$$ALG = 1 + 1 = 2.$$

现在检查这些价格是否构成一个对偶可行解。

对偶约束要求每个集合内部元素价格之和不超过集合成本 1。

对于集合 $S_1 = \{e, f\}$,

$$\alpha_e + \alpha_f = \frac{1}{2} + \frac{1}{2} = 1.$$

这个约束满足。

对于集合 $S_2 = \{f, g\}$,

$$\alpha_f + \alpha_g = \frac{1}{2} + 1 = \frac{3}{2}.$$

但是

$$c(S_2) = 1.$$

所以

$$\alpha_f + \alpha_g > c(S_2).$$

这说明直接令 $y_e = \alpha_e$ 不是对偶可行的。

类似地, 对于集合 $S_3 = \{e, g\}$,

$$\alpha_e + \alpha_g = \frac{1}{2} + 1 = \frac{3}{2} > 1.$$

所以也违反对偶约束。

根据 Dual Fitting 的思想，我们把所有价格除以

$$H_3 = 1 + \frac{1}{2} + \frac{1}{3} = \frac{11}{6}.$$

定义

$$y_e = \frac{\alpha_e}{H_3}.$$

于是

$$y_e = y_f = \frac{1/2}{11/6} = \frac{3}{11},$$

并且

$$y_g = \frac{1}{11/6} = \frac{6}{11}.$$

检查集合约束。

对于 $S_1 = \{e, f\}$:

$$y_e + y_f = \frac{3}{11} + \frac{3}{11} = \frac{6}{11} \leq 1.$$

对于 $S_2 = \{f, g\}$:

$$y_f + y_g = \frac{3}{11} + \frac{6}{11} = \frac{9}{11} \leq 1.$$

对于 $S_3 = \{e, g\}$:

$$y_e + y_g = \frac{3}{11} + \frac{6}{11} = \frac{9}{11} \leq 1.$$

所以缩放之后， y 确实是对偶可行解。

这个例子体现了 Dual Fitting 的典型过程：

1. 贪心运行时给元素分配价格 α_e ;
2. 这些价格总和等于贪心算法成本;
3. 但是 α 不一定满足对偶约束;
4. 证明 α 最多违反约束 H_n 倍;
5. 将 α 整体除以 H_n , 得到对偶可行解 y ;
6. 用弱对偶性证明贪心算法是 H_n 近似。

22.4.10 Dual Fitting 与原始-对偶方法的区别

Dual Fitting 和原始-对偶方法很像，但它们的分析方向不同。

在原始-对偶方法中，我们通常一边构造原始解，一边维护一个对偶可行解。对偶变量的增长过程始终受到对偶约束的控制。

而在 Dual Fitting 中，我们一开始不要求对偶可行。

我们先运行算法，并根据算法行为构造一个“看起来像对偶解”的变量 α 。

这个 α 可能违反对偶约束。

然后我们事后分析它违反约束的程度。

如果能证明它最多违反 ρ 倍，那么把它整体除以 ρ ，就得到一个真正的对偶可行解。

因此 Dual Fitting 的逻辑是：

先构造不一定可行的对偶解 $\implies$ 证明违反程度有限 $\implies$ 缩放成可行解.

对于 Set Cover 的贪心算法，这个违反程度就是 H_n 。

核心记忆 LP 近似三种语言本质相连：舍入处理分数原解（正向）；原始-对偶同步构造解和下界（双向）；Dual fitting 事后修正下界（反向验证）。理解这三者的联系，就掌握了 LP 近似算法的精华。

第二十三章 MAX-SAT

对应课件：23.pdf

主题：随机化、去随机化、LP 舍入

动机：SAT 的优化版本与随机化的典范

MAX-SAT 是 SAT 的优化版本：不要求所有子句都满足，而是**最大化满足的子句总权重**。它是随机化近似算法的绝佳教学案例——从“抛硬币也能保证一半”出发，逐步改进，最后得到 $3/4$ -近似。

23.1 问题定义

定义 23.1. MAX-SAT: 给定 CNF 公式 $\varphi = C_1 \wedge \cdots \wedge C_m$ ，每个子句 C_j 有权重 $w_j \geq 0$ ，目标是找变量赋值最大化 $\sum_{j: C_j \text{ 满足}} w_j$ 。

MAX-SAT 是 NP-hard (SAT 是特殊情形：若所有子句均可满足，MAX-SAT 解为总权重)。

23.2 随机算法：1/2-近似

23.2.1 直觉：“抛硬币也能保证一半”

最简单的算法：每个变量独立以概率 $1/2$ 取真。每个子句被满足的概率至少 $1/2$ (因为子句内任意一个字面量为真就满足，至少有一个字面量以 $1/2$ 的概率为真，且子句长 ≥ 1)。

定理 23.1. 随机赋值算法是 $1/2$ -近似 (MAX-SAT 为最大化问题，等价于近似比 $1/2$)。

证明. 子句 C_j 含 k_j 个字面量，不满足 C_j 的概率 $= 2^{-k_j}$ (需所有字面量均为假)，满足概率 $\geq 1 - 2^{-k_j} \geq 1/2$ 。期望满足权重 $= \sum_j w_j \cdot \Pr[C_j \text{ 满足}] \geq \frac{1}{2} \sum_j w_j \geq \frac{1}{2} \text{OPT}$ 。 $\square$

23.2.2 更细的长度分析

若每个子句长度至少为 k ，则公平随机赋值满足每个子句的概率至少

$$1 - 2^{-k}.$$

因此在这类实例上可得到 $(1 - 2^{-k})$ -近似。例如 MAX-E3SAT 中每个子句恰有 3 个字面量，随机赋值满足任一子句的概率为

$$1 - 2^{-3} = 7/8.$$

课件还指出一个硬度事实：若对任意常数 $\varepsilon > 0$ ，MAX-E3SAT 存在 $(7/8 + \varepsilon)$ -近似算法，则 $P = NP$ 。这说明 $7/8$ 不是单纯分析粗糙造成的，而是与问题本身的近似难度有关。

23.2.3 3 变量 4 子句的具体计算

公式（均等权重 $w_j = 1$ ）：

$$C_1 = (x_1 \vee \neg x_2), \quad C_2 = (\neg x_1 \vee x_3), \quad C_3 = (x_2 \vee \neg x_3), \quad C_4 = (x_1 \vee x_2 \vee x_3).$$

随机赋值，子句长度： $k_1 = 2, k_2 = 2, k_3 = 2, k_4 = 3$ 。

期望满足子句数：

$$\begin{aligned} E[\text{满足}] &= \sum_j (1 - 2^{-k_j}) = (1 - 1/4) + (1 - 1/4) + (1 - 1/4) + (1 - 1/8) \\ &= 3/4 + 3/4 + 3/4 + 7/8 = 3.125. \end{aligned}$$

总权重 = 4，期望 $\geq 3.125 > 4/2 = 2$ ，超过 $1/2$ 近似的保证。

23.3 条件期望去随机化（本章精华）

23.3.1 直觉：“逐步固定，选更好的一侧”

随机算法证明了存在一个赋值使满足权重 $\geq \text{OPT}/2$ ，但我们想要确定性地找到这样的赋值。条件期望方法：逐个固定变量，每次选让条件期望不下降的一侧，最终得到的确定性赋值满足权重不低于初始期望。

```
DERANDOMIZE-MAX-SAT(公式 F, 变量  $x_1, \dots, x_n$ ):
    当前赋值 alpha = {}
    for i = 1 to n:
         $E_T = E[\text{满足权重} \mid \text{alpha}, x_i = T]$  // 条件期望
         $E_F = E[\text{满足权重} \mid \text{alpha}, x_i = F]$ 
        if  $E_T \geq E_F$ :
            alpha( $x_i$ ) = T
        else:
            alpha( $x_i$ ) = F
    return alpha
```

关键性质： $\max(E_T, E_F) \geq (E_T + E_F)/2 = E[\text{满足权重} \mid \text{alpha}]$ 。所以每步条件期望不降，最终赋值的满足权重 $\geq$ 初始期望 $\geq \text{OPT}/2$ 。

23.3.2 完整例子：逐步固定

使用上面的例子 ($k_1 = k_2 = k_3 = 2, k_4 = 3$, 初始期望 = 3.125)。

第 1 步：固定 x_1

$x_1 = \text{T}$:

- $C_1 = (T \vee \neg x_2)$: 已满足, 贡献 1。
- $C_2 = (F \vee x_3) = (x_3)$: 随机 x_3 , 满足概率 1/2, 期望 1/2。
- $C_3 = (x_2 \vee \neg x_3)$: 随机, 期望 3/4。
- $C_4 = (T \vee x_2 \vee x_3)$: 已满足, 贡献 1。

$$E_T = 1 + 1/2 + 3/4 + 1 = 3.25。$$

$x_1 = \text{F}$:

- $C_1 = (F \vee \neg x_2) = (\neg x_2)$: 期望 1/2。
- $C_2 = (T \vee x_3)$: 已满足, 贡献 1。
- $C_3 = (x_2 \vee \neg x_3)$: 期望 3/4。
- $C_4 = (F \vee x_2 \vee x_3) = (x_2 \vee x_3)$: 期望 3/4。

$$E_F = 1/2 + 1 + 3/4 + 3/4 = 3.0。$$

$E_T = 3.25 > E_F = 3.0$, 选 $x_1 = \text{T}$ 。

第 2 步：固定 x_2 (已知 $x_1 = \text{T}$)

当前公式 (化简 $x_1 = \text{T}$): C_1 已满足, $C_2 = (x_3)$, $C_3 = (x_2 \vee \neg x_3)$, C_4 已满足。

$x_2 = \text{T}$: C_3 已满足, $C_2 = (x_3)$ 期望 1/2。总期望 = $1 + 1/2 + 1 + 1 = 3.5$ 。

$x_2 = \text{F}$: $C_3 = (\neg x_3)$, $C_2 = (x_3)$, 两者矛盾, 必有一假。 $x_3 = \text{T}$: C_2 满足, C_3 不满足; $x_3 = \text{F}$: C_2 不满足, C_3 满足。期望满足子句数 (C_3 和 C_2) = 1。总 = $1 + 1 + 1 = 3$ (加上 C_1, C_4)。

$E_T = 3.5 > E_F = 3$, 选 $x_2 = \text{T}$ 。

第 3 步：固定 x_3 (已知 $x_1 = x_2 = \text{T}$)

剩余未确定子句: $C_2 = (x_3)$ (C_1, C_3, C_4 已满足)。

$x_3 = \text{T}$: C_2 满足, 总满足 = 4。 $x_3 = \text{F}$: C_2 不满足, 总满足 = 3。

选 $x_3 = \text{T}$ 。

最终赋值: $x_1 = \text{T}, x_2 = \text{T}, x_3 = \text{T}$, 满足所有 4 个子句, 总满足权重 = 4 = 全部权重。

步骤	固定变量	赋值	E_T	E_F
1	x_1	T	3.25	3.0
2	x_2	T	3.5	3.0
3	x_3	T	4	3

条件期望单调不降 ($3.125 \rightarrow 3.25 \rightarrow 3.5 \rightarrow 4$), 最终达到最优。

23.4 偏置随机：Flipping Biased Coins

公平硬币并不是唯一选择。设每个变量独立地以概率 $p \geq 1/2$ 取真。若实例中没有负单位子句（形如 $\neg x_i$ 的单文字子句），则每个子句被满足的概率至少

$$\min(p, 1 - p^2).$$

证明分两类：

- 若子句是正单位子句 x_i ，满足概率为 p 。
- 若子句长度至少为 2，设其中有 a 个负文字、 b 个正文字。不满足概率为 $p^a(1-p)^b$ 。因为 $p \geq 1/2 \geq 1-p$ 且 $a+b \geq 2$ ，该乘积至多 p^2 ，所以满足概率至少 $1-p^2$ 。

为了最大化最坏保证，令 $p = 1 - p^2$ ，得到

$$p = \frac{\sqrt{5}-1}{2} \approx 0.618.$$

因此在无负单位子句的实例上，可得到约 0.618-近似，比 $1/2$ 更好。

负单位子句需要额外处理。若只有 $\neg x_i$ 而没有对应正单位子句 x_i ，可以引入新变量 y ，把所有 $\neg x_i$ 替换为 y 、所有 x_i 替换为 $\neg y$ ，从而把这个负单位子句变成正单位子句。若同时存在 $\neg x_i$ 和 x_i ，二者无法同时满足。令 v_i 为负单位子句 $\neg x_i$ 的权重（若不存在则为 0），并可假设其权重不超过对应正单位子句权重，则

$$\text{OPT} \leq \sum_j w_j - \sum_i v_i.$$

采用上面的 $p = (\sqrt{5}-1)/2$ 后，可以证明

$$\mathbb{E}[W] \geq p \left(\sum_j w_j - \sum_i v_i \right) \geq p \text{OPT}.$$

这部分的意义是展示：随机算法的概率分布可以根据公式结构调整，而不是只能用 $1/2$ 。

小例子：公式

$$(x_1) \wedge (x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_3)$$

没有负单位子句。取 $p \approx 0.618$ 。第一个子句满足概率为 p ；第二个子句不满足需 $x_2 = F$ 且 $x_3 = T$ ，概率 $(1-p)p$ ，满足概率 $1-p(1-p) > 1-p^2$ ；第三个子句不满足需 $x_1 = T$ 且 $x_3 = F$ ，概率 $p(1-p)$ 。所有子句满足概率都至少 $\min(p, 1-p^2) = p$ 。

23.5 LP 舍人与组合策略

23.5.1 LP 建模

MAX-SAT 的整数规划：变量 $y_i \in \{0, 1\}$ ($y_i = 1$ 表示 $x_i = T$)， $z_j \in \{0, 1\}$ ($z_j = 1$ 表示 C_j 被满足)。

约束（对子句 C_j ，正文字集 P_j ，负文字集 N_j ）：

$$\sum_{i \in P_j} y_i + \sum_{i \in N_j} (1 - y_i) \geq z_j.$$

目标: $\max \sum_j w_j z_j$, $y_i, z_j \in \{0, 1\}$ 。

LP 松弛: 允许 $y_i, z_j \in [0, 1]$ 。

LP 舍入: 以 y_i^* (LP 最优解) 作概率令 $x_i = \mathbb{T}$, 这相当于每个变量掷一枚不同偏置的硬币。

23.5.2 LP 舍入分析

设子句 C_j 长度为 ℓ_j , 正文字集合为 P_j , 负文字集合为 N_j 。在 LP 舍入下, C_j 不满足的概率为

$$\prod_{i \in P_j} (1 - y_i^*) \prod_{i \in N_j} y_i^*.$$

用算术-几何平均不等式,

$$\prod_{i \in P_j} (1 - y_i^*) \prod_{i \in N_j} y_i^* \leq \left(\frac{\sum_{i \in P_j} (1 - y_i^*) + \sum_{i \in N_j} y_i^*}{\ell_j} \right)^{\ell_j}.$$

LP 约束给出

$$\sum_{i \in P_j} y_i^* + \sum_{i \in N_j} (1 - y_i^*) \geq z_j^*.$$

由于

$$\sum_{i \in P_j} (1 - y_i^*) + \sum_{i \in N_j} y_i^* = \ell_j - \left(\sum_{i \in P_j} y_i^* + \sum_{i \in N_j} (1 - y_i^*) \right) \leq \ell_j - z_j^*,$$

所以

$$\Pr[C_j \text{ 不满足}] \leq \left(1 - \frac{z_j^*}{\ell_j} \right)^{\ell_j}.$$

进而

$$\Pr[C_j \text{ 满足}] \geq 1 - \left(1 - \frac{z_j^*}{\ell_j} \right)^{\ell_j} \geq \left(1 - \left(1 - \frac{1}{\ell_j} \right)^{\ell_j} \right) z_j^* \geq (1 - 1/e) z_j^*.$$

最后一个不等式使用 $(1 - 1/\ell_j)^{\ell_j} \leq 1/e$ 。因此

$$\mathbb{E}[W] \geq (1 - 1/e) \sum_j w_j z_j^* \geq (1 - 1/e) \text{OPT}.$$

LP 舍入例子: 对子句

$$C = (x_1 \vee \neg x_2 \vee x_3)$$

若 LP 解给出 $y_1^* = 0.4, y_2^* = 0.7, y_3^* = 0.2$, 则 LP 左端为 $0.4 + (1 - 0.7) + 0.2 = 0.9$, 所以可令 $z^* \leq 0.9$ 。舍入后该子句不满足概率为

$$(1 - 0.4) \cdot 0.7 \cdot (1 - 0.2) = 0.336,$$

满足概率为 0.664。而通用下界为

$$(1 - 1/e) z^* \approx 0.632 \cdot 0.9 = 0.569,$$

实际满足概率高于理论保证。

23.5.3 组合策略

- 随机赋值 ($1/2$ -近似) 对长子句好 (子句越长, 满足概率越接近 1)。
- LP 舍入 ($(1 - 1/e)$ -近似) 对短子句好 (尤其 $k = 1, 2$ 时 LP 约束紧)。

课件给出的组合算法是: 分别运行

- 算法 1: LP 随机舍入, 得到解值随机变量 W_1 ;
- 算法 2: 公平随机赋值, 得到解值随机变量 W_2 ;

然后选择两者中更好的解。由于

$$\max(W_1, W_2) \geq \frac{W_1 + W_2}{2},$$

所以

$$\mathbb{E}[\max(W_1, W_2)] \geq \frac{1}{2}\mathbb{E}[W_1] + \frac{1}{2}\mathbb{E}[W_2].$$

对子句 C_j , 两部分贡献的下界分别为

$$\Pr_{LP}[C_j \text{ 满足}] \geq \left(1 - \left(1 - \frac{1}{\ell_j}\right)^{\ell_j}\right) z_j^*$$

和

$$\Pr_{1/2}[C_j \text{ 满足}] = 1 - 2^{-\ell_j}.$$

因为 $z_j^* \leq 1$, 可把第二项写成至少 $(1 - 2^{-\ell_j})z_j^*$ 。于是每个子句在平均组合中的系数至少为

$$\frac{1}{2} \left(1 - \left(1 - \frac{1}{\ell_j}\right)^{\ell_j}\right) + \frac{1}{2}(1 - 2^{-\ell_j}).$$

对所有 $\ell_j \geq 1$, 该值至少 $3/4$; 例如 $\ell = 1$ 时为 $(1+1/2)/2 = 3/4$, $\ell = 2$ 时为 $(3/4+3/4)/2 = 3/4$, 更长子句只会更好或不低于该下界。因此

$$\mathbb{E}[\max(W_1, W_2)] \geq \frac{3}{4} \sum_j w_j z_j^* \geq \frac{3}{4} \text{OPT}.$$

定理 23.2. MAX-SAT 有 $3/4$ -近似算法 (通过随机赋值与 LP 舍入组合)。

核心记忆 MAX-SAT 是随机化方法的典型范式: (1) 用期望证明存在好解; (2) 用条件期望 (方法论) 去随机化, 构造确定性算法; (3) LP 舍入 + 随机组合可进一步提升近似比。去随机化的核心: 每步选期望不降的分支, 保持”已达到的下界不变”。

第二十四章 算法与数学分析详解

本章把前面各讲中最容易“看懂结论但不会自己推”的部分展开。阅读时建议抓住四个问题：

1. **状态是什么**？算法运行过程中维护哪些量。
2. **不变量是什么**？每一步之后哪些事实仍然成立。
3. **为什么停下时就是答案**？终止条件如何转化为最优性。
4. **每个对象被处理多少次**？复杂度往往来自对顶点、边、状态或增广次数的计数。

24.1 图搜索：BFS 与 DFS

24.1.1 BFS 的层次结构

BFS 的本质是按距离分层。设第 i 层

$$L_i = \{v : \text{dist}(s, v) = i\}.$$

算法用队列保证先处理 L_0 ，再处理 L_1 ，然后处理 L_2 ，依此类推。更精确地说，任意时刻队列中的顶点只可能来自相邻两层，且较小层的顶点排在前面。

不变量 当某顶点 u 出队时， $d[u] = \text{dist}(s, u)$ 。证明用归纳。初始 s 显然成立。若 u 是从 x 发现的，则存在路径长度 $d[x] + 1$ ，所以 $\text{dist}(s, u) \leq d[u]$ 。反过来，若存在更短路径

$$s \rightsquigarrow y \rightarrow u$$

长度小于 $d[u]$ ，则 y 的距离小于 $d[u] - 1$ ，按归纳会更早出队；处理 y 的边时应已发现 u ，矛盾。

复杂度推导 邻接表中，每个顶点最多入队一次、出队一次，所以顶点操作是 $O(|V|)$ 。每条边只在扫描其起点邻接表时被检查一次；无向图中会从两端各检查一次，仍是 $O(|E|)$ 。因此总时间为 $O(|V| + |E|)$ ，空间为队列、距离、父指针数组的 $O(|V|)$ 。

24.1.2 DFS 的括号性质

DFS 的 pre/post 时间具有括号结构：对任意两个顶点 u, v ，区间

$$[\text{pre}(u), \text{post}(u)] \quad \text{和} \quad [\text{pre}(v), \text{post}(v)]$$

要么不相交，要么一个包含另一个。包含关系正对应 DFS 树中的祖先关系。

边分类判定 在有向图中，考察边 (u, v) ：

- 若 v 尚未访问， (u, v) 成为树边。
- 若 v 已访问但未退出， v 是 u 的祖先， (u, v) 是回边。
- 若 v 已退出，依据时间区间可判定为前向边或横叉边。

有向图存在环当且仅当 DFS 出现回边。若存在回边 $u \rightarrow v$ ，树路径 $v \rightsquigarrow u$ 加上该边形成环；反过来，沿环中第一个被 DFS 访问的顶点出发，必会遇到指向尚在递归栈中祖先的边。

24.2 Dijkstra 与 Bellman–Ford

24.2.1 Dijkstra 的贪心证明

Dijkstra 的关键不变量是：集合 S 中所有顶点的距离已经确定，即

$$d[v] = \delta(s, v) \quad \forall v \in S.$$

每次选择 $V \setminus S$ 中 $d[u]$ 最小的顶点 u 加入 S 。设反证存在一条更短路径 P 到 u 。沿 P 从 s 出发，令 y 是第一个不在 S 中的顶点， x 是它前一个顶点，则 $x \in S$ 。当 x 加入 S 时已经松弛过边 (x, y) ，所以

$$d[y] \leq \delta(s, x) + w(x, y) = \delta(s, y).$$

又因为边权非负， P 上从 y 到 u 的后缀长度非负，故

$$\delta(s, y) \leq \delta(s, u) < d[u].$$

于是 $d[y] < d[u]$ ，这与 u 是未确定顶点中估计距离最小者矛盾。

为什么负边破坏证明 如果 y 到 u 的后缀可能为负，就不能推出 $\delta(s, y) \leq \delta(s, u)$ 。也就是说，一个暂时看起来更远的顶点，未来可能通过负边把 u 的距离拉低。

24.2.2 Bellman–Ford 的 DP 解释

定义

$D_k[v]$ = 从 s 到 v 使用至多 k 条边的最短路长度。

则

$$D_k[v] = \min \left\{ D_{k-1}[v], \min_{(u,v) \in E} D_{k-1}[u] + w(u, v) \right\}.$$

若使用两份数组同步更新，Bellman–Ford 的第 k 轮全边松弛正是在把 D_{k-1} 推进到 D_k 。实际实现常用一维数组原地更新；此时一轮内可能沿多条边连续传播，因此不能逐轮解释为“恰好只考虑至多 k 条边的路径”。但原地更新始终只会把距离估计降低到某条真实路径的长度，并且至少包含同步版本能得到的所有改进。所以在没有可达负环时，经过 $|V| - 1$ 轮后，所有简单最短路的改进都已传播完毕，算法仍然正确。

负环检测的数学原因 若没有从 s 可达的负环，则任意最短路可取简单路径。简单路径最多经过 $|V| - 1$ 条边，因此 $|V| - 1$ 轮后所有最短路都应稳定。若第 $|V|$ 轮仍可松弛某条边，说明存在一条更短的、包含至少 $|V|$ 条边的可达走法；其中必有重复顶点，形成环。若该环权重非负，删除它不会变差，因此能继续变短只能由负环解释。

24.3 最小生成树：安全边与交换论证

24.3.1 统一的贪心框架

MST 算法维护一个边集 A ，要求 A 总能扩展成某棵 MST。这样的 A 称为 promising。若某条边 e 加入 A 后仍 promising，则 e 是安全边。

割性质的完整证明 设 A 是某棵 MST T 的子集，割 $(S, V \setminus S)$ 不穿过 A ，即 A 中没有边跨越该割。令 e 是该割上的最轻边。若 $e \in T$ ，无需证明。若 $e \notin T$ ，则 $T + e$ 产生唯一环。这个环从 S 走到 $V \setminus S$ 后还必须走回来，所以存在另一条跨割边 $f \in T$ 。由于 e 是割上最轻边，

$$w(e) \leq w(f).$$

令

$$T' = T + e - f.$$

T' 仍连通且边数为 $|V| - 1$ ，所以是生成树，并且

$$w(T') = w(T) + w(e) - w(f) \leq w(T).$$

因此 T' 也是 MST 且包含 $A \cup \{e\}$ 。

24.3.2 Kruskal 的复杂度细算

Kruskal 的三部分工作：

- 排序 $|E|$ 条边： $O(|E| \log |E|)$ 。
- 每条边做两次 find，若跨分量再 union： $O(|E| \alpha(|V|))$ 。
- 输出 $|V| - 1$ 条边。

由于简单图中 $|E| \leq |V|^2$ ，所以 $\log |E| = O(\log |V|)$ ，常写为

$$O(|E| \log |V|).$$

若边权为小整数，也可用计数排序等方式降低排序成本。

24.3.3 Prim 与 Dijkstra 的相似和不同

二者都维护一个已确定集合并用优先队列取最小键。区别在键值含义：

- Dijkstra 的键是从源点到顶点的当前最短路径估计 $d[v]$ 。
- Prim 的键是连接顶点 v 到当前树 S 的最轻跨割边权。

Dijkstra 的松弛是

$$d[v] \leftarrow \min(d[v], d[u] + w(u, v)),$$

Prim 的松弛是

$$key[v] \leftarrow \min(key[v], w(u, v)).$$

一个累加路径长度，一个只看跨割边。

24.4 动态规划：从状态到递推

24.4.1 设计 DP 的四步

1. **定义状态**：状态必须包含足够信息，使未来决策不依赖更早历史。
2. **写出选择**：最后一步或第一步是什么。
3. **递推与边界**：把大问题拆成已定义的小问题。
4. **计算顺序**：保证用到的状态已经算好。

24.4.2 序列比对的网格图

序列比对可画成 $(m+1) \times (n+1)$ 网格。点 (i, j) 表示已经处理 x 的前 i 个字符和 y 的前 j 个字符。三种边：

- $(i-1, j-1) \rightarrow (i, j)$ ：匹配或替换 x_i, y_j 。
- $(i-1, j) \rightarrow (i, j)$ ：让 x_i 对空位。
- $(i, j-1) \rightarrow (i, j)$ ：让空位对 y_j 。

因此 DP 表就是这个 DAG 上从 $(0, 0)$ 到 (m, n) 的最短路。这个视角解释了为什么填表顺序可以按行、按列或按反对角线，只要拓扑顺序合法即可。

24.4.3 Hirschberg 的空间分析

普通 DP 若保存整张表，空间为 mn 。Hirschberg 只在每层分治保存两行 DP：

$$F[j] = \text{cost}(x_L, y_{1..j}), \quad B[j] = \text{cost}(x_R, y_{j+1..n}).$$

切分点为

$$k = \arg \min_j F[j] + B[j].$$

递归树每一层处理的子问题总面积不超过 mn 的常数倍，因为 x 的长度被对半分。于是时间满足

$$T(m, n) \leq cmn + T(m/2, k) + T(m/2, n - k) = O(mn),$$

而递归栈加每层两行数组空间为 $O(m + n)$ 。

24.4.4 Knapsack FPTAS 的误差推导

设 $V_{\max} = \max_i v_i$, 缩放因子 $K = \varepsilon V_{\max}/n$, 缩放价值

$$v'_i = \lfloor v_i/K \rfloor.$$

对任意物品 i ,

$$Kv'_i \leq v_i < K(v'_i + 1),$$

所以向下取整损失小于 K 。若 S^* 是最优物品集合, 最多有 n 个物品, 因此

$$\sum_{i \in S^*} v_i - K \sum_{i \in S^*} v'_i < nK = \varepsilon V_{\max} \leq \varepsilon \text{OPT}.$$

缩放 DP 找到缩放价值最优的可行集合 S , 故其真实价值至少为

$$\text{value}(S) \geq (1 - \varepsilon) \text{OPT}.$$

24.5 最大流：残量网络的含义

24.5.1 为什么需要反向边

增广路上的反向边表示“撤销之前的一部分流”。例如先把 1 单位流送过 (u, v) , 后来发现更优方案应把这 1 单位改走别的边, 那么残量网络中的反向边 (v, u) 容量为 1, 允许算法把该流量退回来。没有反向边, 贪心选择一旦做错就无法修正。

24.5.2 流值等于割净流

对任意割 (S, T) , 把 S 内所有顶点的流守恒方程相加, 内部边一正一负抵消, 得到

$$|f| = \sum_{u \in S, v \in T} f(u, v) - \sum_{u \in T, v \in S} f(u, v).$$

再由容量约束和非负性:

$$|f| \leq \sum_{u \in S, v \in T} c(u, v) = c(S, T).$$

这就是弱对偶形式: 任意流给下界, 任意割给上界。

24.5.3 Ford–Fulkerson 的终止与最优性

整数容量时, 每次增广量 Δ 是正整数, 所以流值至少增加 1。若最大流值为 F , 增广次数最多 F 次。每次 DFS/BFS 找路径花 $O(|E|)$, 故朴素界为 $O(F|E|)$ 。这不是输入长度的多项式界, 因为 F 可能很大。

终止时, 残量图中没有 s 到 t 的路径。令 S 为残量图中从 s 可达的顶点集合。对任意原图边 (u, v) :

- 若 $u \in S, v \in T$, 则残量容量为 0, 所以 $f(u, v) = c(u, v)$ 。
- 若 $u \in T, v \in S$, 则反向残量容量为 $f(u, v)$; 若 $f(u, v) > 0$, 则 u 会从 v 可达, 矛盾, 所以 $f(u, v) = 0$ 。

代入割净流公式得到 $|f| = c(S, T)$, 因此流和割同时最优。

24.5.4 Edmonds–Karp 的距离单调性

每次选最短增广路。设 $\ell_f(v)$ 是残量网络中从 s 到 v 的 BFS 距离。增广后所有可达顶点的距离不会下降。直观原因是：若某个顶点距离下降，取第一个距离下降的顶点 v ，它的新最短路前驱 u 的距离未下降，而边 (u, v) 在增广前若已存在就会导致旧距离也不大；若是新增反向边，则对应正向边刚在最短路被使用，会推出距离关系矛盾。

一条边 (u, v) 每次成为瓶颈后会从残量图消失。若未来再次出现并再次成为瓶颈，必须先通过反向边把流退回，这会使相关层数至少增加 2。由于顶点距离最多为 $|V| - 1$ ，每条边成为关键瓶颈 $O(|V|)$ 次，总增广次数 $O(|V||E|)$ 。

24.6 线性规划、对偶与单纯形

24.6.1 弱对偶逐行推导

原问题

$$\max c^T x \quad \text{s.t. } Ax \leq b, x \geq 0$$

对偶

$$\min b^T y \quad \text{s.t. } A^T y \geq c, y \geq 0.$$

对任意可行 x, y ,

$$c^T x \leq (A^T y)^T x = y^T Ax \leq y^T b = b^T y.$$

第一步用 $A^T y \geq c$ 且 $x \geq 0$ ，第二步用 $Ax \leq b$ 且 $y \geq 0$ 。这一串不等式是所有 LP 下界/上界证明的核心。

24.6.2 互补松弛的来源

若 x, y 都可行且目标值相等，则上面两处不等式都必须取等。于是：

$$\sum_j x_j ((A^T y)_j - c_j) = 0, \quad \sum_i y_i (b_i - (Ax)_i) = 0.$$

每一项都非负，所以每一项都为 0。这就得到互补松弛条件。它不是额外神秘定理，而是弱对偶等号成立时的逐项条件。

24.6.3 单纯形 pivot 的代数形式

把 LP 写成字典：

$$z = z_0 + \sum_{j \in N} \bar{c}_j x_j, \quad x_i = b_i - \sum_{j \in N} a_{ij} x_j \quad (i \in B).$$

若某个非基变量 x_p 的 reduced cost $\bar{c}_p > 0$ ，增加它会提高目标值。为了保持所有基变量非负，需要

$$x_p \leq \frac{b_i}{a_{ip}} \quad \text{对所有 } a_{ip} > 0.$$

最小比值对应离基变量。pivot 只是把进入变量解出来，替换到其他方程中。最优性条件是所有 $\bar{c}_j \leq 0$ ；无界条件是存在 $\bar{c}_p > 0$ 且所有 $a_{ip} \leq 0$ ，此时 x_p 可无限增大。

24.7 归约与复杂性证明

24.7.1 归约证明的双向逻辑

以 3SAT 到 Independent Set 为例。构造后要证明：

- 若公式可满足，则每个子句选一个为真的文字。子句内三角形保证不会同子句选两个；互补文字不可能同时为真，所以没有冲突边，得到大小 m 的独立集。
- 若图有大小 m 的独立集，由于每个子句三角形最多选一个，而总共选了 m 个，必然每个子句选了一个文字。互补边保证这些文字不矛盾，因此可扩展为满足赋值。

很多归约错误都出在只证明了一个方向。

24.7.2 NP、co-NP 与证书

证明 $L \in NP$ 不是给算法求解 L ，而是给验证器：

$$V(x, c) = 1 \iff c \text{ 是 } x \text{ 的合法 yes 证书.}$$

例如 Hamilton cycle 的证书是一串顶点排列，验证只需检查每个顶点恰好一次且相邻边存在。co-NP 则要求 no 实例有短证书。若一个最优化问题有强对偶定理，常常能同时给出 primal 与 dual 证书，解释它为什么可能落在 $NP \cap coNP$ 或 P 中。

24.8 近似算法的数学分析

24.8.1 Vertex Cover 的匹配下界

设算法选出的未覆盖边集合为 M 。由于每次选边后删除所有 incident 边， M 中任意两条边不共享端点，所以 M 是匹配。任意顶点覆盖必须为 M 中每条边至少选择一个端点，因此

$$OPT \geq |M|.$$

算法为每条匹配边选择两个端点，得到

$$ALG = 2|M| \leq 2OPT.$$

这里下界不是“最优解至少覆盖一半边”之类的模糊说法，而是“匹配中的边两两独立，每条都要付出一个不同顶点”。

24.8.2 Set Cover 贪心的 charging 证明

当贪心选择集合 S 时，设它新覆盖了 r 个元素，成本为 $w(S)$ ，给每个新覆盖元素收费 $w(S)/r$ 。算法总成本等于所有元素收费之和。

取最优解中的任意集合 O ，设它包含 q 个元素，按这些元素被贪心覆盖的先后顺序排列为 $e_1, \dots, e_q$ 。在覆盖 e_i 前，集合 O 至少还能覆盖 $q - i + 1$ 个尚未覆盖的这些元素，因此贪心所选集合的单位成本不超过

$$\frac{w(O)}{q - i + 1}.$$

所以 O 内元素总收费不超过

$$w(O) \left(1 + \frac{1}{2} + \cdots + \frac{1}{q} \right) \leq H_n w(O).$$

对最优解中所有集合求和，得到

$$ALG \leq H_n \text{OPT}.$$

24.8.3 LP 舍人的基本不等式

最小化问题中，LP 松弛可行域包含整数可行域，所以

$$LP^* \leq \text{OPT}.$$

若舍入算法能保证

$$\text{cost}(\text{round}(x)) \leq \alpha \cdot \text{cost}(x)$$

对最优 LP 解 x 成立，则

$$ALG \leq \alpha LP^* \leq \alpha \text{OPT}.$$

因此 LP 近似的全部压力集中在第二个不等式：如何舍入且只损失 α 倍。

24.8.4 MAX-SAT 的期望线性性

令随机变量 X_j 表示子句 C_j 是否被满足，满足则为 1，否则为 0。总满足权重

$$X = \sum_{j=1}^m w_j X_j.$$

即使子句之间不独立，期望线性性仍给出

$$\mathbb{E}[X] = \sum_{j=1}^m w_j \mathbb{E}[X_j].$$

长度为 k_j 的子句只有当所有文字都为假时才不满足，公平随机赋值下概率为 2^{-k_j} ，故

$$\mathbb{E}[X] = \sum_j w_j (1 - 2^{-k_j}) \geq \frac{1}{2} \sum_j w_j \geq \frac{1}{2} \text{OPT}.$$

条件期望去随机化依赖的事实是：

$$\mathbb{E}[X \mid \text{当前部分赋值}] = \frac{1}{2} \mathbb{E}[X \mid x_i = \text{true}] + \frac{1}{2} \mathbb{E}[X \mid x_i = \text{false}].$$

两项中至少有一项不小于当前条件期望，选择它即可保持最终保证。

24.9 如何自己分析一个新算法

遇到新算法时，可按下面的清单写分析：

1. **输入规模**：明确用 $n, m, |V|, |E|$ 还是数值大小 U, W 衡量。
 2. **基本操作**：一次松弛、一次 pivot、一次 BFS、一次 DP 状态转移分别多贵。
 3. **次数上界**：每条边扫描几次，每个状态计算几次，每轮消除多少可能性。
 4. **正确性句柄**：是循环不变量、交换论证、最优子结构、对偶证书还是概率期望。
 5. **终止条件**：停下时缺少什么结构，如无增广路、无正 reduced cost、无未覆盖元素。
 6. **反例边界**：算法依赖哪些条件，如非负边权、三角不等式、整数容量、DAG 顺序。
- 能把这六项写清楚，通常就能把一段算法分析写到考试可得分的细度。

总复习提纲

24.10 算法范式索引

范式	本课程中的代表内容
分治	归并排序、Hirschberg 线性空间序列比对
贪心	Dijkstra、MST、集合覆盖贪心、TSP double-tree
动态规划	序列比对、LCS、Bellman–Ford、树宽 DP、Knapsack
图搜索	BFS、DFS、SCC、残量网络搜索、模型检测可达性
线性规划	LP 对偶、最短路/最大流 LP、LP 松弛、舍入
归约	SAT、3SAT、独立集、顶点覆盖、Hamilton、TSP、Subset Sum
随机化	MAX-SAT、随机舍入、条件期望去随机化
复杂性分类	P、NP、co-NP、NP-hard、NP-complete、PSPACE

24.11 常见证明模板

- **贪心正确性**：找安全选择，使用交换论证或 cut/cycle 性质。
- **动态规划正确性**：定义子问题，证明最优子结构，给出计算顺序。
- **最大流最优性**：用“无增广路 $\Rightarrow$ 构造等值割”。
- **LP 对偶**：弱对偶给界，强对偶给等值，互补松弛验证最优。
- **NP-complete**：先证属于 NP，再从已知 NP-complete 问题归约。
- **近似比**：算法值与一个可证明的下界/上界比较，如匹配、LP、对偶或最优解平均性质。

24.12 考试前快速检查

1. 能否写出 BFS、DFS、Dijkstra、Bellman–Ford、Ford–Fulkerson、Kruskal 的伪代码和复杂度？
2. 能否用一句话说明每个贪心算法为什么安全？
3. 能否从整数规划写出 LP 松弛和对偶？

4. 看到新问题时, 能否判断更像最短路、流、匹配、覆盖、SAT、还是 DP?
5. 证明困难性时, 归约方向是否正确?
6. 证明近似比时, 使用的 OPT 下界是否真的成立?